VOLUME 2

Concurrency & JVM Internals

Java Memory Model, java.util.concurrent, virtual threads, class loading, GC, performance troubleshooting

Java 21 · Spring Boot 3 · SQL · Microservices · AI integration
Interview preparation for 12+ years experience · real scenarios from the RefinePoint platform

Contents

8. Threads & the Java Memory Model

8.1 Process vs thread
8.2 Thread lifecycle
8.3 Creating threads
8.4 The interrupt mechanism
8.5 `synchronized` internals
8.6 wait, notify and notifyAll
8.7 The Java Memory Model
8.8 `volatile` internals
8.9 `final` fields and safe publication
8.10 Race conditions and thread-safety strategies
8.11 Deadlock, livelock and starvation
8.12 ThreadLocal internals and leaks
8.13 Reading a thread dump
8.14 Classic coding questions
8.15 Interview questions and answers

9. java.util.concurrent in Depth

9.1 Why `java.util.concurrent` exists
9.2 `Lock` vs `synchronized` and `ReentrantLock`
9.3 AbstractQueuedSynchronizer (AQS) internals
9.4 `ReadWriteLock` and `StampedLock`
9.5 Atomics and compare-and-swap
9.6 The Executor framework
9.7 `Future` and `CompletableFuture`
9.8 `ForkJoinPool` and work stealing
9.9 Synchronizers
9.10 Graceful shutdown
9.11 Spring `@Async`, `@Scheduled` and their executors
9.12 Real scenarios from RefinePoint
9.13 Interview questions and answers

10. Virtual Threads, Structured Concurrency & Scoped Values (Java 21)

10.1 The problem: thread-per-request hits a wall
10.2 Platform vs virtual threads
10.3 How they work: continuations and carriers
10.4 Pinning
10.5 When virtual threads help, and when they do not
10.6 The JDBC connection pool is the real bottleneck
10.7 `ThreadLocal` with virtual threads
10.8 Spring Boot: `spring.threads.virtual.enabled=true`
10.9 Virtual threads vs reactive
10.10 Structured concurrency

10.11 Scoped values vs `ThreadLocal`
10.12 Migrating and proving it worked
10.13 Debugging and thread dumps
10.14 Interview questions and answers

11. JVM Architecture: Class Loading, Memory & Execution

11.1 The big picture
11.2 Class loading
11.3 Runtime data areas
11.4 Object layout
11.5 Bytecode and the `invoke*` instructions
11.6 The execution engine
11.7 Startup: AppCDS, AOT and native image
11.8 Errors: `StackOverflowError` and the `OutOfMemoryError` family
11.9 JVM flags you should know
11.10 Interview questions and answers

12. Garbage Collection & Performance Troubleshooting

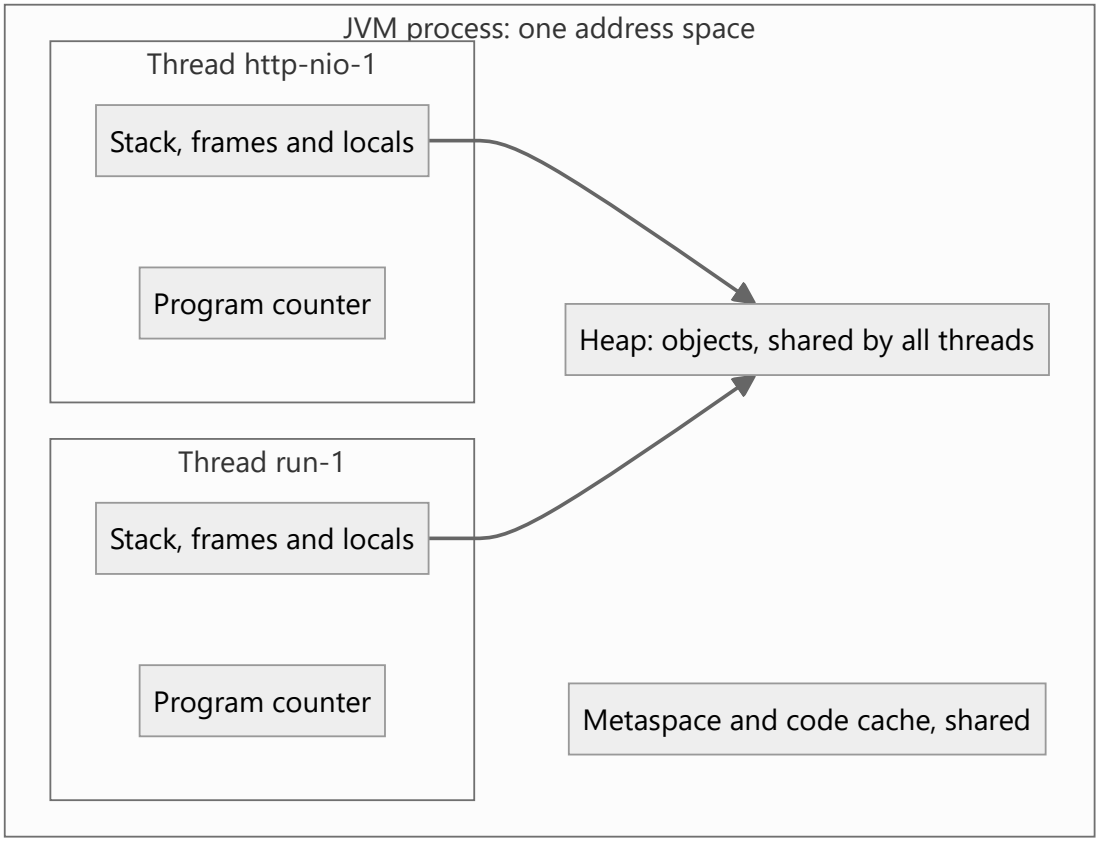
12.1 What garbage is: reachability and GC roots
12.2 The generational hypothesis and the young/old heap
12.3 The collectors
12.4 Tuning flags and GC logs
12.5 Reference types
12.6 Memory leaks in Java
12.7 Thread dumps, JFR and profilers
12.8 The JVM in containers
12.9 The production incident playbook
12.10 Interview questions and answers

8. Threads & the Java Memory Model

Concurrency is where senior interviews separate people who have used `synchronized` from people who can explain why a program with no bug on their laptop fails on a 32-core server. The questions look basic ("what does `volatile` do?") and the follow-ups go straight to the hardware: store buffers, memory barriers, the mark word, happens-before. This chapter builds the model from the bottom: what a thread is, how a monitor works, what the JVM actually guarantees about memory, and how to diagnose the failures. `java.util.concurrent` is in chapter 9 and virtual threads in chapter 10.

8.1 Process vs thread

A **process** is a running program with its own address space. A **thread** is an execution path inside a process: it shares the process's heap and code, and owns its stack and program counter.

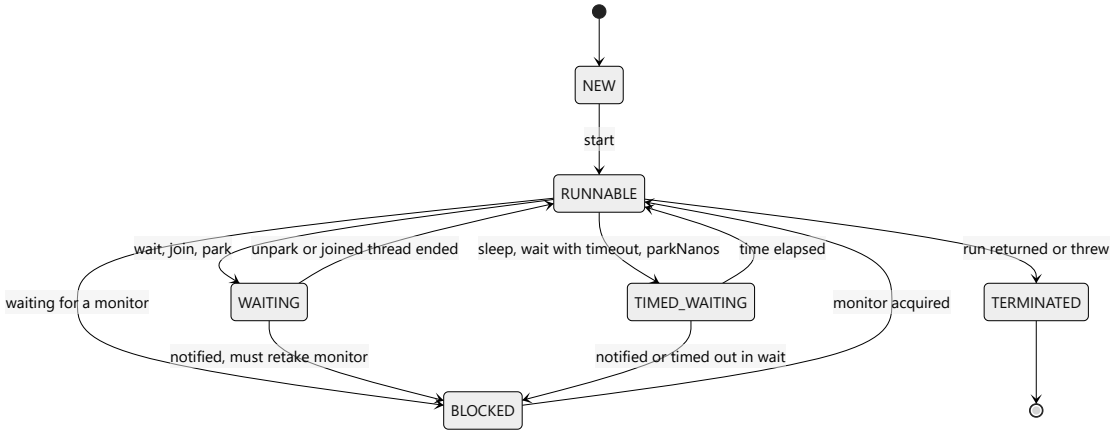


	Process	Thread
Memory	Own address space, isolated	Shared heap, own stack
Creation cost	High (fork, page tables)	Lower, still an OS call and a stack reservation
Context switch	Expensive, flushes address translation state	Cheaper, same address space
Communication	IPC: sockets, pipes, shared memory	Direct: shared objects
Failure	Isolated	An uncaught `Error` such as OOM can harm every thread
Java	One JVM per process	Platform thread = **one OS thread** (1:1 model)

A Java **platform thread** maps 1:1 to an OS thread, so it reserves a stack (default 1 MB on 64-bit Linux, `-Xss`) and the OS schedules it. That is why thousands of them get expensive, which is the motivation for virtual threads. Local variables and parameters live on the stack and are thread-confined; the objects they point to live on the shared heap, and every concurrency bug in this chapter comes from that sharing.

8.2 Thread lifecycle

`Thread.getState()` returns one of six JVM states.



State	Entered by	Notes
`NEW`	`new Thread(r)`	Not started yet
`RUNNABLE`	`start()`	Includes **running**, **ready for a core**, and **blocked in native socket I/O**
`BLOCKED`	Entering a `synchronized` block held by another thread, or re-entering after `wait()`	Only monitors put a thread here
`WAITING`	`Object.wait()`, `Thread.join()`, `LockSupport.park()`	Needs another thread to wake it
`TIMED_WAITING`	`Thread.sleep(n)`, `wait(n)`, `join(n)`, `parkNanos`, `parkUntil`	Wakes on its own too
`TERMINATED`	`run()` finished	Cannot be restarted

Two points that decide thread-dump reading. A thread blocked reading from a socket shows `RUNNABLE`, because the JVM cannot see OS-level blocking, so "all threads RUNNABLE" does not mean busy CPUs. A thread waiting on a `ReentrantLock` shows `WAITING (parking)`, not `BLOCKED`, because `java.util.concurrent` locks are built on `LockSupport.park`. Java has no `RUNNING` state.

8.3 Creating threads

```
// 1. Subclass Thread: rarely right, it couples the task to the mechanism
class Worker extends Thread { @Override public void run() { work(); } }

// 2. Runnable: a task with no result
new Thread(() -> work(), "ingest-1").start();

// 3. Callable + FutureTask: a result and checked exceptions
FutureTask<Integer> task = new FutureTask<>(() -> countRows(file));
new Thread(task).start();
int rows = task.get();

// 4. An executor: how production code does it (chapter 9)
ExecutorService pool = Executors.newFixedThreadPool(4);
Future<Integer> f = pool.submit(() -> countRows(file));

// 5. Java 21 builders
Thread t = Thread.ofPlatform().name("refresh-", 0).daemon().start(task);
Thread v = Thread.ofVirtual().name("io-", 0).start(task);
```

	`Runnable`	`Callable<V>`
Method	`void run()`	`V call() throws Exception`
Result	None	Returned through a `Future`
Checked exceptions	Must be handled inside	Allowed; surface as `ExecutionException`
Accepted by	`Thread`, `Executor.execute`, `submit`	`submit`, `invokeAll`, `FutureTask`

`start()` vs `run()`

`start()` asks the JVM to create an OS thread, which then calls `run()`. Calling `run()` directly is a plain method call on the current thread: no new thread and no concurrency, and nothing warns you. Calling `start()` twice throws `IllegalThreadStateException`.

Daemon threads and priorities

The JVM exits when only **daemon** threads remain, abandoning them mid-work without running their `finally` blocks. A thread inherits daemon status from its creator; `setDaemon` must be called before `start()`. GC threads are daemons, and all virtual threads are daemons.

> **RefinePoint:** every background worker pool sets `t.setDaemon(true)` in its thread factory (`RunWorker`, `RefreshWorker`, `ExportWorker`, `JobScheduler`) and names its threads (`run-1`, `refresh-1`), so a thread dump reads like a list of what the service is doing. Because daemons die abruptly, the claim row's status and heartbeat are in PostgreSQL: a crashed or killed worker leaves a row that can be recovered, and nothing important relies on `finally` running at JVM exit. `@PreDestroy` still calls `pool.shutdownNow()` for orderly stops.

Priorities run 1 to 10 (default 5), are inherited, and are only a hint. On Linux, HotSpot ignores them by default. Never use priorities to get correctness.

Comparing sleep, wait, yield and join

	`Thread.sleep(n)`	`obj.wait()`	`Thread.yield()`	`t.join()`
Defined on	`Thread` (static)	`Object`	`Thread` (static)	`Thread`
Needs the monitor	No	**Yes**, else `IllegalMonitorStateException`	No	No
Releases locks held	**No**, keeps them all	Releases **that** monitor only	No	Releases only the target thread's monitor (for platform threads)
Woken by	Timeout, interrupt	`notify`, `notifyAll`, timeout, interrupt, spuriously	Scheduler, immediately	Thread ending, timeout, interrupt
Purpose	Pause	Wait for a condition	Hint: others may run	Wait for a thread to finish

8.4 The interrupt mechanism

Java has no safe way to kill a thread (`Thread.stop` was deprecated for good reason and has thrown `UnsupportedOperationException` since Java 20). **Interruption is cooperative**: `t.interrupt()` sets a flag on `t`, and `t` decides what to do.

Target thread is...	Effect of `interrupt()`
In `sleep`, `wait`, `join`, `BlockingQueue` ops, `lockInterruptibly`	Throws `InterruptedException`, **clears the flag**
In `LockSupport.park`	Returns silently, flag stays set
In NIO `InterruptibleChannel` I/O	Channel is closed, `ClosedByInterruptException`, flag set
In classic `java.net.Socket` or `FileInputStream` I/O	**Nothing**; close the resource instead
Waiting to enter `synchronized`, or in `lock.lock()`	Nothing; stays blocked, flag set
Running CPU code	Just sets the flag; the code must check

```
public void run() {
    while (!Thread.currentThread().isInterrupted()) {
        try {
            Task task = queue.take();
            process(task);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();     // restore the flag, the exception cleared it
            break;                                  // and actually stop
        }
    }
}
```

`Thread.interrupted()` is static, tests the **current** thread and **clears** the flag; `t.isInterrupted()` tests `t` and leaves it. The two rules for `InterruptedException`: never swallow it, and if you cannot rethrow it, restore the flag so code up the stack can see it.

> **RefinePoint:** `RefreshWorker` stops with both mechanisms: a `volatile boolean stopping` flag so the claim loop stops taking new rows, and `pool.shutdownNow()`, which interrupts workers. `JobScheduler` uses interruption for job cancellation and restores the flag with `Thread.currentThread().interrupt()` where it catches the exception. A worker that swallowed the interrupt would keep running a cancelled job.

8.5 `synchronized` internals

Every Java object can act as a lock, because every object has a header and can be associated with a **monitor**: a mutual-exclusion lock plus a wait set.

Bytecode

```
void inc() { synchronized (lock) { count++; } }
```

```
 6: monitorenter
 7: aload_0              // body: count++
    ...
14: putfield count
17: aload_1
18: monitorexit
19: goto 27
22: astore_2             // compiler-generated handler
23: aload_1
24: monitorexit          // release even when the body threw
25: aload_2
26: athrow
27: return
Exception table: from 7 to 19 -> 22 any ; from 22 to 25 -> 22 any
```

That is why a `synchronized` block always releases on exception. A `synchronized` **method** has no such instructions, only the `ACC_SYNCHRONIZED` flag; the JVM acquires the monitor of `this` (or of the `Class` object for `static`) on entry. Locks are **reentrant**: the owning thread re-enters and a counter tracks depth, so a synchronized method can call another on the same object.

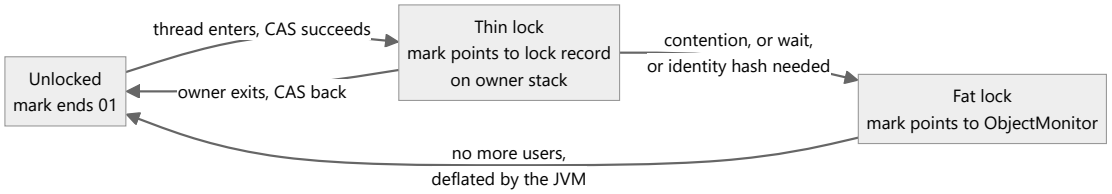
The object header and mark word

A 64-bit HotSpot object starts with a mark word and a class pointer. The low bits of the mark word say what the rest means:

```
Object layout (compressed class pointers):
+------------------+----------------+------------------+---------+
| Mark word 8 bytes| Klass ptr 4 B  | instance fields  | padding |
+------------------+----------------+------------------+---------+

Mark word, 64 bits:
State        | Contents                                          | Low bits
-------------+---------------------------------------------------+---------
Unlocked     | unused:25  hash:31  unused:1  age:4  unused:1     |   01
Thin locked  | pointer to lock record on owner's stack:62        |   00
Fat locked   | pointer to ObjectMonitor:62                       |   10
GC marked    | forwarding pointer:62                             |   11
```

Lock states

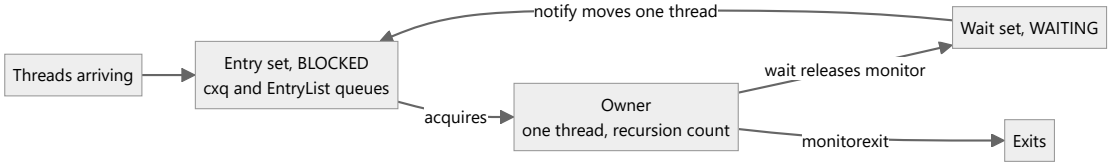


1. **Thin (lightweight) lock.** On entry the thread copies the mark word into a *lock record* in its stack frame and tries a CAS to replace the mark word with a pointer to that record. An uncontended lock therefore costs one CAS and no OS call. Re-entry pushes a lock record with a null copy.
2. **Fat (heavyweight) lock.** On contention, `wait()`, or a hash request while thin-locked, the lock **inflates**: the JVM allocates an `ObjectMonitor` and points the mark word at it. Competing threads spin briefly (adaptive spinning, since holds are usually short), then park through the OS.
3. **Biased locking** is gone. It let one thread lock an object repeatedly with no CAS at all by writing its thread ID into the mark word. It helped old `Vector` / `Hashtable`-style code, but revocation needed safepoints and the code was complex. Disabled and deprecated in Java 15 (JEP 374), removed in Java 18. Older material describing "biased → thin → fat" is out of date for Java 21.

Java 21 also adds an alternative thin-lock implementation (`-XX:LockingMode=2`, using a per-thread lock stack rather than a displaced header), which became the default in Java 23. The concept stays the same: fast CAS when uncontended, inflate when not.

The JIT optimises further: **lock elision** removes a lock on an object that escape analysis proves is thread-local, and **lock coarsening** merges adjacent lock/unlock pairs on the same object.

The monitor



`synchronized` is unfair by design: a running thread can barge past queued ones, which gives better throughput and a risk of starvation.

8.6 wait, notify and notifyAll

`wait`, `notify` and `notifyAll` are on `Object` because the monitor belongs to the object. They must be called while holding that monitor, or they throw `IllegalMonitorStateException`. The check prevents the **lost wake-up**: testing the condition and going to sleep must be atomic with respect to the thread that changes the condition.

`wait()` atomically releases the monitor and joins the wait set. `notify()` moves one arbitrary waiter to the entry set, and the woken thread continues only after it re-acquires the monitor, once the notifier has left the block.

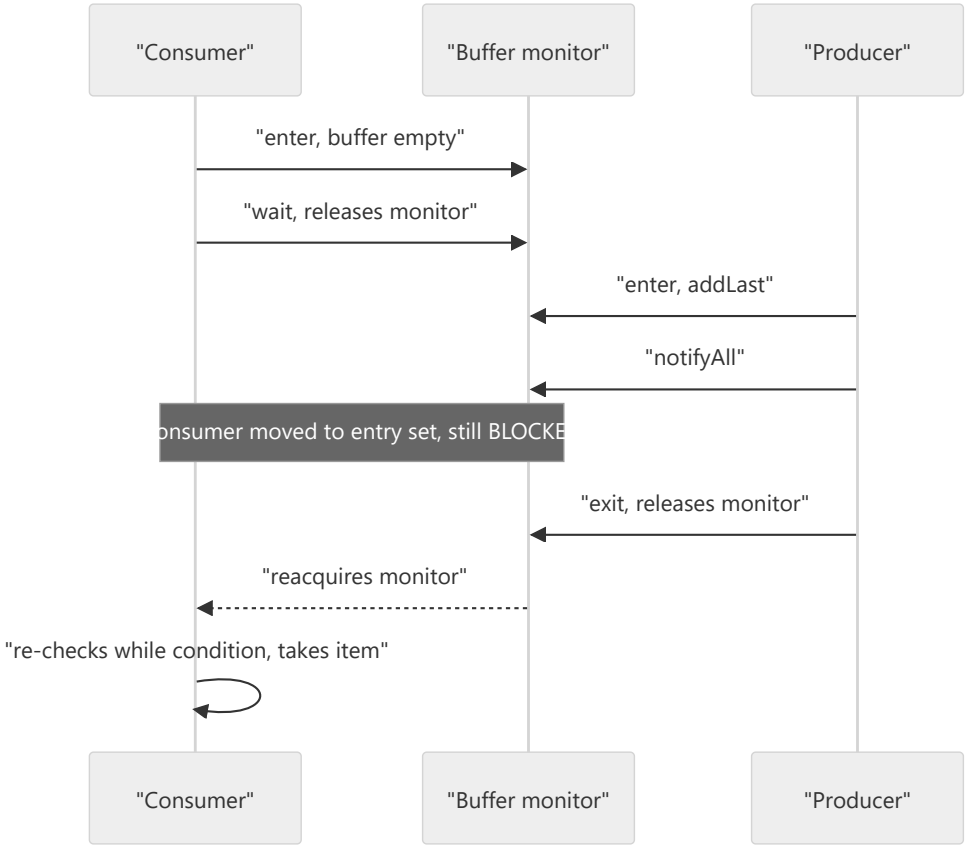
Producer-consumer

```
public final class BoundedBuffer<T> {
    private final Deque<T> items = new ArrayDeque<>();
    private final int capacity;

    public BoundedBuffer(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (items.size() == capacity) wait();     // ALWAYS a loop
        items.addLast(item);
        notifyAll();
    }

    public synchronized T take() throws InterruptedException {
        while (items.isEmpty()) wait();
        T item = items.removeFirst();
        notifyAll();
        return item;
    }
}
```



Why `while`, never `if`:

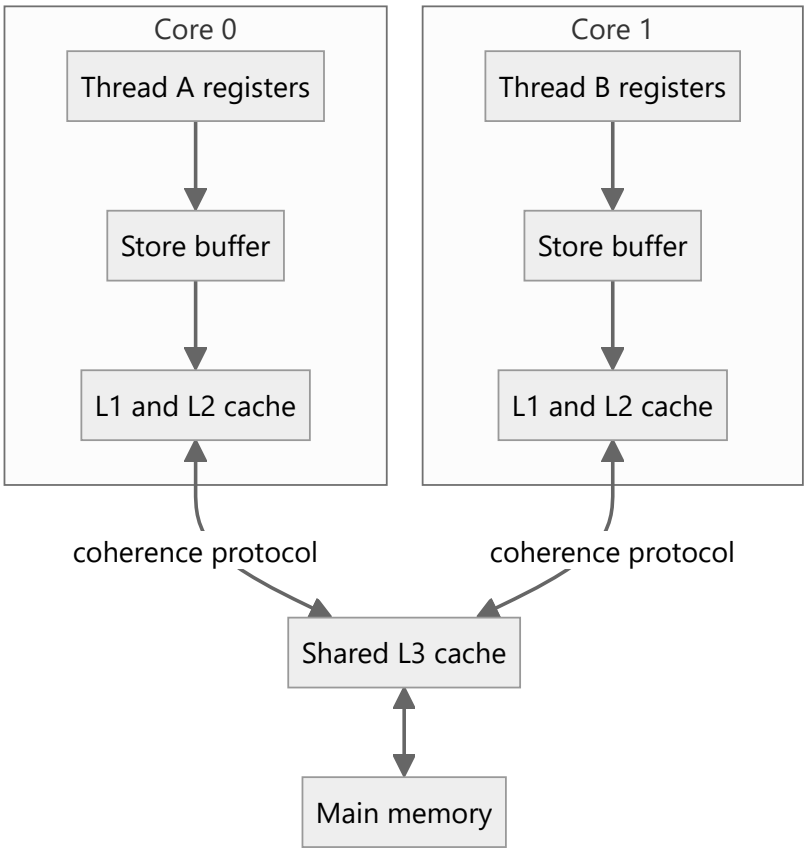
1. **Spurious wakeups.** The specification allows `wait()` to return with no notify, since the underlying OS primitives may behave this way.
2. **Stolen conditions.** Between `notify` and re-acquiring the monitor, another thread can take the item that made the condition true.
3. **Different conditions on one monitor.** With a single wait set, producers and consumers both wait there, so a wake-up may be for the other kind.

`notify` vs `notifyAll`: `notify` can wake a thread of the wrong kind, for example a producer woken by a producer when the buffer is full, and everyone then sleeps forever. Use `notify` only when all waiters are interchangeable and one can make progress; otherwise `notifyAll`. Its cost is a thundering herd, and the better fix is two `Condition`s on a `ReentrantLock`, shown in section 8.14. In production use `ArrayBlockingQueue`, but interviewers still ask for this by hand.

8.7 The Java Memory Model

Your source code is not what the CPU runs, and writes by one core are not immediately visible to others. Three layers reorder or delay operations:

1. **The compiler and JIT** reorder and eliminate reads and writes when a single thread cannot tell the difference, for instance hoisting a field read out of a loop.
2. **The CPU** executes out of order.
3. **The memory hierarchy.** Writes sit in a core's **store buffer** before reaching cache. Caches are kept coherent by a protocol such as MESI, so "each core has a stale cached copy forever" is a simplification; the observable delays come largely from store buffers, invalidation queues and compiler optimisation.



The JMM (JLS chapter 17, JSR-133, Java 5) is the contract above all of that: it defines which values a read is **allowed** to see, on any hardware.

The classic reordering surprise

```
int a = 0, b = 0, r1, r2;     // shared, not volatile
// Thread 1                   // Thread 2
a = 1;                        b = 1;
r1 = b;                       r2 = a;
```

Can `r1 == 0 && r2 == 0`? By reading the code it looks impossible, since something had to run first. It does happen, including on x86: each core's write is still in its store buffer when it reads the other variable. This is store-load reordering, the one kind that x86's strong hardware model allows.

Three properties

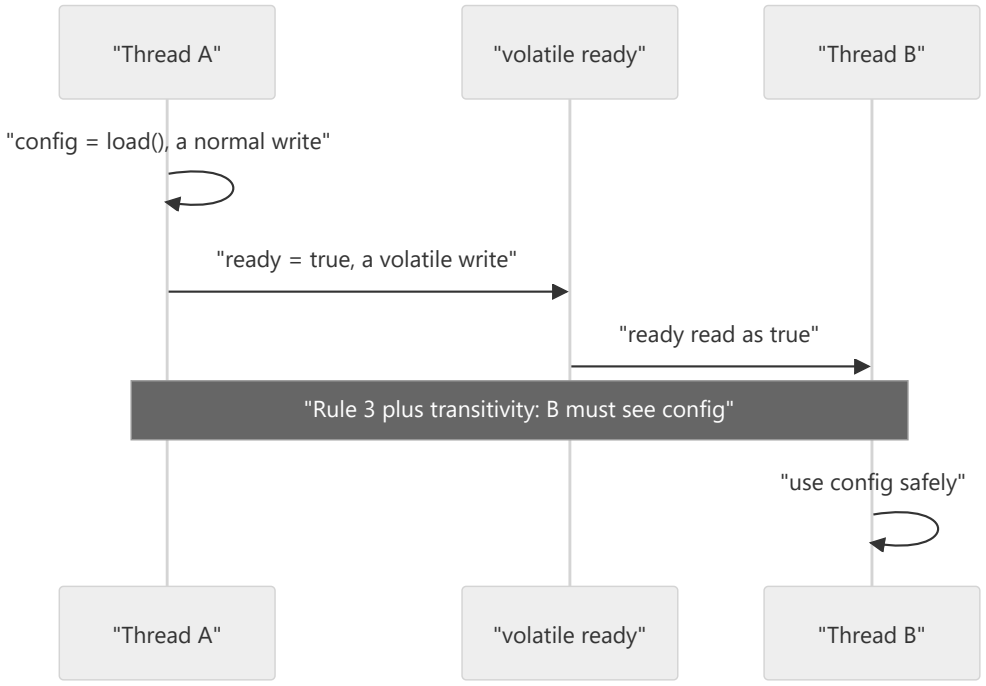
Property	Question	Provided by
Atomicity	Can another thread see a half-done operation?	`synchronized`, locks, atomics. Not `volatile` for compound actions
Visibility	Will another thread see my write?	`synchronized`, `volatile`, atomics, `final`
Ordering	Can actions appear to happen out of order?	`synchronized`, `volatile`, atomics

Happens-before

If action A **happens-before** B, then B is guaranteed to see A's effects (and everything before A). If two conflicting accesses, at least one a write, are not ordered by happens-before, there is a **data race** and the JMM gives very weak guarantees. The rules:

#	Rule	Meaning
1	Program order	Within a thread, each action happens-before later ones
2	Monitor lock	Unlock happens-before every later lock of the **same** monitor
3	Volatile	Write happens-before every later read of the **same** variable
4	Thread start	`t.start()` happens-before anything in `t`
5	Thread termination	Anything in `t` happens-before another thread seeing `t.join()` return or `isAlive()` false
6	Interruption	`t.interrupt()` happens-before `t` detecting it
7	Finalizer	End of constructor happens-before the start of the finalizer
8	Transitivity	A before B and B before C gives A before C

`java.util.concurrent` adds more: `Executor.submit` happens-before the task runs, a task's completion happens-before `Future.get` returns, a `BlockingQueue.put` happens-before the matching `take`, `CountDownLatch.countDown` before `await` returns, and so on.



Happens-before is not the same as "earlier in wall-clock time". One action can be earlier in time with no HB edge, and then the later read may legally miss it. Rule 3 also shows **piggybacking**: a single volatile write publishes every normal write made before it.

DRF-SC: a program with no data races behaves as if sequentially consistent. That is the practical takeaway: remove data races, and you may reason about your program as if statements interleave in source order.

Even a racy program has some floors: reads never see "out of thin air" values, and reads and writes of references and primitives are atomic, except non-volatile `long` and `double`, which the JLS permits to be split into two 32-bit halves (64-bit JVMs do not tear in practice).

8.8 `volatile` internals

`volatile` gives a field **visibility** and **ordering**, and makes its individual reads and writes atomic. It does **not** make compound actions atomic.

```
private volatile int count;
count++;      // still three steps: read, add, write. Lost updates remain. Use
AtomicInteger/LongAdder.
```

Internally the JIT must not reorder across the access or cache the field in a register, and it inserts **memory barriers**:

```
Volatile WRITE                  Volatile READ
  StoreStore barrier              load value
  store value                   LoadLoad barrier
  StoreLoad barrier             LoadStore barrier
```

Barrier	Guarantee
`LoadLoad`	Earlier loads complete before later loads
`StoreStore`	Earlier stores become visible before later stores
`LoadStore`	Earlier loads complete before later stores
`StoreLoad`	Earlier stores are visible before later loads, the expensive one that drains the store buffer

On x86 most of these are no-ops in the hardware: a volatile read is a plain `mov`, and a volatile write is a `mov` followed by a locked instruction (HotSpot typically emits `lock addl $0, (%rsp)`) for StoreLoad. So volatile reads are cheap and writes cost tens of cycles. On ARM the JVM uses acquire and release instructions (`ldar`, `stlr`). Hence a program can appear correct on x86 and break on ARM, although the JIT's own reordering already breaks racy code on x86.

`volatile` is enough when: writes do not depend on the current value (or a single thread writes), and the field is not part of an invariant with other fields. Typical uses are a stop flag, publishing an immutable snapshot, and double-checked locking. **Not enough:** counters, check-then-act, `if (x < max) x++`. Also, `volatile int[] a` makes only the reference volatile, not the elements; use `AtomicIntegerArray`.

Why a stop flag needs volatile

```
private boolean stop;                              // not volatile
void run() { while (!stop) { doWork(); } }         // may never stop
```

Nothing in the loop writes `stop`, so if `doWork` is inlined and proved not to touch it, the JIT may transform this into `if (!stop) while (true) doWork();`. It runs fine for a while in interpreted mode and hangs after JIT compilation, which is why such bugs appear only under load.

RefinePoint: `RunWorker`, `RefreshWorker` and `ExportWorker` each have `private volatile boolean stopping;`. `@PreDestroy` writes it from the shutdown thread and the claim loop reads it from another. `PaddleWebhookSources` uses the other classic: `private volatile List<int[]> cidrs`, so a refresh replaces the whole list with one write and readers always see a complete snapshot, no lock on the read path.

Double-checked locking

```
public final class Registry {
    private static volatile Registry instance;

    public static Registry get() {
        Registry local = instance;                // one volatile read on the fast path
        if (local == null) {
            synchronized (Registry.class) {
                local = instance;
                if (local == null) {
                    local = new Registry();
                    instance = local;
                }
            }
        }
        return local;
    }
}
```

Why `volatile` is needed. `new Registry()` is not one step: allocate, run the constructor, assign the reference. Without a barrier the assignment can be reordered before the constructor's writes, so a second thread at the first `if`, which takes no lock, may see a non-null reference to a half-built object. `volatile` forbids that reordering.

Usually a simpler pattern is better. The **holder idiom** is lazy and thread-safe through the JVM's class-initialisation lock:

```
public final class Registry {
    private Registry() {}
    private static final class Holder { static final Registry INSTANCE = new Registry(); }
    public static Registry get() { return Holder.INSTANCE; }
}
```

An `enum` singleton gives the same with serialization safety. In Spring, none of this is needed: a singleton bean is created once by the container.

RefinePoint: the gateway's `StatusController.current()` is a real double-checked pattern with a TTL: read `cached` (volatile), return it if fresh, else enter `synchronized (this)`, re-check, then write `cachedAt` and `cached`. The order of those writes matters. `cached` is written last and read first, so a reader who sees the new value is guaranteed by happens-before to see the matching `cachedAt`. It keeps a status page from turning into a stampede of health checks.

8.9 `final` fields and safe publication

Final field semantics

When a constructor finishes, its `final` fields are **frozen**. Any thread that obtains the reference afterwards sees the correct values of those fields, and of objects reachable through them as of the freeze, **even if the reference was published through a data race**. The JIT implements this with a StoreStore barrier at the end of the constructor. Normal fields have no such guarantee: under racy publication another thread may see defaults.

The condition is that `this` must not escape during construction:

```
public class Listener {
    private final Config config;
    public Listener(EventBus bus, Config config) {
        bus.register(this);       // BAD: another thread can call this object before config is
set
        this.config = config;
    }
}
```

Fix: a static factory that constructs fully, then registers. This is the guarantee that makes `String` safe to share with no synchronisation.

Safe publication

An object is **safely published** when its reference and its state become visible together. Four standard ways:

1. A **static initializer** (`static final Foo F = new Foo();`), covered by the class-init lock.
2. A `volatile` **field** or `AtomicReference` .
3. A `final` **field** of a properly constructed object.
4. A field **guarded by a lock**, which includes putting it in a `ConcurrentHashMap` , `BlockingQueue` or other thread-safe collection.

Immutable objects with all-final fields can be shared even without safe publication. Effectively immutable objects, never changed after publication, need safe publication and then no further synchronisation. Mutable objects need safe publication and also a lock or thread-safety for every access.

8.10 Race conditions and thread-safety strategies

A **race condition** means correctness depends on timing. A **data race** is a JMM term for unsynchronised conflicting access. They overlap but differ: you can have a race condition with no data race (each step synchronised, the sequence not), and a data race with no observable failure.

```
// Check-then-act: both threads can see absent, both create
if (!map.containsKey(k)) map.put(k, create());
map.computeIfAbsent(k, x -> create());          // atomic on ConcurrentHashMap

// Read-modify-write: both read 5, both write 6
count++;
counter.incrementAndGet();
```

```
// Compound on a thread-safe list is still a race
if (!syncList.isEmpty()) syncList.remove(0);    // another thread may empty it between calls
```

Strategy	How	When
Confinement	Stack locals, `ThreadLocal`, one owner thread	Default: do not share
Immutability	Records, final fields, unmodifiable collections	Shared read-mostly data
Copy-on-write snapshot	`volatile` reference to an immutable value, swap whole value	Config, reference data
Atomics / CAS	`AtomicLong`, `LongAdder`, `compareAndSet`	Single-variable state
Concurrent collections	`ConcurrentHashMap`, `BlockingQueue`	Shared maps and hand-offs
Locks	`synchronized`, `ReentrantLock`	Invariants across several fields
Database	Row locks, `SKIP LOCKED`, unique constraints	Coordination across JVMs

Lock the smallest scope, and never call unknown code (callbacks, remote calls) while holding a lock.

RefinePoint: the gateway's `RateLimitFilter` combines three of these. A `ConcurrentHashMap<String, Bucket>` with `computeIfAbsent` creates each caller's bucket atomically. Each `Bucket` has `synchronized take` and `remaining`, because `tokens` and `last` form one invariant that two threads must not interleave. Having one lock per bucket means two different callers never contend. The critical section is only arithmetic, so it is safe on the gateway's reactive event loop too.

RefinePoint: `OAuthConnectionService.accessToken` takes a per-connection lock, `synchronized (locks.computeIfAbsent(connectionId, id -> new Object()))`, so two concurrent callers in one instance do not both spend a refresh token. Know its limits when you discuss it. It guards one JVM only, so several instances need a database lock. And the method is `@Transactional` with the entity loaded before the lock, so a waiter can read a pre-commit row, because the lock is released before the proxy commits. Locks inside transactions are a classic review finding.

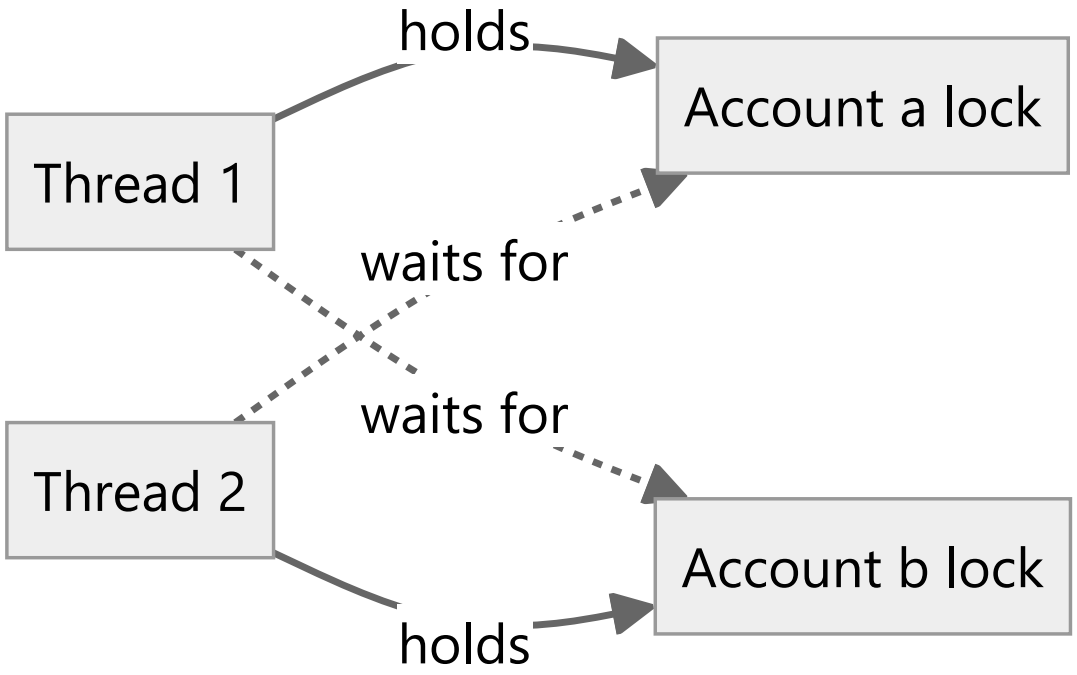
8.11 Deadlock, livelock and starvation

Deadlock

Threads wait for each other forever. All four **Coffman conditions** must hold: **mutual exclusion**, **hold and wait**, **no preemption**, **circular wait**. Remove one and deadlock is impossible.

```
void transfer(Account from, Account to, BigDecimal amount) {
    synchronized (from) {
        synchronized (to) {          // transfer(a, b) and transfer(b, a) concurrently: deadlock
            from.debit(amount);
            to.credit(amount);
        }
```

```
    }
}
```



Detection with a thread dump (`jcmd <pid> Thread.print` , `jstack <pid>` , or `kill -3` to stdout):

```
Found one Java-level deadlock:
=============================
"http-nio-8080-exec-1":
  waiting to lock monitor 0x00007f3a1c003f58 (object 0x00000007167c0a10, a com.x.Account),
  which is held by "http-nio-8080-exec-2"
"http-nio-8080-exec-2":
  waiting to lock monitor 0x00007f3a1c006a48 (object 0x00000007167c0a28, a com.x.Account),
  which is held by "http-nio-8080-exec-1"
```

The JVM finds cycles for both monitors and `java.util.concurrent` locks. In code, `ThreadMXBean.findDeadlockedThreads()` does the same and can drive a health alert. Take three dumps some seconds apart, because threads stuck in the same place in all three are the real problem.

Prevention:

```
void transfer(Account from, Account to, BigDecimal amount) {
    Account first = from.id().compareTo(to.id()) < 0 ? from : to;      // global lock order
    Account second = first == from ? to : from;
    synchronized (first) { synchronized (second) { from.debit(amount); to.credit(amount); } }
}

if (lock1.tryLock(1, SECONDS)) {                                       // or break hold-and-wait
with a timeout
    try {
        if (lock2.tryLock(1, SECONDS)) {
            try { work(); } finally { lock2.unlock(); }
        }
    } finally { lock1.unlock(); }
}
```

Others: hold one lock at a time, make **open calls** (call out only after releasing), and use a higher-level structure instead of nested locks.

Deadlocks the JVM will not report, which are more common in real services:

- **Thread-pool starvation deadlock.** Tasks in a fixed pool submit subtasks to the same pool and block on the result. All threads wait for work that has no thread to run on.
- **Connection-pool deadlock.** Each thread holds a JDBC connection and needs a second (a `REQUIRES_NEW` call in an outer transaction). The pool empties and everyone waits for `connectionTimeout`.
- **Database deadlocks.** PostgreSQL detects a cycle and aborts one transaction with SQLSTATE `40P01`. Fix by a consistent update order and retrying.
- **Distributed deadlocks** between services calling each other synchronously, which only timeouts can break.

Livelock and starvation

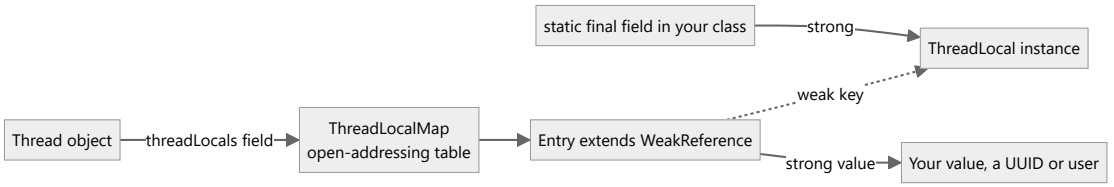
Livelock: threads are active but make no progress, like two people stepping aside in a corridor in the same direction. Typical cause: two `tryLock` loops that release and immediately retry in lock-step, or a poison message retried forever. Fix with randomised backoff and a retry limit.

Starvation: a thread never gets what it needs because others keep winning. Causes: unfair locks, a long-held lock, a flood of high-priority work. `new ReentrantLock(true)` gives FIFO fairness at a throughput cost.

> **RefinePoint:** starvation shows up at the tenant level too. `RefreshWorker.claim()` selects `... ORDER BY (SELECT count(*) ... WHERE x.org_id = r.org_id AND x.status = 'RUNNING'), r.created_at LIMIT 1 FOR UPDATE SKIP LOCKED`. Workspaces with fewer running refreshes go first, so one tenant queuing hundreds of refreshes cannot starve the others, and `SKIP LOCKED` means workers do not block on each other's rows. Claims of one row per transaction with no second lock leave no cycle to deadlock on.

8.12 ThreadLocal internals and leaks

A `ThreadLocal<T>` gives each thread its own copy of a value. The values are **not stored in the ThreadLocal**; they are stored in a map on the `Thread` object.



`get()` is `Thread.currentThread().threadLocals.getEntry(this)`, so there is no shared structure and no locking. `ThreadLocalMap` uses linear probing, and each `ThreadLocal` gets a hash code stepped by the golden-ratio constant `0x61c88647` to scatter entries evenly. Each `Entry` holds the key **weakly** and the value **strongly**.

The two leaks

1. **Memory.** If the `ThreadLocal` becomes unreachable, the key clears, but the value is still strongly held by the thread's map until a later `get`, `set` or `remove` happens to probe and expunge that stale slot. In a pool, threads live forever. The usual case is worse: `ThreadLocal`s are `static final`, so the key is never collected at all and the value stays until overwritten. A value whose class came from an undeployed web app also pins that class loader, the classic redeploy metaspace leak.
2. **Stale data, a correctness and security bug.** Thread 7 serves request A and sets `currentOrg = A`, with no cleanup. Thread 7 then serves request B, which reads `currentOrg`. That is a **tenant data leak**, and far more serious than the memory growth.

```
private static final ThreadLocal<UUID> ORG = new ThreadLocal<>();

ORG.set(orgId);
try {
    handle();
} finally {
    ORG.remove();        // always in finally, and remove rather than set(null): remove frees the
entry
}
```

RefinePoint: org context comes in two flavours. Inside a request, `OrgContextService` is stateless: it reads the current request through Spring's `RequestContextHolder` (a `ThreadLocal` managed by the framework, which clears it), and takes the `X-Org-Code` header and token claims from there. Outside a request (runs, schedules) there is no request, so `ryfn-expression` has its own scope that restores the previous value in `finally`:

```
public final class ExpressionScope {
    private static final ThreadLocal<UUID> ORG = new ThreadLocal<>();

    public static <T> T inWorkspace(UUID orgId, Supplier<T> body) {
        UUID before = ORG.get();
        ORG.set(orgId);
        try {
            return body.get();
        } finally {
            if (before == null) ORG.remove(); else ORG.set(before);   // safe to nest; leaves
pool threads clean
        }
    }
}
```

The callback shape means no caller can forget the cleanup: it is impossible to set the value without the `finally` running. Rule evaluation reads workspace lookup tables through this scope, and these run on pooled worker threads, so a missed `remove` would mean rules in one workspace reading another workspace's tables.

Where ThreadLocal context breaks

Spring keeps a lot in thread-locals: `SecurityContextHolder`, `RequestContextHolder`, `TransactionSynchronizationManager`, `LocaleContextHolder`, and MDC for logging. All of it is **lost on a**

thread hop: `@Async`, `CompletableFuture.supplyAsync`, `parallelStream`, your own executors. Fix with a `TaskDecorator` that copies and clears the context, or pass the values explicitly. In WebFlux a request hops threads constantly, so reactive code uses Reactor `Context` instead.

`InheritableThreadLocal` copies values to a child at **thread creation**, which is useless or wrong with pools, because workers are created once, inheriting whatever was current then.

Virtual threads support `ThreadLocal`, but a million threads means a million copies, and they are never pooled. Java 21 previews `ScopedValue`, which is immutable and bounded by a block (chapter 7).

8.13 Reading a thread dump

```
"run-3" #41 daemon prio=5 os_prio=0 cpu=812.33ms elapsed=3600.12s tid=0x00007f.. nid=0x1a2f
waiting on condition
   java.lang.Thread.State: WAITING (parking)
        at jdk.internal.misc.Unsafe.park(Native Method)
        - parking to wait for  <0x00000007166a0> (a
java.util.concurrent.locks.AbstractQueuedSynchronizer$ConditionObject)
        at java.util.concurrent.LinkedBlockingQueue.take(LinkedBlockingQueue.java:435)
```

What you see	Meaning	Likely action
Many `BLOCKED (on object monitor)` on one address	One hot lock	Find the holder (`- locked <addr>`), shrink the critical section
Pool threads `WAITING` on `queue.take`	Idle workers	Normal
`RUNNABLE` in `socketRead0` / `NioSocketImpl`	Waiting on a remote call or DB	Check timeouts, the downstream
`TIMED_WAITING` in `HikariPool.getConnection`	Connection pool exhausted	Long transactions, pool size, leak detection
`RUNNABLE` with high `cpu=` growing across dumps	Spinning or heavy work	Map `nid` to `top -H`, profile with JFR
`Found one Java-level deadlock`	Deadlock	Fix lock order

Take the dump with `jcmd <pid> Thread.print -l` (`-l` adds `java.util.concurrent` ownership). For virtual threads use `jcmd <pid> Thread.dump_to_file` (chapter 10).

8.14 Classic coding questions

Print odd and even numbers with two threads

```
public final class OddEven {
    private final Object lock = new Object();
    private final int max;
    private int current = 1;

    OddEven(int max) { this.max = max; }

    void print(int remainder) {
```

```
        synchronized (lock) {
            while (current <= max) {
                if (current % 2 == remainder) {
                    System.out.println(Thread.currentThread().getName() + " " + current++);
                    lock.notifyAll();
                } else {
                    try {
                        lock.wait();
                    } catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                        return;
                    }
                }
            }
        }
    }

    public static void main(String[] args) {
        OddEven p = new OddEven(10);
        new Thread(() -> p.print(1), "odd").start();
        new Thread(() -> p.print(0), "even").start();
    }
}
```

The final `notifyAll` happens on the last print, and the other thread wakes, finds `current > max` and exits, so neither thread is stranded in `wait`.

Ping-pong with semaphores

```
Semaphore ping = new Semaphore(1), pong = new Semaphore(0);

Runnable pinger = () -> repeat(() -> { ping.acquire(); System.out.println("ping");
pong.release(); });
Runnable ponger = () -> repeat(() -> { pong.acquire(); System.out.println("pong");
ping.release(); });
```

Each thread hands a permit to the other. The strict alternation comes from the initial permits (1 and 0), and no shared flag is needed. The helper `repeat` loops a fixed number of times and handles `InterruptedException`. Generalise to N threads printing in turn with an array of semaphores: thread `i` acquires `s[i]` and releases `s[(i + 1) % n]`.

Bounded buffer with `ReentrantLock` and two conditions

```
public final class BoundedQueue<T> {
    private final Object[] items;
    private int head, tail, count;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();

    public BoundedQueue(int capacity) { items = new Object[capacity]; }

    public void put(T item) throws InterruptedException {
        lock.lockInterruptibly();
        try {
            while (count == items.length) notFull.await();
```

```
            items[tail] = item;
            tail = (tail + 1) % items.length;
            count++;
            notEmpty.signal();             // wake exactly a consumer
        } finally { lock.unlock(); }
    }

    @SuppressWarnings("unchecked")
    public T take() throws InterruptedException {
        lock.lockInterruptibly();
        try {
            while (count == 0) notEmpty.await();
            T item = (T) items[head];
            items[head] = null;            // help GC
            head = (head + 1) % items.length;
            count--;
            notFull.signal();              // wake exactly a producer
            return item;
        } finally { lock.unlock(); }
    }
}
```

Separate wait sets are why `signal` is safe here and no herd occurs. This is essentially `ArrayBlockingQueue`.

8.15 Interview questions and answers

Q1. What is the difference between a process and a thread?

A process has its own address space; threads share the process's heap and code and each own a stack and program counter. Threads communicate cheaply through shared objects, and that sharing is the source of every concurrency bug. A Java platform thread is one OS thread, so the OS schedules it and it reserves a stack (1 MB by default on 64-bit Linux).

Q2. How many threads can a JVM create?

There is no JVM constant. It is limited by OS limits (`ulimit -u`, cgroup `pids.max`, `vm.max_map_count`) and address space for stacks, and failure appears as `OutOfMemoryError: unable to create native thread`, even with a free heap. In containers the `pids` limit often hits first. In practice a few thousand platform threads is where memory and context switching hurt, while virtual threads go to millions.

Q3. How many threads should a pool have?

For CPU-bound work, about the number of cores. For I/O-bound work, Goetz's formula: `threads = cores × target utilisation × (1 + wait time / compute time)`. Then verify by load test, because the real limit is usually downstream, such as the database pool. Running 200 threads against a 10-connection pool only moves the queue.

Q4. Explain the thread lifecycle. What is the difference between BLOCKED and WAITING?

NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED. BLOCKED means waiting for a monitor to enter `synchronized`; the thread wakes automatically when it gets the lock. WAITING means it voluntarily

parked for another thread's action, through `wait`, `join` or `park`. A `ReentrantLock` waiter shows WAITING rather than BLOCKED, since it parks.

Q5. What state is a thread in during a blocking socket read?

`RUNNABLE`. The JVM only sees a native call in progress and cannot tell that the OS has the thread sleeping. That is why a thread dump showing everything RUNNABLE in `socketRead` can coincide with an idle CPU: all threads are waiting on a slow downstream, so look for missing timeouts.

Q6. What happens if you call `start()` twice? What if you call `run()` directly?

A second `start()` throws `IllegalThreadStateException`, whether or not the first run has finished, since threads cannot be restarted. Calling `run()` runs the method on the current thread with no new thread and no error, and it is a common bug that passes tests because the result is correct, just sequential.

Q7. `Runnable` vs `Callable`?

`Runnable.run` returns nothing and cannot throw checked exceptions. `Callable.call` returns a value and may throw `Exception`, and it is used with `submit` and `Future`. `FutureTask` implements `Runnable` and wraps a `Callable`, which lets a `Thread` run it.

Q8. What happens to an uncaught exception in a thread?

The thread dies, the `UncaughtExceptionHandler` runs (per thread, then default, by default printing to stderr), and other threads keep going. The JVM does not exit. In an executor, `execute` lets the exception reach the handler and the pool replaces the worker; `submit` captures it in the `Future`, so **it vanishes unless someone calls `get()`**. That is the most common "my scheduled task silently stopped" cause: `scheduleAtFixedRate` also cancels future runs once a task throws.

Q9. What is a daemon thread? Does `finally` run?

A background thread that does not keep the JVM alive. At exit, once only daemons remain, the JVM stops them abruptly and their `finally` blocks **do not run**. Do not hold durable state in them without recovery. Set it before `start()`, or it throws `IllegalThreadStateException`. A thread created by a daemon is a daemon by default.

Q10. Do thread priorities work?

They are a hint, and on Linux HotSpot ignores them by default. Never rely on them for correctness or ordering. Virtual threads always have the default priority. To favour some work, use separate pools or queues.

Q11. `sleep` vs `wait`?

`sleep` is a static `Thread` method that pauses without releasing any lock and needs no monitor. `wait` is an `Object` method that must be called while holding the monitor, releases that monitor, and resumes on notify

or timeout. Sleeping inside `synchronized` makes everyone else wait for the whole duration, a real production slowdown.

Q12. Why are `wait` and `notify` on `Object` rather than `Thread`?

Because you wait on a **condition guarded by a monitor**, and the monitor belongs to whichever object is the lock. Any object can be a lock, so the methods must be available on any object. A thread waits *for* something; it does not wait "on a thread".

Q13. Why must `wait` be called in a synchronized block?

To avoid the **lost wake-up**. Without the monitor, a consumer could check "empty", then a producer adds and notifies with nobody waiting yet, and then the consumer waits forever. Holding the monitor makes check-and-wait atomic relative to change-and-notify. The JVM enforces it with `IllegalMonitorStateException`.

Q14. Why must `wait` be called in a loop?

Because waking up does not mean the condition is true: spurious wakeups are permitted, another thread may have consumed the state before this one re-acquired the monitor, and with one wait set the notification may have been for another condition. So `while (!condition) wait();`. `if` is a bug even when tests pass.

Q15. `notify` vs `notifyAll`?

`notify` moves one arbitrary waiter to the entry set; `notifyAll` moves them all, and each re-checks its condition. With `notify`, a producer may wake only another producer, and the system stalls. Use `notify` only if all waiters are equivalent and each notification lets exactly one proceed. Separate `Condition`s are the clean way to avoid the herd.

Q16. Does the woken thread run immediately after `notify`?

No. It moves to the entry set and is BLOCKED until the notifying thread exits the synchronized block, and then it competes with other threads for the monitor. Barging means another thread may get in first and change the state, which is one reason for the `while` loop.

Q17. How does `Thread.join()` work?

For platform threads it has traditionally been implemented with `wait()` on the `Thread` object, in a loop on `isAlive()`, and the JVM calls `notifyAll` on that object as the thread exits. So `t.join()` releases `t`'s monitor but no other locks the caller holds. It is also why you should never use a `Thread` as your own lock or wait target. Completion of `t` happens-before `join` returning.

Q18. What does `Thread.yield()` do? `Thread.onSpinWait()`?

`yield` hints that the current thread is willing to give up the CPU, and the scheduler may ignore it. It is rarely useful in application code. `Thread.onSpinWait` (Java 9) tells the CPU you are in a spin loop (x86 `pause`), which saves power and improves the exit from the loop. It does not give up the thread's turn.

Q19. How do you stop a thread?

Cooperatively: set a `volatile` flag and/or call `interrupt()`, and have the thread check and exit cleanly. `Thread.stop` could leave invariants broken by releasing all monitors mid-update and now throws. For pools: `shutdown()` then `awaitTermination`, then `shutdownNow()`, which interrupts workers. Code ignoring interruption never stops.

Q20. What should you do when you catch `InterruptedException`?

Either propagate it, or restore the flag with `Thread.currentThread().interrupt()` and exit the task. Do not swallow it: the exception cleared the flag, so an empty `catch` destroys the only signal that someone wanted this thread to stop. It then ignores `shutdownNow`, and graceful shutdown hangs.

Q21. `interrupted()` vs `isInterrupted()`?

`Thread.interrupted()` is static, checks the current thread and **clears** the flag. `t.isInterrupted()` is an instance method that does not clear. Calling `interrupted()` twice in a row returns `true` then `false`, a typical output question.

Q22. Will this thread stop?

```
Thread t = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) {
        try { Thread.sleep(1000); }
        catch (InterruptedException e) { System.out.println("woken"); }
    }
});
t.start(); t.interrupt();
```

No. `sleep` throws, the flag is cleared, the catch only prints, and the loop continues forever. Fix by re-setting the flag or breaking in the `catch`.

Q23. Can you interrupt a thread blocked on `synchronized` or a socket read?

No for both. A thread waiting for a monitor stays blocked with the flag set, and classic socket reads ignore interruption; close the socket or use read timeouts instead. To get interruptible waiting use `lock.lockInterruptibly()` or `tryLock(timeout)`, or NIO channels. Socket I/O on a virtual thread is interruptible.

Q24. What does `synchronized` guarantee?

Mutual exclusion on that object's monitor, plus visibility and ordering: everything before an unlock is visible to the next locker of the same monitor. It is reentrant and releases on exception. Different threads locking **different** objects get no protection, so the lock object must be the same for all accesses.

Q25. Instance vs static synchronized methods: can two threads run them simultaneously?

An instance method locks `this`; a static method locks the `Class` object. These are different locks, so yes, one thread can be in a static method and another in an instance method at once. If both touch a static field, that field is unprotected. Likewise, two threads can run synchronized instance methods on two different instances.

Q26. What should you not synchronize on?

On `String` literals and boxed values such as `Integer` (interned or cached, so shared with unrelated code), `Boolean`, on a non-final field (reassignment changes the lock), on `this` in public classes (outsiders can lock it too), and on `Class` of a non-final class via `getClass()` (differs per subclass). Use a `private final Object lock = new Object();`.

Q27. Predict the behaviour.

```
private Integer count = 0;
void inc() { synchronized (count) { count++; } }
```

Broken. `count++` assigns a new `Integer`, so the lock object changes all the time. Threads synchronise on different objects, updates get lost, and small values are shared with the whole JVM through the Integer cache.

Q28. Explain how `synchronized` is implemented in the JVM.

A block compiles to `monitorenter` / `monitorexit`, with an exception-table handler that releases on throw; a method uses the `ACC_SYNCHRONIZED` flag. At runtime the JVM uses the object's mark word. An uncontended lock is a CAS installing a pointer to a lock record on the owner's stack (thin lock). On contention, `wait` or a hash request it inflates to an `ObjectMonitor` with an owner, recursion count, entry queues and wait set; waiters spin briefly then park.

Q29. What is the mark word and what does it hold?

The first 8 bytes of an object's header on 64-bit HotSpot. It multiplexes identity hash code and GC age when unlocked, a lock-record pointer when thin-locked, an `ObjectMonitor` pointer when inflated, and a forwarding pointer during GC, with low tag bits saying which. Once a lock displaces it, the hash lives in the lock record or monitor. Compact object headers in later JDKs shrink this layout.

Q30. What was biased locking and why was it removed?

An optimisation where the first thread to lock an object writes its ID into the mark word and later relocks with no atomic operation. It paid off for legacy uncontended synchronised classes. Revocation required safepoints, and the code was hard to maintain, while modern code uses `java.util.concurrent` and short-lived objects. Disabled and deprecated in Java 15 (JEP 374), removed in 18.

Q31. `synchronized` vs `ReentrantLock`?

`synchronized` is simpler and always released automatically. `ReentrantLock` adds `tryLock`, timed and interruptible acquisition, fairness, and multiple `Condition`s, at the cost of a mandatory `finally { unlock(); }`. Uncontended performance is similar today. Default to `synchronized` or a higher-level utility; choose `ReentrantLock` for the extra features, or on Java 21 where blocking inside `synchronized` pins virtual threads. More in chapter 9.

Q32. What is the Java Memory Model?

The part of the JLS (chapter 17) defining which values a read may observe in a multi-threaded program. It lets compilers and CPUs optimise freely, while promising that correctly synchronised programs behave sequentially consistently. It is expressed through happens-before rules for monitors, `volatile`, thread start and join, and `final` fields, and it applies on any hardware.

Q33. What is happens-before? Does it mean earlier in time?

A partial order: if A happens-before B, B must see A's effects. It is not time. Two actions with a wall-clock order can have no HB edge, so the later read may legally see an old value. Conversely, an HB pair may be reordered by the JIT if no observer can tell. Name the rules: program order, monitor, volatile, start, join, interruption, transitivity.

Q34. What is a data race, and what is DRF-SC?

A data race is two threads accessing the same variable, at least one writing, without a happens-before order between them. DRF-SC (data-race-free implies sequential consistency) says that a program without data races behaves as if all actions ran in some single interleaved order respecting each thread's source order. So you do not have to reason about reordering; you only have to eliminate data races.

Q35. Visibility vs atomicity with an example.

Visibility: a thread sees another's write. Atomicity: an operation cannot be observed half done. `volatile int count` gives visibility, yet `count++` loses updates, because read-add-write is three steps. A `synchronized` counter has both. A non-volatile flag can be atomic to write and still never be seen by the reader.

Q36. Predict: can this print 0?

```
class Holder { int x; boolean ready; }
// Thread 1            // Thread 2
h.x = 42;              if (h.ready) System.out.println(h.x);
h.ready = true;
```

Yes, without `volatile`. The writes can be reordered or seen out of order, so thread 2 can see `ready == true` and `x == 0`. It may also never see `ready` as true. Making `ready` volatile fixes both, since the volatile write publishes `x` by transitivity. Making `x` volatile instead does not.

Q37. What does `volatile` do in hardware terms?

It stops the JIT from caching or reordering the access, and adds memory barriers. On x86 a volatile read is a plain load, and a write adds a StoreLoad fence (`lock addl` on the stack pointer), flushing the store buffer before later loads. On ARM the JVM uses `ldar` and `stlr`. It does not bypass the CPU caches, which is a popular wrong answer: cache coherence still handles that.

Q38. Is `volatile` a replacement for `synchronized`?

Only when writes do not depend on the current value and no invariant spans several fields, such as flags and immutable-snapshot publication. It offers no mutual exclusion. Good pattern: reads through a volatile reference with writes in `synchronized`, the cheap read-write lock from *Java Concurrency in Practice*.

Q39. Is a `volatile long` needed? Are array elements of a `volatile` array volatile?

`long` and `double` writes may be non-atomic per JLS 17.7 unless volatile, though 64-bit JVMs are atomic in practice. Still, `volatile` is needed for visibility. For `volatile int[] a`, only the reference is volatile, and `a[0] = 1` is an ordinary write. Use `AtomicIntegerArray` or `VarHandle`.

Q40. What is false sharing?

Two independent volatile or frequently written variables on the same 64-byte cache line. Each write invalidates the line in the other core, so threads slow each other down with no logical conflict. It shows up as a counter array that scales negatively. Fix by padding, or `@Contended` (with `-XX:-RestrictContended` for your own code). `LongAdder` uses this inside its `Cell`s.

Q41. Explain double-checked locking and why it was broken.

Check without the lock for speed, lock, check again, then create. Before Java 5, and today without `volatile`, the reference assignment could become visible before the constructor's writes, so a thread at the first check could use a half-built object. The Java 5 JMM made `volatile` forbid this reordering. The holder idiom is simpler when you do not need instance-level laziness.

Q42. Show all the ways to write a thread-safe singleton.

Eager `static final` (simple, created at class init); static holder (lazy, class-init lock, no synchronisation cost); `enum` (serialization and reflection safe); `volatile` DCL (lazy, usable for instance-level fields); synchronised method (correct, every call locks). In Spring: a singleton bean. A singleton bean must still be stateless or thread-safe, since one instance serves all threads.

Q43. What guarantee do `final` fields give?

Any thread obtaining a reference after the constructor finishes sees the final fields' correct values, including what they point to as of the freeze, even with racy publication, provided `this` did not escape from the constructor. Mutating an object reached through a final field later still needs synchronisation: `final` means the reference, not deep immutability.

Q44. What is safe publication? Is this safe?

```
public Config config;                 // public, non-final, non-volatile
void init() { config = new Config(load()); }
```

Safe publication makes reference and state visible together. This one is unsafe unless `Config`'s fields are all final: a reader could see a non-null `config` with default field values. Options: `volatile`, `final` set in a constructor, a static initializer, a lock, or a thread-safe collection. Records with immutable components are safe by construction, because their fields are final.

Q45. What does "`this` escapes the constructor" mean?

Another thread can reach the object before construction completes, voiding the final-field guarantee and invariants. Causes: registering a listener, starting a thread in the constructor, or an inner class instance that captures `this`. Fix with a private constructor and a static factory, or start after construction. In Spring, do it in `@PostConstruct` or on `ApplicationReadyEvent`.

Q46. What is a race condition? Show three forms.

Correctness that depends on timing. Check-then-act: `if (!map.containsKey(k)) map.put(k, v)`. Read-modify-write: `count++`. Compound invariants: updating `min` and `max` separately. Each step can be individually thread-safe and the sequence still wrong. A distributed version: two service instances check that a row is absent, then both insert, for which you need a unique constraint.

Q47. Is `HashMap` read-only access thread-safe? `SimpleDateFormat`?

A `HashMap` that is safely published and never modified can be read concurrently; a single concurrent writer makes it unsafe (lost entries, corruption, and on Java 7 a resize could create an infinite loop). `SimpleDateFormat` holds mutable state in a `Calendar` field and is unsafe, so a shared one produces wrong dates under load, while `DateTimeFormatter` is immutable and fine.

Q48. Are Spring singleton beans thread-safe?

Only if you write them that way. The container creates one instance and all request threads call it concurrently, with no synchronisation added. Stateless services with final dependencies are naturally safe. Mutable fields, such as a per-request list or a `SimpleDateFormat`, are races. Keep request state in locals, parameters or request-scoped beans.

Q49. What are the four conditions for deadlock?

Mutual exclusion, hold and wait, no preemption, circular wait. All are necessary, so prevention breaks one: lock ordering removes circular wait, `tryLock` with a timeout effectively removes no preemption and hold and wait, and lock-free design removes mutual exclusion.

Q50. Write code that deadlocks.

```
Object a = new Object(), b = new Object();
new Thread(() -> { synchronized (a) { pause(); synchronized (b) { } } }).start();
new Thread(() -> { synchronized (b) { pause(); synchronized (a) { } } }).start();
```

The short pause makes each thread hold its first lock before asking for the second. Be ready to take `jcmd <pid> Thread.print` and read the "Found one Java-level deadlock" section.

Q51. How do you diagnose a hung service in production?

Take three thread dumps 10 seconds apart (`jcmd <pid> Thread.print -l`) and check for a reported deadlock. Then group threads by stack: many BLOCKED on one lock (hot lock), parked in `HikariPool.getConnection` (pool exhaustion), `RUNNABLE` in socket read (slow downstream with no timeout). Check `top -H` and CPU, plus GC logs for pauses, and pool metrics. Deadlock is only one cause; in my experience pool exhaustion and missing timeouts are far more common.

Q52. Can you recover from a deadlock in Java?

Not for `synchronized` : a stuck monitor cannot be taken away, so you restart. With `ReentrantLock` you can use timed `tryLock` or `lockInterruptibly` to back out. So detection (`ThreadMXBean.findDeadlockedThreads` as a health signal) needs to end in a restart, with the fix being lock ordering. A database can recover by aborting a victim, which is why you need retries around such transactions.

Q53. What is thread-pool starvation deadlock?

Tasks in a bounded pool wait for subtasks in the **same** pool. With all threads occupied by parents, the subtasks sit in the queue forever, and no monitor cycle exists, so the JVM reports nothing. Fix: separate pools per level, `ForkJoinPool` (which helps while joining), or do not block on subtasks. On Spring, `@Async` calling `@Async` and then `.get()` on one executor is the common way to hit it.

Q54. Livelock vs deadlock vs starvation?

Deadlock: threads blocked forever, 0% CPU. Livelock: threads busy but reacting to each other with no progress, high CPU. Starvation: the system makes progress but some thread never does. Backoff with jitter fixes livelock, consistent ordering fixes deadlock, and fairness or isolation fixes starvation.

Q55. How does `ThreadLocal` work internally?

Each `Thread` has a `threadLocals` field holding a `ThreadLocalMap` , with open addressing, keyed by the `ThreadLocal` instance through a `WeakReference` , with strong references to values. `get` looks in the current thread's map, so it needs no locks. Stale entries (key collected) are cleared opportunistically on later probing, and fully on `remove` .

Q56. Why does `ThreadLocal` leak in a thread pool? Why is the key weak?

Pool threads never die, so the map and its values live on. A weak key lets an unreachable `ThreadLocal` be collected and its slot be identified as stale; the value stays strong because nothing else may hold it. So the value lingers until the slot is probed. With the usual `static final` `ThreadLocal`, it lingers forever. Always `remove()` in `finally`. The bigger risk is the next task reading the previous task's value.

Q57. How do you carry a multi-tenant org context across threads?

Pass it explicitly where possible. In RefinePoint, a request resolves the workspace from the URL or `X-Org-Code` through `OrgContextService` and `RequestContextHolder`, and queue work carries `org_id` on its claimed row, so `RunWorker`-style code calls `ExpressionScope.inWorkspace(orgId, ...)`, which cleans up in `finally`. Across `@Async` use a `TaskDecorator`. Queries still filter by `org_id`, so a context bug cannot widen access on its own: that is defence in depth.

Q58. Why does `SecurityContextHolder` return null in an `@Async` method?

It uses a `ThreadLocal` by default, and the async executor runs on a different thread. Fix with `DelegatingSecurityContextAsyncTaskExecutor`, a `TaskDecorator`, or pass the needed identity as a parameter. `MODE_INHERITABLETHREADLOCAL` looks like a fix but is wrong with pools, as workers keep whichever context they inherited at creation, which can be another user's.

Q59. Compare the thread-safety strategies you would apply to a shared cache.

Immutable snapshot behind a `volatile` reference for read-mostly reference data: lock-free reads, and a whole-map swap on refresh. `ConcurrentHashMap` with `computeIfAbsent` for per-key loading with short mappings (the mapping function holds a bin lock, so no I/O in it). A `synchronized` map only for small, low-traffic use. For multi-instance consistency: Redis or DB. State the trade-off in terms of staleness, as much as speed.

Q60. Write three threads that print A, B, C in order, repeatedly.

```
Semaphore[] s = { new Semaphore(1), new Semaphore(0), new Semaphore(0) };
for (int i = 0; i < 3; i++) {
    final int me = i;
    new Thread(() -> {
        for (int n = 0; n < 5; n++) {
            s[me].acquireUninterruptibly();
            System.out.print((char) ('A' + me));
            s[(me + 1) % 3].release();
        }
    }).start();
}
```

Prints `ABCABCABCABCABC`. A wait/notify version uses a shared `turn` integer: each thread loops on `while (turn % 3 != me) wait();`, prints, increments and calls `notifyAll`. Compare the two aloud, as the semaphore form has no shared mutable state to guard.

Q61. Implement a thread-safe counter in three ways, and rank them.

`synchronized` (simple, always correct, slow under heavy contention); `AtomicLong.incrementAndGet` (a CAS loop, and fast until contention forces retries); `LongAdder.increment` (per-cell striping, best under heavy write contention, but `sum()` is not an atomic snapshot). Choose `LongAdder` for metrics and `AtomicLong` when each increment's returned value matters, such as an ID.

Q62. What's wrong with using `synchronized` in a `@Transactional` method?

The proxy commits **after** the method returns, so the lock is released before the transaction commits. Another thread can enter, read the pre-commit state (read committed), and act on stale data: two threads both reserve the last seat. Fix: put the lock outside the transaction boundary, or better, use a DB lock (`SELECT … FOR UPDATE` , optimistic version, unique constraint), since a JVM lock does not cover other instances anyway.

Q63. How do you test concurrent code?

Unit tests catch little, because timing decides. Use a `CountDownLatch` start gate with many threads and iterations to raise contention, run on a multi-core machine, assert invariants, and use `jcstress` for JMM-level behaviour. Static analysis (`@GuardedBy` , SpotBugs) and code review catch more per hour, and designs that reduce sharing avoid the problem altogether. Awaitility beats `Thread.sleep` for waiting on async results.

Q64. Tell me about a concurrency decision in your project.

RefinePoint's run queue. Rather than in-JVM locks or leader election, workers claim rows with `FOR UPDATE SKIP LOCKED` , so instances coordinate through PostgreSQL and we scale by adding instances. Inside each JVM a fixed daemon pool runs claimed work, stopped with a `volatile stopping` flag plus `shutdownNow` . Status and heartbeat live in the row, so a dead instance's work is recoverable. Shared mutable state in the JVM is nearly zero, and most of the concurrency model becomes easy to reason about because the database holds it.

9. java.util.concurrent in Depth

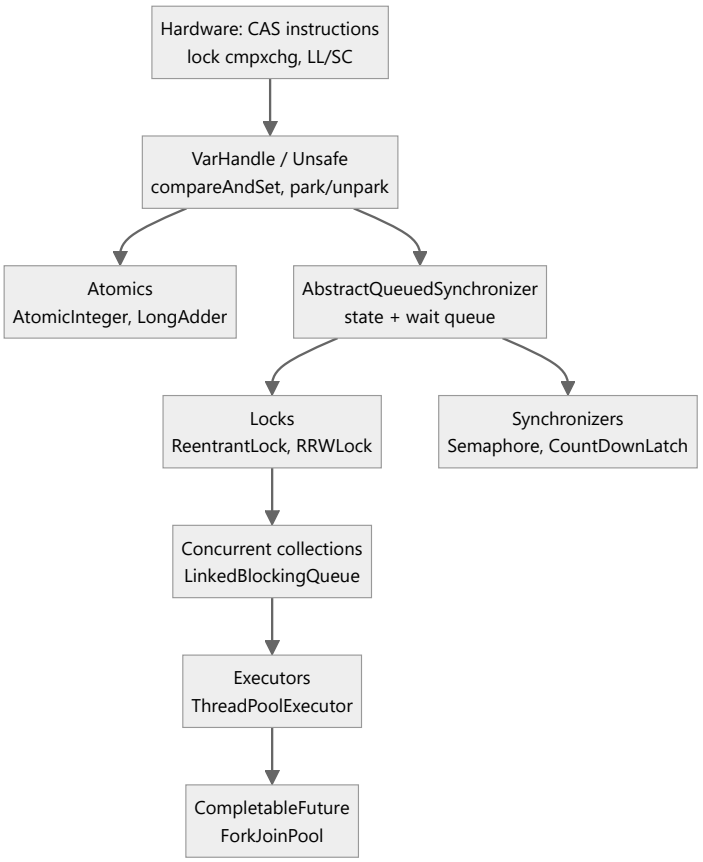
Chapter 8 covered threads, `synchronized`, `volatile` and the Java Memory Model. This chapter is about the toolkit Doug Lea built on top of those primitives: explicit locks, the AbstractQueuedSynchronizer that powers almost every one of them, atomics and CAS, the executor framework, `CompletableFuture`, fork/join and the synchronizers. Senior interviews rarely ask "what is a `CountDownLatch`?" in isolation. They ask "your thread pool is rejecting tasks at 2 a.m., walk me through why", or "call three services in parallel and combine the results with a timeout". Every section below explains the internals first, because the internals are what let you answer the scenario questions.

9.1 Why `java.util.concurrent` exists

`synchronized` is simple and, since Java 6, fast. But it has hard limits:

Need	`synchronized`	`java.util.concurrent` answer
Give up if the lock is busy	Impossible, blocks forever	`tryLock()`, `tryLock(timeout)`
Cancel a thread waiting for a lock	Not interruptible	`lockInterruptibly()`
Fair (FIFO) ordering	No guarantee	`new ReentrantLock(true)`
Several wait-sets on one lock	One (`wait/notify`)	Many `Condition` s
Many readers, one writer	Readers serialise	`ReentrantReadWriteLock`, `StampedLock`
Lock-free counters	Not possible	`AtomicLong`, `LongAdder`
Manage threads as a resource	`new Thread()` per task	`ExecutorService`, `ThreadPoolExecutor`
Compose async results	Manual `wait/notify`	`CompletableFuture`
Coordinate phases	Hand-written	`CountDownLatch`, `CyclicBarrier`, `Phaser`, `Semaphore`

The package splits into four layers. Knowing the layering is a good opening for any "explain JUC" question.



9.2 `Lock` vs `synchronized` and `ReentrantLock`

`Lock` is an interface: `lock()`, `unlock()`, `tryLock()`, `tryLock(long, TimeUnit)`, `lockInterruptibly()`, `newCondition()`. `ReentrantLock` is the standard implementation. Like `synchronized`, it is **reentrant** (the owner can acquire again; a hold count tracks depth) and gives the same memory-visibility guarantees: an `unlock()` *happens-before* every subsequent `lock()` of the same lock.

The idiom is non-negotiable — unlock in `finally`, and acquire *outside* the `try`, so a failed `lock()` never triggers an `unlock()` you do not own:

```
private final ReentrantLock lock = new ReentrantLock();

public void transfer(Account from, Account to, BigDecimal amount) {
    lock.lock();                // outside try
    try {
        from.debit(amount);
        to.credit(amount);
    } finally {
        lock.unlock();          // always released, even on exception
    }
}
```

Fairness

`new ReentrantLock()` is **non-fair** (default); `new ReentrantLock(true)` is **fair**.

- Non-fair: a thread calling `lock()` first tries to CAS the state from 0 to 1 *even if others are queued* ("barging"). If the lock was just released and the woken queued thread has not been scheduled yet, the

barger wins. This avoids a context switch and gives much higher throughput.

- Fair: `tryAcquire` first checks `hasQueuedPredecessors()`; if anyone is waiting, the newcomer joins the end of the queue. Throughput drops (often by an order of magnitude under contention) but no thread starves.
- `tryLock()` *without* a timeout barges **even on a fair lock**. `tryLock(0, SECONDS)` respects fairness.

> **Interview tip:** say "fairness trades throughput for predictability; I use the default unless I have measured starvation". Then mention that fairness of the lock does not mean fairness of the thread scheduler.

`tryLock` — avoiding deadlock and blocking

```
if (lock.tryLock(200, TimeUnit.MILLISECONDS)) {
    try { refreshCache(); } finally { lock.unlock(); }
} else {
    log.info("Refresh already in progress, skipping");
}
```

`tryLock` breaks the "hold and wait" condition of deadlock: acquire lock A, `tryLock` lock B, and if B fails release A, back off with jitter and retry.

`lockInterruptibly` — cancellable waiting

A thread blocked on `synchronized` ignores `interrupt()`. A thread in `lockInterruptibly()` throws `InterruptedException` and leaves the queue. That matters for `shutdownNow()`: executor workers stuck on a lock can actually stop.

`Condition` — multiple wait-sets

`Condition` is the `Lock` equivalent of `wait/notify`, but you can have several per lock. The classic bounded buffer needs two: "not full" for producers and "not empty" for consumers, so a `put` wakes only consumers.

```
class BoundedBuffer<T> {
    private final Object[] items;
    private int putIdx, takeIdx, count;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull  = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();

    BoundedBuffer(int capacity) { items = new Object[capacity]; }

    void put(T x) throws InterruptedException {
        lock.lock();
        try {
            while (count == items.length) notFull.await();   // while, never if: spurious wakeups
            items[putIdx] = x; putIdx = (putIdx + 1) % items.length; count++;
            notEmpty.signal();
        } finally { lock.unlock(); }
    }

    @SuppressWarnings("unchecked")
    T take() throws InterruptedException {
        lock.lock();
        try {
```

```
            while (count == 0) notEmpty.await();
            T x = (T) items[takeIdx]; items[takeIdx] = null; takeIdx = (takeIdx + 1) %
items.length; count--;
            notFull.signal();
            return x;
        } finally { lock.unlock(); }
    }
}
```

This is almost exactly how `ArrayBlockingQueue` is written. `await()` atomically releases the lock and parks; on wake-up it re-acquires the lock before returning. Calling `await` / `signal` without holding the lock throws `IllegalMonitorStateException`.

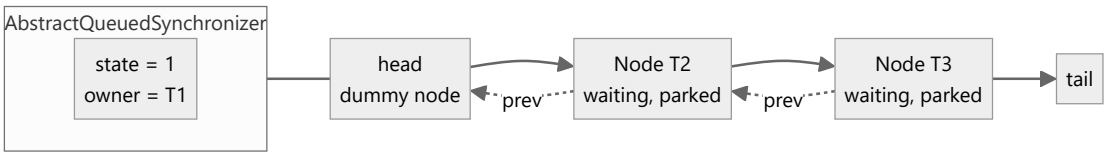
	`synchronized`	`ReentrantLock`
Acquire/release	Implicit, block-scoped	Explicit, must `unlock()` in `finally`
Timeout / try	No	Yes
Interruptible wait	No	Yes
Fairness	No	Optional
Conditions	One wait-set	Many
Diagnostics	Thread dump shows monitor	`getQueueLength()`, `isLocked()`, `getOwner()` (protected)
Virtual threads (Java 21)	Pins the carrier	Does not pin
Risk	None of forgetting to unlock	Forgotten `unlock()` = permanent lock

9.3 AbstractQueuedSynchronizer (AQS) internals

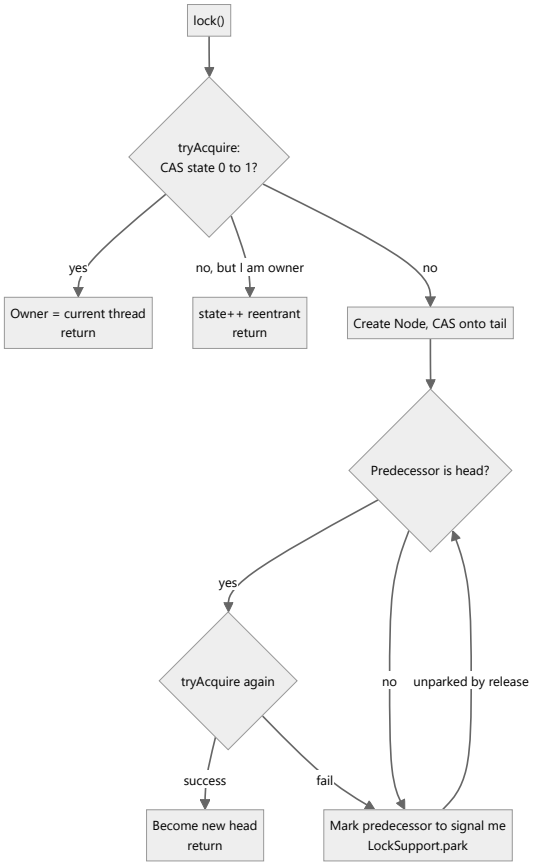
`ReentrantLock`, `ReentrantReadWriteLock`, `Semaphore`, `CountDownLatch` and `ThreadPoolExecutor.Worker` are all thin wrappers over `AbstractQueuedSynchronizer`. AQS provides two things:

1. A single `volatile int state`, changed with CAS. Its meaning belongs to the subclass: for `ReentrantLock` it is the hold count (0 = free), for `Semaphore` the available permits, for `CountDownLatch` the remaining count, for `ReentrantReadWriteLock` two 16-bit halves (upper = shared read holds, lower = exclusive write holds).
2. A **FIFO wait queue** of parked threads — a variant of the Craig, Landin and Hagersten (CLH) lock queue, implemented as a doubly linked list with `head` and `tail` pointers.

The subclass only implements `tryAcquire` / `tryRelease` (exclusive mode) or `tryAcquireShared` / `tryReleaseShared` (shared mode). AQS does the queueing, parking, waking, timeouts and interruption.



The acquire path (exclusive mode)



Key details interviewers probe:

- **Why a CLH-style queue?** Each waiter spins/parks on its *own* node's status, not a shared variable, so a release wakes exactly one successor (no thundering herd as with `notifyAll`).
- **Enqueue is lock-free**: a new node is linked with `compareAndSetTail`. The `prev` link is set before the CAS so it is always reliable; the `next` link can lag, which is why `unparkSuccessor` may walk backwards from `tail` to find the real successor.
- **Release**: `tryRelease` sets state to 0 (only when the hold count reaches zero), then `unparkSuccessor(head)`. The woken thread loops back, sees its predecessor is head, and calls `tryAcquire`. In a non-fair lock a barger may have stolen it in between — the woken thread simply parks again.
- **Parking** uses `LockSupport.park/unpark`, which maps to an OS futex (Linux) per thread. `unpark` before `park` is remembered as a one-permit token, so wake-ups are never lost.
- **Cancellation**: timeouts and interrupts mark a node CANCELLED; it is unlinked lazily.
- **Conditions** have their own singly linked *condition queue*. `await` moves the thread there and fully releases the lock (saving the hold count); `signal` transfers the first node to the main sync queue, where it competes for the lock again.
- JDK 14 rewrote AQS (JDK-8229442) with simpler node status values and better support for `ForkJoinPool` managed blocking, but the model — state + CLH queue + park — is unchanged.

Writing your own synchronizer on AQS is a common senior question. A one-shot latch:

```
class OneShotLatch {
    private final Sync sync = new Sync();
```

```
    private static final class Sync extends AbstractQueuedSynchronizer {
        protected int tryAcquireShared(int ignored) { return getState() == 1 ? 1 : -1; } // open?
        protected boolean tryReleaseShared(int ignored) { setState(1); return true; }      //
open it
    }
    public void await() throws InterruptedException { sync.acquireSharedInterruptibly(0); }
    public void open() { sync.releaseShared(0); }
}
```

9.4 `ReadWriteLock` and `StampedLock`

`ReentrantReadWriteLock`

Many threads may hold the **read** lock together; the **write** lock is exclusive and excludes readers. It pays off only when reads vastly outnumber writes *and* the critical section is long enough to outweigh the extra bookkeeping (each read acquire is a CAS on shared state, which itself becomes a contention point on many cores).

Rules worth knowing:

- **Downgrade is allowed**: holding the write lock, acquire the read lock, then release the write lock. You keep a consistent view without a gap.
- **Upgrade is not allowed**: holding the read lock and calling `writeLock().lock()` deadlocks, because the writer waits for all readers — including itself.
- **Writer starvation**: in non-fair mode, a new reader blocks if the first queued thread is a writer, which limits (but does not eliminate) writer starvation.
- The write lock supports `Condition`; the read lock does not.

`StampedLock` (Java 8)

`StampedLock` adds an **optimistic read** that does not write to shared memory at all — just reads a version stamp. That makes it scale on many-core machines where even the read-lock CAS of `ReentrantReadWriteLock` becomes a bottleneck.

```
class Point {
    private double x, y;
    private final StampedLock sl = new StampedLock();

    double distanceFromOrigin() {
        long stamp = sl.tryOptimisticRead();     // no locking, just a version number
        double cx = x, cy = y;                   // read into locals
        if (!sl.validate(stamp)) {               // a writer intervened?
            stamp = sl.readLock();               // fall back to a real read lock
            try { cx = x; cy = y; } finally { sl.unlockRead(stamp); }
        }
        return Math.hypot(cx, cy);
    }

    void move(double dx, double dy) {
        long stamp = sl.writeLock();
        try { x += dx; y += dy; } finally { sl.unlockWrite(stamp); }
```

```
    }
}
```

Trap: `StampedLock` is **not reentrant** (re-acquiring the write lock in the same thread self-deadlocks), has no `Condition`, and is not owner-aware. Inside an optimistic section, only read fields into locals; never act on them until `validate` succeeds, because you may have read a torn, inconsistent state.

	`synchronized` / `ReentrantLock`	`ReentrantReadWriteLock`	`StampedLock`
Concurrent readers	No	Yes	Yes, plus optimistic
Reentrant	Yes	Yes	**No**
Conditions	Yes	Write lock only	No
Best for	General mutual exclusion	Long reads, rare writes	Very hot, short reads

9.5 Atomics and compare-and-swap

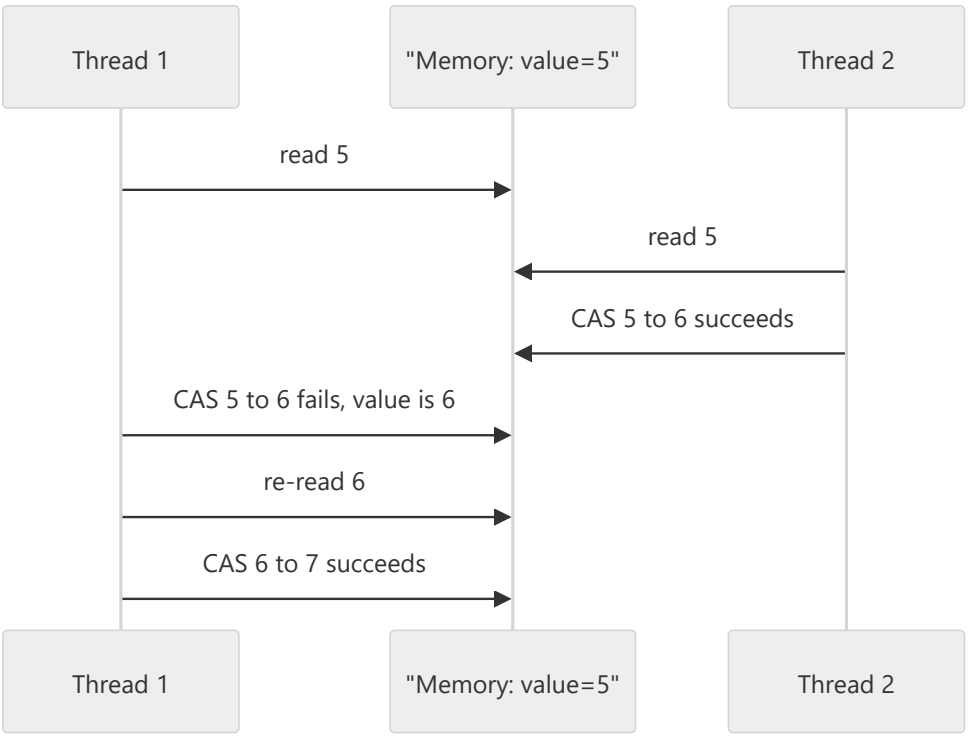
CAS at the CPU level

Compare-and-swap is a single hardware instruction: "if memory location M still holds expected value E, write new value N; tell me whether it worked". On x86 it is `LOCK CMPXCHG`: the `LOCK` prefix makes the read-compare-write atomic with respect to all cores via the cache-coherence protocol (the core holds the cache line in Exclusive/Modified state for the duration) and acts as a full memory fence. ARM uses load-linked/store-conditional (`LDXR` / `STXR`) loops or, from ARMv8.1, the single `CAS` instruction (LSE).

Java exposes CAS through `VarHandle.compareAndSet` (Java 9+) and, internally, `jdk.internal.misc.Unsafe`. The JIT replaces these calls with the instruction itself — they are *intrinsics*.

```
// What AtomicInteger.incrementAndGet conceptually does
int prev, next;
do {
    prev = value;            // volatile read
    next = prev + 1;
} while (!VALUE.compareAndSet(this, prev, next));   // retry if someone else won
return next;
```

(On x86 the JIT actually emits `LOCK XADD` for `getAndAdd`, which never fails, so there is no retry loop at all.)



CAS is **lock-free** (some thread always makes progress) but not **wait-free** (an unlucky thread can retry many times). Under extreme contention CAS loops burn CPU and cache-line bouncing dominates — which is why `LongAdder` exists.

The atomic classes

Class	Use
`AtomicInteger`, `AtomicLong`, `AtomicBoolean`	Counters, flags, sequence numbers
`AtomicReference<V>`	Swap an immutable object atomically (copy-on-write state)
`AtomicIntegerArray` etc.	Per-element atomic arrays
`AtomicIntegerFieldUpdater`	CAS on a `volatile` field of many objects without an extra wrapper object per instance
`AtomicStampedReference`, `AtomicMarkableReference`	Reference + version / mark, to defeat ABA
`LongAdder`, `LongAccumulator`, `DoubleAdder`	High-contention aggregation

Useful methods: `updateAndGet(fn)`, `accumulateAndGet(x, fn)` (the function may run more than once, so it must be side-effect free), `compareAndExchange` (returns the witness value), `lazySet` / `setRelease` (weaker ordering, cheaper), `getPlain` / `getOpaque` / `getAcquire`.

The ABA problem

CAS checks *value*, not *history*. Thread 1 reads A, is descheduled; thread 2 changes A → B → A; thread 1's CAS(A → C) succeeds although the world changed. With plain integers this is usually harmless. With references in

lock-free data structures it is fatal: in a lock-free stack, the node "A" at the top may have been popped, freed/reused and pushed back, and its `next` pointer is now stale.

```
Lock-free stack top:  A -> B -> C
T1: reads top=A, next=B  ... paused
T2: pop A, pop B, push A            => top: A -> C
T1: CAS(top, A, B) succeeds         => top: B  (B was already popped!) -> corruption
```

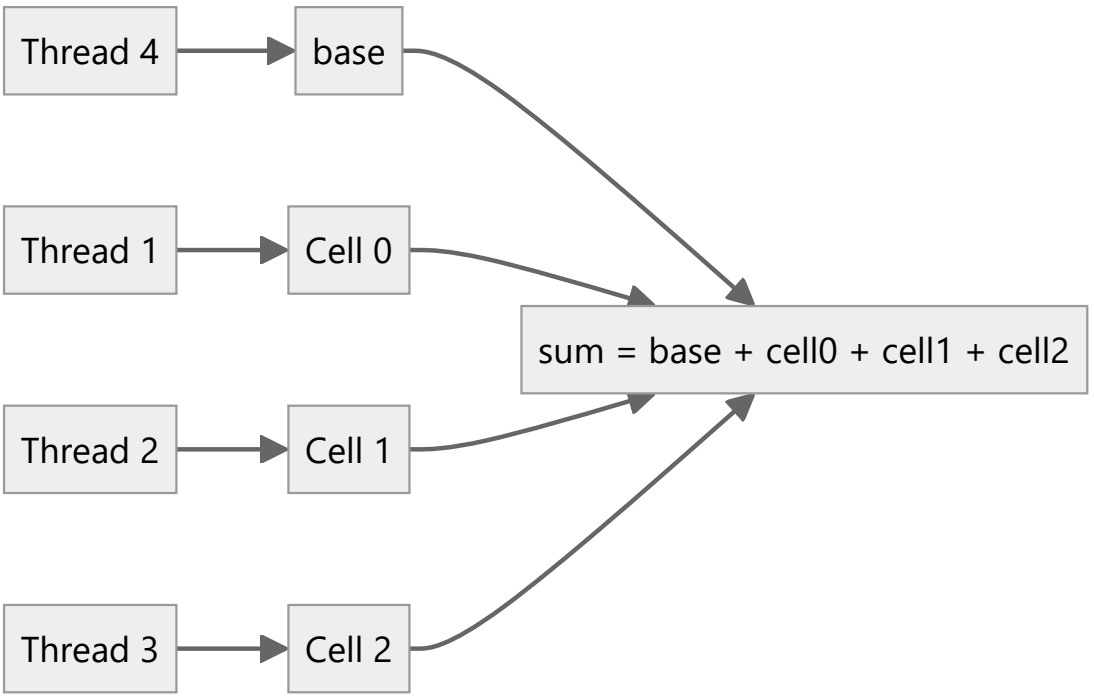
Fix: version the reference.

```
AtomicStampedReference<Node> top = new AtomicStampedReference<>(null, 0);
int[] stampHolder = new int[1];
Node current = top.get(stampHolder);
top.compareAndSet(current, current.next, stampHolder[0], stampHolder[0] + 1); // fails if stamp
moved
```

In Java, garbage collection already prevents the "freed and reused memory" variant (a node cannot be recycled while someone references it), so ABA mainly bites when you *reuse* node objects or when the value semantics matter (e.g., a balance that went 100 → 0 → 100).

LongAdder VS AtomicLong

`AtomicLong` is one memory word; every increment from every core fights for the same cache line. `LongAdder` (in `Striped64`) keeps a `base` plus a lazily created array of `Cell`s, each padded with `@Contended` to sit on its own cache line. A thread first CASes `base`; if that fails (contention detected) it hashes to a cell using a per-thread probe value and CASes that cell. The table doubles on repeated collisions, up to the number of CPUs. `sum()` adds `base` and all cells.

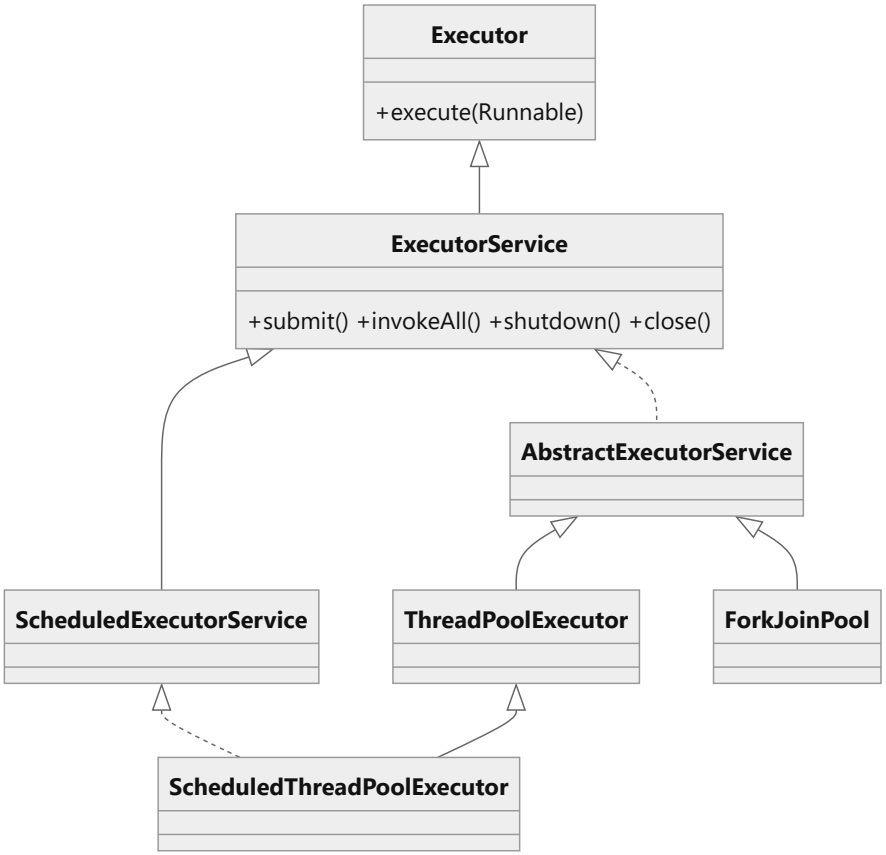


	`AtomicLong`	`LongAdder`
Write under contention	Degrades, CAS retries + cache-line ping-pong	Scales nearly linearly
Read	Exact, cheap	`sum()` walks cells; not an atomic snapshot
Memory	1 long	base + up to NCPU padded cells
`incrementAndGet` / CAS	Yes	No (only `add`, `increment`, `sum`)
Use for	Sequences, IDs, values you branch on	Metrics, hit counters, statistics

> **RefinePoint:** `RunWorker.poll()` can be entered from the scheduled tick and from the wake-up thread, so it starts with `if (!polling.compareAndSet(false, true)) return;` — an `AtomicBoolean` as a non-blocking "someone is already polling" guard. `DuckDbEngine` keeps an `AtomicInteger ACTIVE` of open engines so that the DuckDB memory budget is shared between running reconciliations. Neither is a metric: the exact value matters, so `LongAdder` would be wrong. It is right for write-heavy counters read only by a metrics scrape (Micrometer's counters use `DoubleAdder` internally).

9.6 The Executor framework

Creating a platform thread per task costs ~1 MB of reserved stack, a kernel thread and tens of microseconds; unbounded thread creation eventually fails with `OutOfMemoryError: unable to create native thread`. Executors decouple **task submission** from **thread management**.

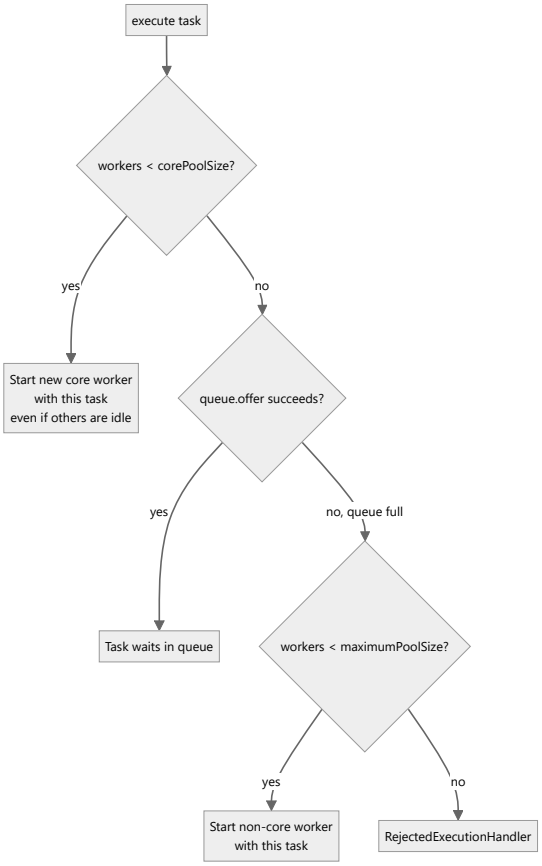


`ThreadPoolExecutor` parameters

```
new ThreadPoolExecutor(
    4,                                  // corePoolSize: threads kept even when idle
    16,                                 // maximumPoolSize: hard ceiling
    60, TimeUnit.SECONDS,               // keepAliveTime for threads above core
    new ArrayBlockingQueue<>(500),      // workQueue: BOUNDED
    Thread.ofPlatform().name("export-", 0).factory(),   // threadFactory: names!
    new ThreadPoolExecutor.CallerRunsPolicy());         // RejectedExecutionHandler
```

The task-submission flow

This is the single most asked executor question, and most candidates get it wrong: **the pool does not grow beyond core until the queue is full.**

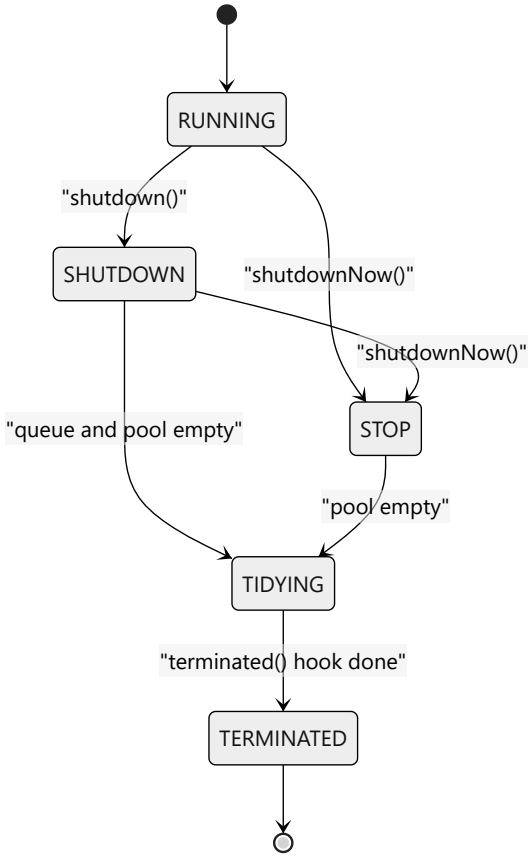


Consequences:

- With an **unbounded** `LinkedBlockingQueue`, `offer` never fails, so `maximumPoolSize` is meaningless — the pool never exceeds core size, and the queue grows until OOM.
- With a `SynchronousQueue` (zero capacity), `offer` succeeds only if a worker is idle and waiting, so every burst creates threads up to max — that is how `newCachedThreadPool` works.
- Non-core threads die after `keepAliveTime` idle; core threads too if `allowCoreThreadTimeOut(true)`.
- `prestartAllCoreThreads()` warms the pool before traffic.

Internally, `ThreadPoolExecutor` packs run state and worker count into one `AtomicInteger ctl` (upper 3 bits state, lower 29 bits count) so both can be checked and changed with one CAS. Each `Worker` is itself an AQS (a

non-reentrant lock held while running a task, so `shutdown()` can tell idle workers from busy ones and only interrupt the idle ones).



Rejection policies

Policy	Behaviour	Use when
`AbortPolicy` (default)	Throws `RejectedExecutionException`	Caller must know and handle overload (return 503/429)
`CallerRunsPolicy`	Submitting thread runs the task itself	Natural **back-pressure**: the producer slows down. Beware: if the caller is a Tomcat request thread, the request now takes longer
`DiscardPolicy`	Silently drops	Almost never — invisible data loss
`DiscardOldestPolicy`	Drops the head of the queue, retries	Latest-value-wins work such as UI refresh or sampling
Custom	Log, metric, persist to DB, retry later	Durable work must never be lost

Sizing a pool

- **CPU-bound** work: `threads ≈ Ncpu` (or `Ncpu + 1` to cover the occasional page fault). More threads only add context switching.
- **I/O-bound** work (Brian Goetz, *Java Concurrency in Practice*): `threads = Ncpu × Utarget × (1 + W/C)`, where W/C is wait time over compute time. 8 cores, 100% target, a call that waits 90 ms and computes 10 ms → 8 × 1 × (1 + 9) = 80 threads.

- **Little's law** cross-check: required concurrency = throughput × latency. 400 req/s × 0.2 s = 80 in flight.
- The real ceiling is usually **downstream**: 80 threads hitting a database with a 20-connection Hikari pool means 60 of them queue on the pool. Size to the narrowest resource, then measure (pool active count, queue size, task wait time) and adjust.
- `Runtime.availableProcessors()` respects container CPU limits since JDK 10 (cgroup aware).

`Executors` factory pitfalls

Factory	Hidden configuration	Failure mode
`newFixedThreadPool(n)`	core = max = n, **unbounded** `LinkedBlockingQueue`	Slow consumers → queue grows → `OutOfMemoryError`
`newSingleThreadExecutor()`	1 thread, unbounded queue	Same OOM; also not reconfigurable
`newCachedThreadPool()`	core 0, max `Integer.MAX_VALUE`, `SynchronousQueue`	Burst → thousands of threads → `unable to create native thread`
`newScheduledThreadPool(n)`	Unbounded `DelayedWorkQueue`, max ignored	Queue growth; long tasks delay others
`newWorkStealingPool()`	`ForkJoinPool`, async mode	Not for blocking I/O
`newVirtualThreadPerTaskExecutor()` (21)	New virtual thread per task	No pooling; limit concurrency with a `Semaphore`

> **Trap:** "We used `Executors.newFixedThreadPool(10)` for email notifications; the SMTP server slowed down, 2 million tasks queued, heap filled, the service died." Production pools should be built with an explicit **bounded** queue, named threads and a deliberate rejection policy. Many coding standards ban the `Executors` factories outright.

`submit` vs `execute` and lost exceptions

`execute(Runnable)` lets an exception propagate: the worker thread dies (the pool replaces it) and the `UncaughtExceptionHandler` prints it. `submit()` wraps the task in a `FutureTask`, which **catches the exception and stores it in the Future**. If nobody calls `get()`, the exception disappears silently. Either call `get()`, catch inside the task, or override `afterExecute(Runnable r, Throwable t)` and unwrap the `Future`.

`ScheduledExecutorService`

```
ScheduledExecutorService ses = Executors.newScheduledThreadPool(2);
ses.scheduleAtFixedRate(this::relay, 0, 1, TimeUnit.SECONDS);      // start-to-start period
ses.scheduleWithFixedDelay(this::sweep, 0, 5, TimeUnit.SECONDS);   // end-to-start delay
```

- **Fixed rate**: runs every period measured from start times. If a run takes longer than the period, the next starts late — runs of the same task **never overlap**.
- **Fixed delay**: waits the delay after each run ends. Safer for jobs whose duration varies.

- If a run throws, **all future runs are silently cancelled** (the exception is captured in the returned `ScheduledFuture`). Always wrap the body in `try/catch (Exception e)`.
- `ScheduledExecutorService` uses relative time (`System.nanoTime`); it is not a cron engine and does not survive restarts.

9.7 `Future` and `CompletableFuture`

`Future<T>` (Java 5) is a handle to a result: `get()` blocks, `get(timeout)`, `cancel(mayInterrupt)`, `isDone()`. It cannot be composed, cannot be completed manually, and has no callbacks — you end up blocking a thread per future. (Java 19 added `resultNow()`, `exceptionNow()` and `state()`.)

`CompletableFuture<T>` (Java 8) implements both `Future` and `CompletionStage`: a pipeline of stages, each triggered when its input completes.

Method	Input → output	Analogy
`thenApply(fn)`	`T → U`	`map`
`thenCompose(fn)`	`T → CompletionStage<U>`	`flatMap` (avoid `CF<CF<U>>`)
`thenCombine(other, bifn)`	`T, U → V` (both)	zip
`thenAccept` / `thenRun`	consume / just run	side effect
`allOf(cfs...)`	`CF<Void>` when all complete	barrier
`anyOf(cfs...)`	`CF<Object>` first to complete	race
`exceptionally(fn)`	`Throwable → T` on failure only	catch
`handle(bifn)`	`(T, Throwable) → U` always	catch + map
`whenComplete(biconsumer)`	observe, result unchanged	finally
`orTimeout` / `completeOnTimeout` (9)	fail / default after timeout	timeout
`exceptionallyCompose` (12)	`Throwable → CompletionStage<T>`	async fallback

Which thread runs a stage?

- `supplyAsync(supplier)` with no executor runs on `ForkJoinPool.commonPool()` (parallelism `Ncpu − 1`; if that is ≤ 1, a new thread per task).
- **Non-async** `thenApply(fn)`: runs on the thread that completes the previous stage, or **on the calling thread immediately** if the previous stage is already complete. Cheap, but unpredictable.
- `thenApplyAsync(fn)` submits `fn` to the common pool; `thenApplyAsync(fn, executor)` to your executor.

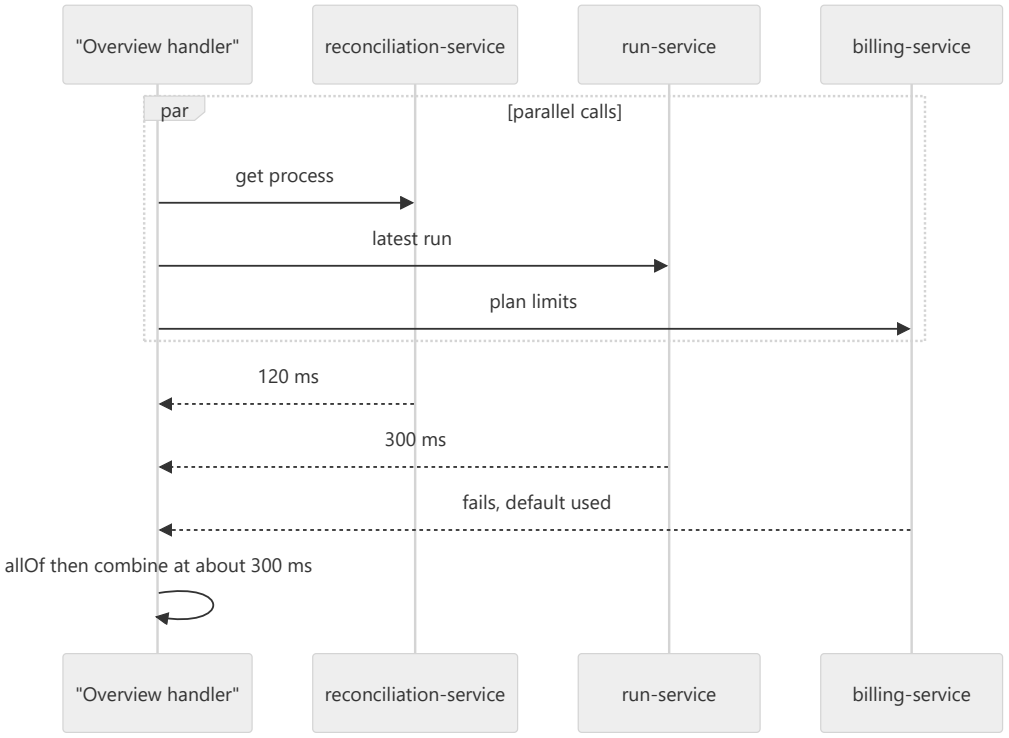
> **Trap:** blocking I/O (HTTP, JDBC) in `supplyAsync` without an executor occupies the tiny common pool shared by every parallel stream and every other async call in the JVM. Always pass a dedicated executor for I/O (or a virtual-thread executor on Java 21).

Fan-out to three services and combine

```
ExecutorService io = Executors.newVirtualThreadPerTaskExecutor();   // or a bounded pool

CompletableFuture<Recon> recon = CompletableFuture
        .supplyAsync(() -> reconClient.get(id), io)
        .orTimeout(2, TimeUnit.SECONDS);
CompletableFuture<RunSummary> run = CompletableFuture
        .supplyAsync(() -> runClient.latest(id), io)
        .completeOnTimeout(RunSummary.unknown(), 2, TimeUnit.SECONDS);   // degrade gracefully
CompletableFuture<PlanLimits> plan = CompletableFuture
        .supplyAsync(() -> billingClient.limits(orgId), io)
        .exceptionally(ex -> PlanLimits.defaults());                     // fallback

ReconOverview overview = CompletableFuture.allOf(recon, run, plan)
        .thenApply(v -> new ReconOverview(recon.join(), run.join(), plan.join()))
        .join();    // total latency ≈ slowest call, not the sum
```



Interview tip: the three clients above are illustrative, but the shape applies to any aggregation over service-to-service calls (ours go through `InternalClient`). Name the three properties out loud: the slow call bounds the latency, the optional call has a fallback, and each call has its own timeout so one sick service cannot hang the page.

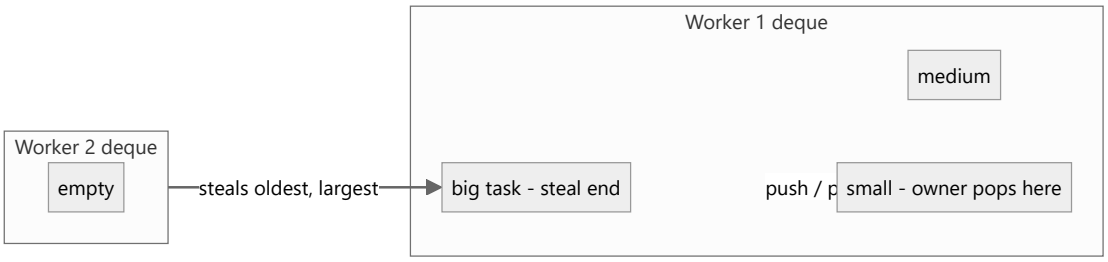
Exceptions in a pipeline

An exception in a stage completes that stage exceptionally; every dependent stage is skipped (completed exceptionally with a `CompletionException` wrapping the cause) until an `exceptionally` / `handle` / `whenComplete` is reached. `join()` throws unchecked `CompletionException`; `get()` throws checked `ExecutionException`. `cancel(true)` does **not** interrupt the running task —

`CompletableFuture` has no reference to the thread — it only completes the future with `CancellationException`.

9.8 `ForkJoinPool` and work stealing

`ForkJoinPool` is built for divide-and-conquer: a task splits into subtasks (`fork`), runs one itself, and `join`s the others. Each worker owns a **double-ended queue**. It pushes and pops its own tasks at one end (LIFO — the most recently split, smallest, cache-hot task); an idle worker **steals** from the *other* end of a victim's deque (FIFO — the oldest and therefore largest chunk of work). Owner and thief rarely touch the same end, so contention is low.




```
class SumTask extends RecursiveTask<Long> {
    private final long[] a; private final int lo, hi;
    SumTask(long[] a, int lo, int hi) { this.a = a; this.lo = lo; this.hi = hi; }
    @Override protected Long compute() {
        if (hi - lo <= 10_000) { long s = 0; for (int i = lo; i < hi; i++) s += a[i]; return s; }
        int mid = (lo + hi) >>> 1;
        SumTask left = new SumTask(a, lo, mid);
        left.fork();                                    // push left half for stealing
        long right = new SumTask(a, mid, hi).compute(); // do right half here
        return right + left.join();                      // join: helps or waits
    }
}
long total = ForkJoinPool.commonPool().invoke(new SumTask(data, 0, data.length));
```

Notes: `join()` does not simply block — the worker *helps* by running other tasks while it waits. The **common pool** is shared by parallel streams and `CompletableFuture` async defaults; tune it with `-Djava.util.concurrent.ForkJoinPool.common.parallelism=N`, or run a parallel stream inside your own pool with `pool.submit(() -> list.parallelStream()...).get()`. Blocking inside fork/join tasks starves the pool unless you use `ForkJoinPool.ManagedBlocker`, which lets the pool add a compensating thread.

9.9 Synchronizers

Synchronizer	What it does	Reusable	Typical use
`CountDownLatch(n)`	`await()` blocks until `countDown()` called n times	No, one-shot	Wait for N services to start, or N tasks to finish
`CyclicBarrier(n, action)`	n threads `await()` each other, then the barrier action runs, then all proceed	Yes	Iterative parallel algorithms, simulations
`Semaphore(permits)`	`acquire()` / `release()` permits	Yes	Limit concurrency to a resource
`Phaser`	Dynamic parties, numbered phases	Yes	Multi-phase work where parties join/leave
`Exchanger<V>`	Two threads swap objects at a rendezvous	Yes	Double-buffering between a producer and consumer

```
// CountDownLatch: wait for three parallel loaders
CountDownLatch done = new CountDownLatch(3);
for (Loader l : loaders) pool.execute(() -> { try { l.load(); } finally { done.countDown(); } });
if (!done.await(30, TimeUnit.SECONDS)) throw new TimeoutException("loaders too slow");

// CyclicBarrier: each round all workers wait, then the action merges results
CyclicBarrier barrier = new CyclicBarrier(4, () -> mergePartialResults());
// worker loop: computeChunk(); barrier.await();   // BrokenBarrierException if one fails/times
out

// Phaser: parties can register and deregister dynamically
Phaser phaser = new Phaser(1);                      // the coordinator
for (Task t : tasks) { phaser.register(); pool.execute(() -> { t.phase1();
phaser.arriveAndAwaitAdvance(); t.phase2(); phaser.arriveAndDeregister(); }); }
phaser.arriveAndDeregister();                       // coordinator steps out
```

`Semaphore` deserves its own note: permits are **not owned** — any thread can `release()`, and releasing more than acquired *increases* the permit count (a bug, not an error). Always release in `finally`, and only if the acquire succeeded.

Rate limiting / bulkheading with a `Semaphore`

```
@Component
class StatementExtractionGate {
    private final Semaphore permits = new Semaphore(5, true);   // max 5 concurrent LLM calls

    ExtractionResult extract(Upload upload) throws InterruptedException {
        if (!permits.tryAcquire(2, TimeUnit.SECONDS)) {
            throw new TooBusyException("AI extraction busy, retry shortly");   // maps to 429/503
        }
        try {
            return llmClient.complete(buildRequest(upload));
        } finally {
            permits.release();
        }
    }
}
```

A semaphore limits **concurrency** (how many at once), not **rate** (how many per second). For rate you need a token bucket or sliding window.

> **RefinePoint:** both shapes exist in the platform. The api-gateway's `RateLimitFilter` is a token bucket per IP and per caller held in a `ConcurrentHashMap` (requests per second → 429), and `SlidingWindowLimiter` in ryfn-platform limits calls per time window. A semaphore-style bulkhead is the right tool for protecting a slow, expensive resource such as an LLM provider or the DuckDB engine, where the problem is *simultaneous* work, not request rate.

9.10 Graceful shutdown

```
void stop(ExecutorService pool) {
    pool.shutdown();                                        // 1. stop accepting, finish queued work
    try {
        if (!pool.awaitTermination(30, TimeUnit.SECONDS)) {
            List<Runnable> never = pool.shutdownNow();   // 2. interrupt running, return queued
            log.warn("{} tasks dropped", never.size());
            if (!pool.awaitTermination(10, TimeUnit.SECONDS)) log.error("pool did not
terminate");
        }
    } catch (InterruptedException e) {
        pool.shutdownNow();
        Thread.currentThread().interrupt();              // preserve interrupt status
    }
}
```

- `shutdown()` is non-blocking; queued tasks still run.
- `shutdownNow()` interrupts workers — tasks that ignore interruption keep running. Write tasks that check `Thread.interrupted()` or use interruptible blocking calls.
- Java 19+: `ExecutorService` is `AutoCloseable`; `close()` = `shutdown()` + wait until done, so try-with-resources works.
- Worker threads should be non-daemon if work must finish, and you need a shutdown hook or Spring lifecycle to call the above. Spring's `ThreadPoolTaskExecutor` does it when you set `setWaitForTasksToCompleteOnShutdown(true)` and `setAwaitTerminationSeconds(n)`; Spring Boot's `server.shutdown=graceful` with `spring.lifecycle.timeout-per-shutdown-phase` drains HTTP requests first.

> **RefinePoint:** our work queues live in PostgreSQL, so a hard kill never loses work. `FOR UPDATE SKIP LOCKED` is held only for the short claim transaction, which commits `status = 'RUNNING'` with the instance name; each instance then writes a heartbeat, and any row whose heartbeat is older than two minutes goes back to `QUEUED` (or to `FAILED` after its last attempt). On a clean stop, `RunWorker`'s `@PreDestroy` re-queues its runs at once and calls `pool.shutdownNow()`. Graceful shutdown is then about not wasting work, not about correctness — a much easier problem.

9.11 Spring `@Async`, `@Scheduled` and their executors

`@EnableAsync` makes Spring wrap beans with `@Async` methods in a proxy that submits the call to a `TaskExecutor`.

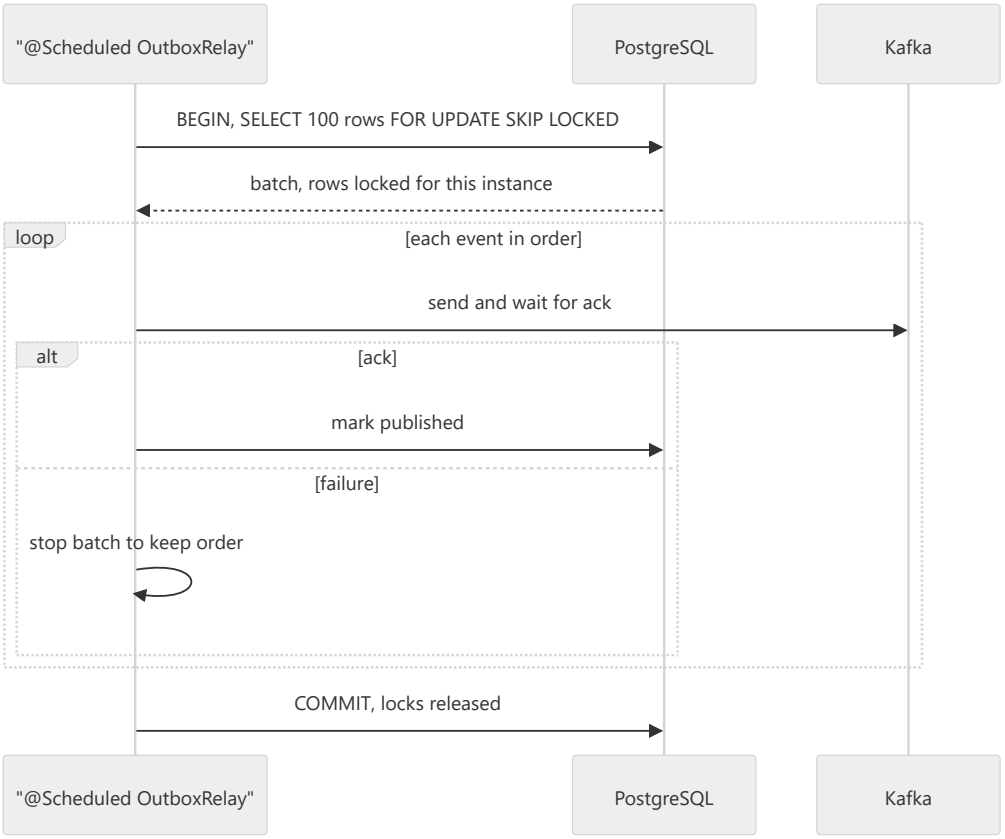
- **Which executor?** If there is a unique `TaskExecutor` bean, or one named `taskExecutor`, it is used. Spring Boot auto-configures `applicationTaskExecutor`, a `ThreadPoolTaskExecutor` with **core 8, unbounded queue**, so max size is effectively unused (tune with `spring.task.execution.pool.core-size`, `queue-capacity`, `max-size`). With `spring.threads.virtual.enabled=true` (Boot 3.2+) it becomes a `SimpleAsyncTaskExecutor` on virtual threads. Plain Spring without Boot falls back to `SimpleAsyncTaskExecutor` — a new thread per call, no pool.
- **Choose per method**: `@Async("exportExecutor")`.
- **Return types**: `void`, `Future`, `CompletableFuture`. Exceptions from `void` methods go to `AsyncUncaughtExceptionHandler` (default: log). From `CompletableFuture`, they complete the future.
- **Self-invocation does not work**: calling `this.sendAsync()` from the same bean bypasses the proxy and runs synchronously.
- **Context propagation**: `ThreadLocal`s (MDC trace id, `SecurityContextHolder`, tenant context) do not cross to the pool thread. Use a `TaskDecorator` that captures and restores them.
- `@Scheduled` uses a separate `ThreadPoolTaskScheduler` whose **default pool size is 1** in Spring Boot (`spring.task.scheduling.pool.size`). One slow scheduled method delays all the others.

```
@Bean(name = "exportExecutor")
ThreadPoolTaskExecutor exportExecutor() {
    var ex = new ThreadPoolTaskExecutor();
    ex.setCorePoolSize(4);
    ex.setMaxPoolSize(8);
    ex.setQueueCapacity(100);                       // bounded!
    ex.setThreadNamePrefix("export-");
    ex.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
    ex.setTaskDecorator(task -> {                   // carry MDC and tenant across threads
        Map<String, String> mdc = MDC.getCopyOfContextMap();
        return () -> {
            if (mdc != null) MDC.setContextMap(mdc);
            try { task.run(); } finally { MDC.clear(); }
        };
    });
    ex.setWaitForTasksToCompleteOnShutdown(true);
    ex.setAwaitTerminationSeconds(30);
    return ex;
}
```

9.12 Real scenarios from RefinePoint

Scheduled outbox relay

The business transaction writes to `outbox_events`; a scheduled relay publishes to Kafka. The concurrency design avoids every in-memory primitive in this chapter — because the coordination point is the database.



- `SKIP LOCKED` lets several instances relay concurrently without picking the same row; no leader election, no `ReentrantLock`.
- The scheduler pool size matters: with the default single scheduler thread, a stuck Kafka send would delay every other `@Scheduled` method in that service. That is why the relay waits a bounded time for each ack (`kafka.send(...).get(10, TimeUnit.SECONDS)`); also size `spring.task.scheduling.pool.size` for the number of scheduled jobs.
- `fixedDelay` rather than `fixedRate`: a slow batch should not cause the next run to start immediately. A full batch of 100 loops straight into the next one, so a backlog drains without waiting for the tick.
- Consumers are idempotent (`ConsumedEvents.firstTime`), so a relay crash between "sent" and "marked published" only causes a duplicate, never a loss.

Export worker

`ExportWorker` in run-service claims rows from `run_exports` with `FOR UPDATE SKIP LOCKED` and writes large files. The design questions are JUC questions:

```
private final Set<UUID> working = ConcurrentHashMap.newKeySet();
private volatile boolean stopping;
// pool = Executors.newFixedThreadPool(workers, namedDaemonFactory);  exports.workers defaults to
2

@Scheduled(fixedDelayString = "${exports.poll-ms:2000}")
public void poll() {
    while (!stopping && working.size() < workers) {   // claim only what we can run now
        UUID id = claim();                            // short tx: SKIP LOCKED, set RUNNING +
instance
        if (id == null) return;
        working.add(id);
        pool.execute(() -> { try { prepare(id); } finally { working.remove(id); } });
```

```
    }
}
```

- **How many concurrent exports per instance?** Few (default 2). Exports stream rows through a writer and memory is the binding resource, so we do not size for I/O wait. Scaling is "add an instance", since the DB queue distributes work.
- **Isn't `newFixedThreadPool` banned?** Its unbounded queue is the danger, but here it cannot grow: the poller claims a row only while `working.size() < workers`, so a task is never submitted without a free thread. The bound is enforced before submission, and the real queue is the table.
- **Shutdown**: `@PreDestroy` sets `stopping` and calls `pool.shutdownNow()`. The writer checks `stopping || Thread.currentThread().isInterrupted()` every 2,000 rows and gives up, so interruption really stops it. A separate `@Scheduled` heartbeat re-queues exports of an instance that stopped reporting for two minutes, up to three attempts. The final `UPDATE ... WHERE id = ? AND status = 'RUNNING' AND instance = ?` makes a worker that was declared dead harmless: it updates zero rows and deletes its file.

> **Interview tip:** when asked "how would you build a job processor", the senior answer starts with *where the queue lives*. An in-memory `BlockingQueue` loses work on crash and cannot be shared across instances; a database or Kafka queue survives both. JUC primitives then handle only the local, per-instance concurrency.

9.13 Interview questions and answers

Q1. What is the difference between `synchronized` and `ReentrantLock`?

Both give mutual exclusion, reentrancy and the same happens-before visibility; `ReentrantLock` adds `tryLock` (with or without timeout), interruptible acquisition, optional fairness, multiple `Condition`s and introspection (`getQueueLength`, `isHeldByCurrentThread`). The cost is that you must unlock in `finally` yourself. `synchronized` is a JVM monitor in the object header (thin lock via CAS, inflated to an `ObjectMonitor` under contention), `ReentrantLock` is pure Java on AQS. Performance is similar today; choose `ReentrantLock` when you need one of its features, and — on Java 21 with virtual threads — to avoid pinning carriers during blocking work inside the critical section.

Q2. Why must `lock()` be called before the `try` block, not inside it?

If `lock()` is inside `try` and throws (for example an `Error`, or `lockInterruptibly` being interrupted), the `finally` runs `unlock()` on a lock you never acquired and throws `IllegalMonitorStateException`, masking the original exception. Acquire, then `try { ... } finally { unlock(); }`.

Q3. What is lock fairness, and why is non-fair the default?

A fair lock grants access in FIFO order of arrival; a non-fair lock lets an arriving thread grab a just-released lock ahead of queued threads. Non-fair is the default because waking a parked thread takes microseconds; letting a running thread take the lock in that gap keeps the CPU busy and gives far higher throughput. Fair locks prevent starvation and make latency more predictable at the cost of throughput. Note `tryLock()` without timeout ignores fairness.

Q4. What does `lockInterruptibly()` give you that `lock()` does not?

A thread waiting in `lockInterruptibly()` responds to `Thread.interrupt()` by throwing `InterruptedException` and leaving the wait queue. `lock()` (and `synchronized`) ignores interrupts while waiting and only re-asserts the interrupt flag after acquiring. Use it for anything that must be cancellable: tasks in executors that may receive `shutdownNow()`, or request handling with deadlines.

Q5. How do `Condition.await()` and `signal()` work, and why is `await` always in a loop?

`await()` must be called while holding the lock; it atomically releases the lock (saving the reentrant hold count), parks the thread on the condition queue, and on wake-up re-acquires the lock before returning. `signal()` moves one waiter from the condition queue to the lock's sync queue. It is in a `while` loop because of spurious wake-ups and because between the signal and re-acquiring the lock, another thread may have changed the state (consumed the item). Always re-check the predicate.

Q6. Explain AbstractQueuedSynchronizer in two minutes.

AQS is a framework for building blocking synchronizers out of one `volatile int state` and a FIFO queue of waiting threads. Subclasses define what the state means and implement `tryAcquire/tryRelease` (exclusive) or `tryAcquireShared/tryReleaseShared` (shared) using CAS on the state. AQS handles everything else: if the try fails, the thread is wrapped in a node, appended to a CLH-style doubly linked queue with a CAS on `tail`, and parked with `LockSupport.park`. On release, the head's successor is unparked and retries. It supports timeouts, interrupts, cancellation and condition queues. `ReentrantLock`, `Semaphore`, `CountDownLatch`, `ReentrantReadWriteLock` and the `ThreadPoolExecutor.Worker` are all built on it.

Q7. What does `state` mean in each AQS-based class?

`ReentrantLock`: hold count, 0 means free, owner thread stored separately. `Semaphore`: number of available permits. `CountDownLatch`: remaining count; `await` succeeds when it is 0. `ReentrantReadWriteLock`: upper 16 bits = total read holds, lower 16 bits = write reentrancy count (so each is limited to 65,535). `ThreadPoolExecutor.Worker`: 0 unlocked/idle, 1 running a task, −1 before start.

Q8. Why does AQS use a CLH-style queue instead of `wait/notifyAll`?

Each waiter waits on its own node and a release wakes exactly one successor, so there is no thundering herd where all waiters wake up and fight. Enqueueing is lock-free (one CAS on tail), the FIFO order makes fairness cheap to implement, and per-node status lets cancelled waiters be skipped. It scales to many waiters with O(1) work per release.

Q9. What is the difference between exclusive and shared mode in AQS?

Exclusive mode: only one thread can acquire (`ReentrantLock`, write lock). Shared mode: several can acquire at once (`Semaphore` permits, read lock, an open `CountDownLatch`). In shared mode, a successful acquire *propagates* — the woken thread wakes its successor too if more can succeed, which is how all latch waiters are released when the count reaches zero.

Q10. What is `LockSupport.park / unpark`?

The low-level blocking primitive under AQS. `park()` blocks the current thread unless a permit is available; `unpark(t)` makes a permit available for `t` (at most one, not counted). Because the permit is remembered, `unpark` before `park` is not lost — unlike `notify` before `wait`. `park` can also return spuriously or on interrupt (without throwing), so callers loop. Parked threads show as `WAITING (parking)` in a thread dump, with the blocker object if `park(blocker)` was used.

Q11. When would you use `ReentrantReadWriteLock`, and when is it slower than a plain lock?

When reads dominate writes heavily and the read critical section is long (e.g., a large in-memory configuration map read on every request, rebuilt rarely). It is slower when critical sections are tiny or writes are frequent: every read acquire is a CAS on the shared state word, which becomes a cache-line hotspot across cores, and bookkeeping per reader (thread-local hold counts) adds cost. For read-mostly *immutable snapshots*, a `volatile` reference to an immutable map (copy-on-write) beats both.

Q12. Can you upgrade a read lock to a write lock in `ReentrantReadWriteLock`?

No — it deadlocks: the write lock waits for all read holds to be released, including the thread's own. Downgrading is supported: acquire the write lock, acquire the read lock, release the write lock. For upgrade-like behaviour use `StampedLock.tryConvertToWriteLock(stamp)`, which succeeds if conversion is possible and returns 0 otherwise.

Q13. What is `StampedLock`, and what is an optimistic read?

A Java 8 lock with three modes — write, read and optimistic read — where every acquire returns a `long` stamp. An optimistic read (`tryOptimisticRead`) acquires nothing; it records the current version. You read fields into locals, then `validate(stamp)` : if no write happened, your reads are consistent; otherwise you retry with a real read lock. Because optimistic readers never write to shared memory, it scales far better for short, hot reads. Caveats: not reentrant, no conditions, not interruptible in the basic methods, and it is easy to misuse.

Q14. What is CAS and how is it implemented in hardware?

Compare-and-swap atomically compares a memory location with an expected value and, only if they match, writes a new value, returning success. On x86 it is `LOCK CMPXCHG` ; the lock prefix gives the core exclusive ownership of the cache line for the operation and acts as a full fence. ARM uses `LDXR/STXR` (load-exclusive/store-exclusive) loops or the ARMv8.1 `CAS` instruction. Java reaches it through `VarHandle.compareAndSet` and internal `Unsafe` methods, which the JIT compiles to the single instruction.

Q15. Is CAS always faster than locking?

No. Uncontended CAS is cheaper than a lock (no OS involvement). Under heavy contention, CAS loops retry repeatedly, each failure costs a cache-line transfer between cores, and throughput can be worse than a lock that parks losers. That is why `LongAdder` spreads contention over cells and why some highly contended

structures still use locks. CAS is also limited to one variable; multi-variable invariants need a lock or an immutable object swapped through `AtomicReference`.

Q16. What is the ABA problem and how do you solve it?

A CAS succeeds because the value is A again, although it changed A → B → A in between, so the assumption behind the CAS is wrong. It matters for lock-free structures that reuse nodes and for values whose history matters. Fix with a version stamp: `AtomicStampedReference` (reference + int stamp, both compared) or `AtomicMarkableReference` (reference + boolean). In Java, GC prevents the classic C/C++ variant (memory freed and reused while referenced), so ABA in Java usually needs explicit object reuse.

Q17. `AtomicLong` vs `LongAdder` — which and why?

`AtomicLong` for values you need exactly and atomically (IDs, CAS-based state, `incrementAndGet` results you branch on). `LongAdder` for high-contention counters you only read occasionally (metrics, hit counts): writes go to per-thread-hashed, cache-line-padded cells, so they scale with cores, but `sum()` is not an atomic snapshot and there is no compare-and-set. Under low contention both behave about the same, since `LongAdder` only creates cells after a CAS on its base fails.

Q18. What is false sharing and how does `@Contended` help?

Two variables written by different threads that sit on the same 64-byte cache line force the line to bounce between cores even though the threads never touch the same variable.
`@jdk.internal.vm.annotation.Contended` pads a field or class to its own cache line (enabled for JDK classes; for user code needs `-XX:-RestrictContended`). `LongAdder` cells and `Thread`'s random-probe fields use it; manual padding with dummy `long` fields was the older trick.

Q19. What do `updateAndGet` and `accumulateAndGet` do, and what is the catch?

They apply a function in a CAS retry loop: read, compute, CAS, retry on failure. The catch: the function may run multiple times under contention, so it must be pure — no logging, no I/O, no incrementing other counters inside it.

Q20. What is `AtomicIntegerFieldUpdater` for?

CAS on a `volatile int` field of existing objects without wrapping each in an `AtomicInteger`. If you have millions of nodes each needing an atomic status, an updater saves one object (16+ bytes) per node. JDK code like `ConcurrentHashMap` and `FutureTask` used updaters; Java 9+ code uses `VarHandle`, which does the same with fewer restrictions.

Q21. Explain all `ThreadPoolExecutor` constructor parameters.

`corePoolSize` threads are kept alive even when idle; `maximumPoolSize` is the ceiling; `keepAliveTime` is how long threads above core may idle before dying (also core threads with `allowCoreThreadTimeOut`); `workQueue` holds tasks waiting for a thread; `threadFactory` creates threads (name them, set daemon status and uncaught-exception handler); `handler` decides what happens when both the queue and max threads are full.

Q22. Walk through what happens when a task is submitted to a `ThreadPoolExecutor`.

If fewer than core threads are running, a new thread is started with the task as its first task — even if existing core threads are idle. Otherwise the task is offered to the queue. If the queue refuses (full), a new non-core thread is started if below max. If at max, the rejection handler runs. There is also a re-check after queuing: if the pool was shut down meanwhile the task is removed and rejected, and if no workers exist one is started.

Q23. Why doesn't my pool grow past `corePoolSize` even though tasks are waiting?

Because the queue is unbounded (or very large). Threads above core are only created when `offer` to the queue fails. With `LinkedBlockingQueue()` (capacity `Integer.MAX_VALUE`), it never fails, so `maximumPoolSize` is dead configuration. Spring Boot's default `applicationTaskExecutor` has exactly this shape. Fix: bounded queue, or set core = max and accept queuing, or use `allowCoreThreadTimeOut` with core = max for elasticity.

Q24. Which queue types can a `ThreadPoolExecutor` use, and what do they imply?

`SynchronousQueue`: no storage, hand-off only — pool grows quickly to max, then rejects; good for bursty, short tasks with a sensible max. `ArrayBlockingQueue(n)`: bounded, one lock, predictable memory — the default choice. `LinkedBlockingQueue(n)`: bounded if given capacity, separate put/take locks, higher throughput; unbounded without capacity (dangerous). `PriorityBlockingQueue`: unbounded, orders by priority (tasks must be comparable; `submit` wraps them in `FutureTask`, losing comparability unless you override `newTaskFor`). `DelayedWorkQueue`: used by the scheduled executor.

Q25. Explain the four rejection policies and which you'd use for what.

`AbortPolicy` throws `RejectedExecutionException` — the caller sees overload and can return 503/429. `CallerRunsPolicy` makes the submitting thread run the task — built-in back-pressure that slows producers. `DiscardPolicy` silently drops — rarely acceptable. `DiscardOldestPolicy` drops the oldest queued task and retries — fine for "latest value wins". For durable work, a custom handler persists the task (database row, Kafka) for later retry and increments a metric.

Q26. How do you size a thread pool?

CPU-bound: about the number of cores; more only adds context switches. I/O-bound: `Ncpu × U × (1 + W/C)`, e.g., 8 cores × (1 + 90/10) = 80. Cross-check with Little's law (throughput × latency = concurrency needed). Then respect downstream limits — threads beyond the DB connection pool size just wait on Hikari. Finally measure: active count, queue depth, task wait time and latency percentiles, and adjust. In containers, `availableProcessors()` reflects the CPU quota.

Q27. Why do many coding standards ban the `Executors` factory methods?

They hide dangerous defaults: `newFixedThreadPool` and `newSingleThreadExecutor` use unbounded queues (OOM under slow consumers), `newCachedThreadPool` allows `Integer.MAX_VALUE` threads (native thread exhaustion), and `newScheduledThreadPool` has an unbounded delay queue. Constructing `ThreadPoolExecutor` explicitly forces the developer to choose a bounded queue, a named thread factory and

a rejection policy. (`newVirtualThreadPerTaskExecutor` is fine but needs a concurrency limit for downstream resources.)

Q28. What is the difference between `execute()` and `submit()`?

`execute(Runnable)` returns nothing; an exception propagates to the worker, kills that worker thread (the pool replaces it) and is printed by the uncaught-exception handler. `submit()` accepts `Runnable` or `Callable`, returns a `Future`, and the exception is captured inside the future — invisible unless someone calls `get()`. Many "the task silently did nothing" bugs are `submit` plus a discarded future.

Q29. How do you catch exceptions from tasks in a thread pool?

Options: wrap the task body in try/catch and log; keep the `Future` and call `get()`; override `afterExecute(Runnable r, Throwable t)` — for `submit`ted tasks `t` is null and you must check `((Future<?>) r).isDone()` and call `get()` to extract it; set an `UncaughtExceptionHandler` via the thread factory (only for `execute`). For `CompletableFuture`, attach `exceptionally` / `whenComplete`.

Q30. `scheduleAtFixedRate` vs `scheduleWithFixedDelay`?

Fixed rate schedules runs at `initialDelay + n × period` from the start times; if a run overruns, the next starts immediately after it ends (never concurrently). Fixed delay waits the delay after each run completes, so the gap is constant. Use fixed delay for polling jobs whose duration varies, such as an outbox relay or a job claimer. In both, an uncaught exception cancels all future executions silently — wrap the body in try/catch.

Q31. What is the difference between `shutdown()`, `shutdownNow()` and `awaitTermination()`?

`shutdown()` stops accepting tasks but lets running and queued tasks finish; it returns immediately. `shutdownNow()` also drains the queue (returning the unstarted tasks) and interrupts running workers. `awaitTermination(timeout)` blocks until all tasks complete or the timeout elapses. The graceful pattern is shutdown → await → shutdownNow → await, restoring the interrupt flag if interrupted. On Java 19+, `close()` does shutdown plus waiting.

Q32. How does `ThreadPoolExecutor` store its state?

In one `AtomicInteger ctl`: the top 3 bits hold run state (RUNNING, SHUTDOWN, STOP, TIDYING, TERMINATED) and the lower 29 bits the worker count. Packing both into one word lets the executor check "still running and below max" and increment the count in a single CAS. Workers are held in a `HashSet` guarded by a `mainLock`.

Q33. What is the difference between `Future` and `CompletableFuture`?

`Future` is a read-only handle: block with `get()`, poll `isDone()`, `cancel()`. You cannot attach callbacks, combine futures, or complete it yourself. `CompletableFuture` can be completed manually (`complete`, `completeExceptionally`), and it is a `CompletionStage` with non-blocking composition: transform (`thenApply`), chain async calls (`thenCompose`), combine (`thenCombine`, `allOf`, `anyOf`), handle errors

(`exceptionally` , `handle`) and time out (`orTimeout`). It lets you write a whole async workflow without blocking a thread per step.

Q34. `thenApply` vs `thenCompose` ?

`thenApply` maps the result with a synchronous function `T → U` . `thenCompose` takes a function that returns another `CompletionStage` and flattens it, so `thenCompose(id -> fetchAsync(id))` yields `CF<Result>` instead of `CF<CF<Result>>` . It is `map` vs `flatMap` . Use `thenCompose` whenever the next step is itself asynchronous.

Q35. `thenCombine` vs `allOf` ?

`thenCombine` waits for exactly two futures and combines their results with a `BiFunction` — typed and neat. `allOf` takes any number and returns `CompletableFuture<Void>` ; you then `join()` each input (already complete, so no blocking) to read results. `allOf` completes exceptionally if any input fails, but only after all have completed — it does not short-circuit. `anyOf` completes with the first result *or* exception, typed `Object` .

Q36. `exceptionally` vs `handle` vs `whenComplete` ?

`exceptionally(fn)` runs only on failure and provides a replacement value. `handle((r, ex) -> ...)` runs always and can transform success and failure into a new value. `whenComplete((r, ex) -> ...)` runs always but cannot change the result — for logging or metrics; if it throws, the stage fails with that exception only if the original succeeded. The throwable you receive is usually a `CompletionException` wrapping the real cause, so unwrap with `ex.getCause()` .

Q37. Which thread executes `thenApply` vs `thenApplyAsync` ?

`thenApply` runs on whichever thread completes the previous stage — or on the caller's thread right away if the previous stage is already complete. `thenApplyAsync(fn)` always submits to `ForkJoinPool.commonPool()` ; `thenApplyAsync(fn, executor)` to the given executor. Use the non-async variant for cheap transformations, and the async-with-executor variant for anything blocking or heavy, so you control which pool does the work.

Q38. Why is running blocking calls in `CompletableFuture.supplyAsync()` without an executor dangerous?

The default is the JVM-wide common `ForkJoinPool` with only `Ncpu − 1` threads, shared with parallel streams and every other library using async defaults. A few blocking HTTP calls fill it, and everything else that uses it stalls — a cross-cutting latency incident with no obvious cause. Always pass an I/O executor (bounded pool or virtual threads). The common pool can also be a single thread in a 2-CPU container.

Q39. How do you add a timeout to a `CompletableFuture` ?

Java 9+: `orTimeout(2, SECONDS)` completes it exceptionally with `TimeoutException` ; `completeOnTimeout(default, 2, SECONDS)` completes it with a fallback value. Neither cancels the underlying work — the HTTP call keeps running — so also configure client-level timeouts (connect/read timeouts on the

HTTP client) to actually free resources. On Java 8 you needed a scheduled executor racing `completeExceptionally`.

Q40. Does `CompletableFuture.cancel(true)` interrupt the running task?

No. A `CompletableFuture` does not know which thread is computing it; `cancel` just completes it with `CancellationException` and dependent stages fail. The task keeps running. If you need real cancellation, keep the `Future` returned by `executor.submit` (which does interrupt), or check a cancellation flag in the task. Structured concurrency (Chapter 10) solves this properly.

Q41. `join()` vs `get()` on a `CompletableFuture`?

Both block. `get()` throws checked `InterruptedException` and `ExecutionException` and has a timeout overload. `join()` throws unchecked `CompletionException` (and `CancellationException`), making it convenient in lambdas and streams, but it is not interruptible. In request code prefer `get(timeout)` or `orTimeout` + `join()` so a thread never waits forever.

Q42. How does `ForkJoinPool` work-stealing work?

Each worker has its own deque. It pushes forked subtasks and pops them from the same end (LIFO), which favours small, cache-hot tasks and avoids contention. Idle workers steal from the opposite end of other workers' deques (FIFO), taking the oldest — typically largest — tasks, so one steal moves a big chunk of work. `join()` lets the waiting worker execute other tasks instead of blocking. The result is high CPU utilisation for recursive divide-and-conquer with minimal synchronisation.

Q43. When should you not use `ForkJoinPool` or parallel streams?

For blocking I/O (workers block and the small pool starves), for small data sets (splitting overhead exceeds the gain), for sources that split badly (`LinkedList`, `Stream.iterate`), for tasks with shared mutable state or ordering requirements, and inside request-handling code of a busy server where all cores are already used by requests. Measure before parallelising.

Q44. `CountDownLatch` vs `CyclicBarrier`?

A latch is one-shot: some threads count down, others wait for zero; the counting threads do not wait. A barrier is reusable: N parties all wait for each other at the barrier, then an optional barrier action runs and all are released, and the barrier resets for the next cycle. If one party is interrupted or times out, the barrier is *broken* and all others get `BrokenBarrierException`. Latch: "wait until startup completes". Barrier: "every worker finishes iteration k before anyone starts k+1".

Q45. What is a `Phaser` and when is it better than a latch or barrier?

A reusable synchronizer with a dynamic number of parties and numbered phases. Parties `register()` and `arriveAndDeregister()` at any time; `arriveAndAwaitAdvance()` is the barrier wait; `onAdvance` can terminate it. Use it when the set of participants changes during the computation (tasks spawning subtasks that must join

the current phase), or when you need several phases without creating a new latch each time. Phasers can also be tiered into a tree to reduce contention for very many parties.

Q46. How does a `Semaphore` differ from a lock?

A semaphore holds N permits and has no owner: any thread can release, and releasing without acquiring increases the permit count. A lock has exactly one owner, is reentrant (for `ReentrantLock`) and must be released by the owner. A binary semaphore (1 permit) can act as a mutex that is released by a different thread, which is occasionally useful (hand-off) but usually a smell.

Q47. How would you limit concurrent calls to a slow third-party API to 10?

A `Semaphore(10)` around the call, with `tryAcquire(timeout)` so callers fail fast with a clear "busy" error instead of piling up, and `release()` in `finally` only after a successful acquire. Alternatively a dedicated bounded executor (10 threads, small queue) as a bulkhead, or Resilience4j's `Bulkhead`. If the limit must hold across multiple instances, an in-process semaphore is not enough — divide the budget per instance or use a distributed limiter (Redis).

Q48. What is an `Exchanger`?

A rendezvous where exactly two threads swap objects: each calls `exchange(x)` and blocks until the partner arrives, then each receives the other's object. Classic use is double buffering: the producer fills buffer A while the consumer drains buffer B, then they exchange. Rare in business code but a common "name all synchronizers" question.

Q49. How does Spring's `@Async` work and what are the pitfalls?

`@EnableAsync` registers a post-processor that proxies beans with `@Async` methods; the proxy submits the call to a `TaskExecutor` and returns immediately (`void`) or returns a `CompletableFuture`. Pitfalls: self-invocation bypasses the proxy; the method must be public and the bean Spring-managed; Spring Boot's default executor has an unbounded queue; `ThreadLocal` context (MDC, security, tenant, transaction) is not propagated — use a `TaskDecorator`; exceptions in `void` methods go only to the `AsyncUncaughtExceptionHandler`; and `@Transactional` on the same method starts a *new* transaction on the pool thread.

Q50. Which executor does `@Async` use in Spring Boot, and how do you change it?

Spring Boot auto-configures `applicationTaskExecutor` (`ThreadPoolTaskExecutor`, core 8, unbounded queue, threads named `task-N`), configured through `spring.task.execution.pool.*`. With `spring.threads.virtual.enabled=true` it is a `SimpleAsyncTaskExecutor` on virtual threads. Define your own `TaskExecutor` bean named `taskExecutor`, implement `AsyncConfigurer.getAsyncExecutor()`, or name a specific bean in the annotation: `@Async("exportExecutor")`. Separate pools per workload are a bulkhead — a flood of exports cannot starve notification sending.

Q51. Why can one `@Scheduled` method delay all the others?

Spring Boot's `ThreadPoolTaskScheduler` has a default pool size of 1, so all `@Scheduled` methods in the application share one thread. A slow or hung method (a Kafka send without timeout, a long query) delays every other scheduled task. Set `spring.task.scheduling.pool.size`, add timeouts to the work, or hand long work off to a dedicated executor from the scheduled method.

Q52. How do you propagate MDC, security context or tenant context to a thread pool?

Capture the values on the submitting thread and restore them in the worker, then clear them: a Spring `TaskDecorator` for `ThreadPoolTaskExecutor`, `DelegatingSecurityContextExecutor` for Spring Security, or Micrometer's context-propagation library for tracing. Clearing in `finally` is essential — pool threads are reused, and a leftover tenant id would leak into the next task (a cross-tenant data bug in a multi-tenant system).

Q53. How would you implement "call three services in parallel, return within 500 ms with partial results"?

Start three `supplyAsync` calls on an I/O executor, give each `completeOnTimeout(fallback, 500, ms)` or `exceptionally(fallback)`, combine with `allOf(...).thenApply(v -> merge(a.join(), b.join(), c.join()))`, and `join()`. Total latency is the slowest call capped at 500 ms. Also set HTTP client timeouts so abandoned calls do not keep consuming connections, mark which parts are degraded in the response, and record a metric per fallback. On Java 21+ with preview, `StructuredTaskScope` does this with automatic cancellation.

Q54. Design a job processor that must not lose jobs when the instance crashes.

Persist jobs in a durable queue — a PostgreSQL table claimed with `SELECT ... FOR UPDATE SKIP LOCKED` (our run-service, job-service and export worker all do this) or Kafka. Each instance runs a poller (`@Scheduled(fixedDelay)`) that claims only as many jobs as it has free worker slots, executes them on a bounded pool, and marks them done in the same transaction or with a status update guarded by the claim. A crashed instance's uncommitted claims are released by the database; a heartbeat/lease timeout handles jobs that were committed as "running" by an instance that later died. Scaling out is adding instances — no leader election needed.

Q55. A thread pool in production is rejecting tasks. How do you investigate?

Check pool metrics (active threads, queue size, completed count — Micrometer exposes `executor.*` for Spring executors) and a thread dump (`jcmd <pid> Thread.print`): are workers busy computing, blocked on a lock, or waiting on I/O (a downstream slowdown)? Usually the pool is saturated because a dependency got slow, so tasks take longer and Little's law says more are in flight. Fixes, in order: add timeouts to the slow call, add a bulkhead so one dependency cannot consume all threads, apply back-pressure (CallerRuns, 429/503 to clients), and only then resize. Increasing threads or queue size alone just moves the failure to memory or the downstream.

Q56. Is `ConcurrentHashMap.computeIfAbsent` a replacement for a lock?

For per-key atomic initialisation, yes: the mapping function runs at most once per absent key, while the bin is locked. But the function must be short and must not touch the same map (recursive update throws `IllegalStateException` in Java 9+), because other writers to that bin block meanwhile. Our gateway's `RateLimitFilter` creates its buckets exactly this way — `buckets.computeIfAbsent("ip:" + address, k -> new Bucket(perAddress, now))` — and each `Bucket` then guards its own refill-and-consume step with a `synchronized take(now)`.

Q57. How do you detect and avoid deadlocks with explicit locks?

Detection: a thread dump lists "Found one Java-level deadlock" for both monitors and `ReentrantLock`s (ownable synchronizers), and `ThreadMXBean.findDeadlockedThreads()` can be polled. Avoidance: acquire locks in a global order (e.g., by account id), use `tryLock` with timeout and back off, keep critical sections small, never call alien code (callbacks, remote calls) while holding a lock, and prefer higher-level constructs (queues, immutable snapshots, database row locks) over hand-rolled multi-lock code.

Q58. What changed in `java.util.concurrent` in recent Java versions?

Java 9: `CompletableFuture` timeouts and `delayedExecutor`, `VarHandle`, `Flow` (reactive streams interfaces). Java 12: `exceptionallyCompose`. Java 14: AQS rewrite. Java 19: `ExecutorService` is `AutoCloseable`, `Future.resultNow/exceptionNow/state`. Java 21: virtual threads final (`Executors.newVirtualThreadPerTaskExecutor`, `Thread.ofVirtual`), structured concurrency and scoped values in preview. In the JDK, many `synchronized` blocks in I/O classes were replaced with `ReentrantLock` to be virtual-thread friendly.

10. Virtual Threads, Structured Concurrency & Scoped Values (Java 21)

Virtual threads (JEP 444, final in Java 21) are the biggest change to Java concurrency since `java.util.concurrent`. Interviewers rarely stop at "they are lightweight". They ask what happens when one blocks, why pooling them is wrong, why a service got slower after enabling them, and what the database connection pool has to do with any of this. The one sentence to remember: **virtual threads make threads cheap; they do not make downstream resources cheap, and they do not make code faster.**

10.1 The problem: thread-per-request hits a wall

A platform thread is a thin wrapper over an OS thread: about 1 MB of reserved stack, kernel scheduling and microsecond context switches. Most servers can hold a few thousand at most. Meanwhile a typical microservice request spends nearly all of its time waiting on sockets.

```
Tomcat with 200 platform threads, each request waits 100 ms on I/O and computes 1 ms:
  Max throughput = 200 / 0.101 s         ≈ 1,980 req/s
  CPU in use     = 1,980 × 1 ms          ≈ 2 cores out of 8
  => threads run out long before the CPU does
```

Until Java 21 there were two ways out: size the pool up (memory and scheduler cost) or go asynchronous (`CompletableFuture`, WebFlux), which frees threads but costs readable stack traces, `ThreadLocal`-based context, simple debugging and plain `try/catch`. Virtual threads keep the blocking style and remove the thread cost.

10.2 Platform vs virtual threads

	Platform thread	Virtual thread
Backed by	One OS thread for its whole life	Any carrier, only while running
Stack	Fixed reservation, ~1 MB (`-Xss`)	Growable, on the heap as stack chunks
Creation cost	~tens of µs + kernel call	~1 µs, a plain object allocation
How many	Thousands	Millions
Scheduling	OS, preemptive time slices	JDK `ForkJoinPool`, cooperative at blocking points
Priority	Honoured as a hint	Always `NORM_PRIORITY`; `setPriority` is ignored
Daemon	Configurable	Always daemon; `setDaemon(false)` throws
Pool them?	Yes, they are expensive	**Never**, they are disposable
Best for	CPU-heavy work, native calls	Waiting

Virtual threads are still `java.lang.Thread`, so existing code keeps working:

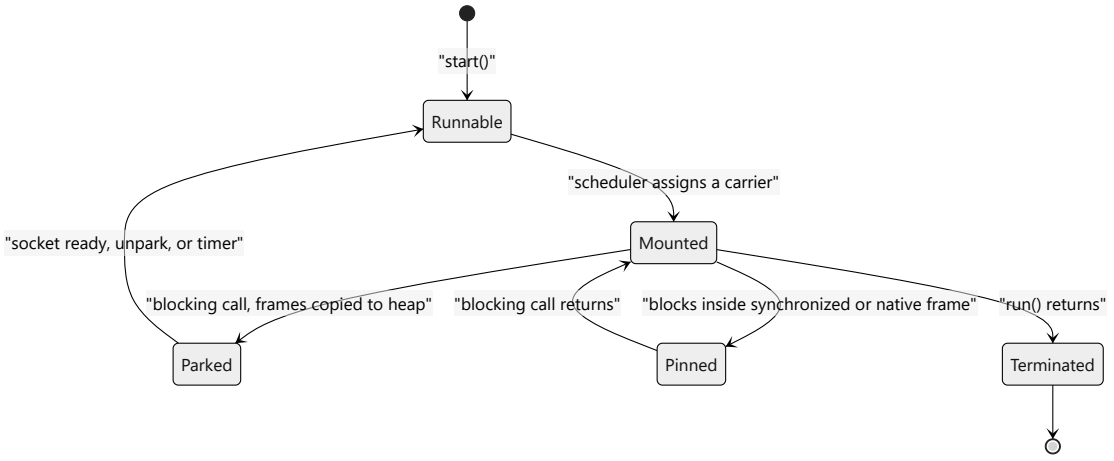
```
Thread vt = Thread.startVirtualThread(() -> handle(request));
Thread named = Thread.ofVirtual().name("recon-", 0).start(task);     // recon-0, recon-1, ...
ThreadFactory factory = Thread.ofVirtual().name("io-", 0).factory();

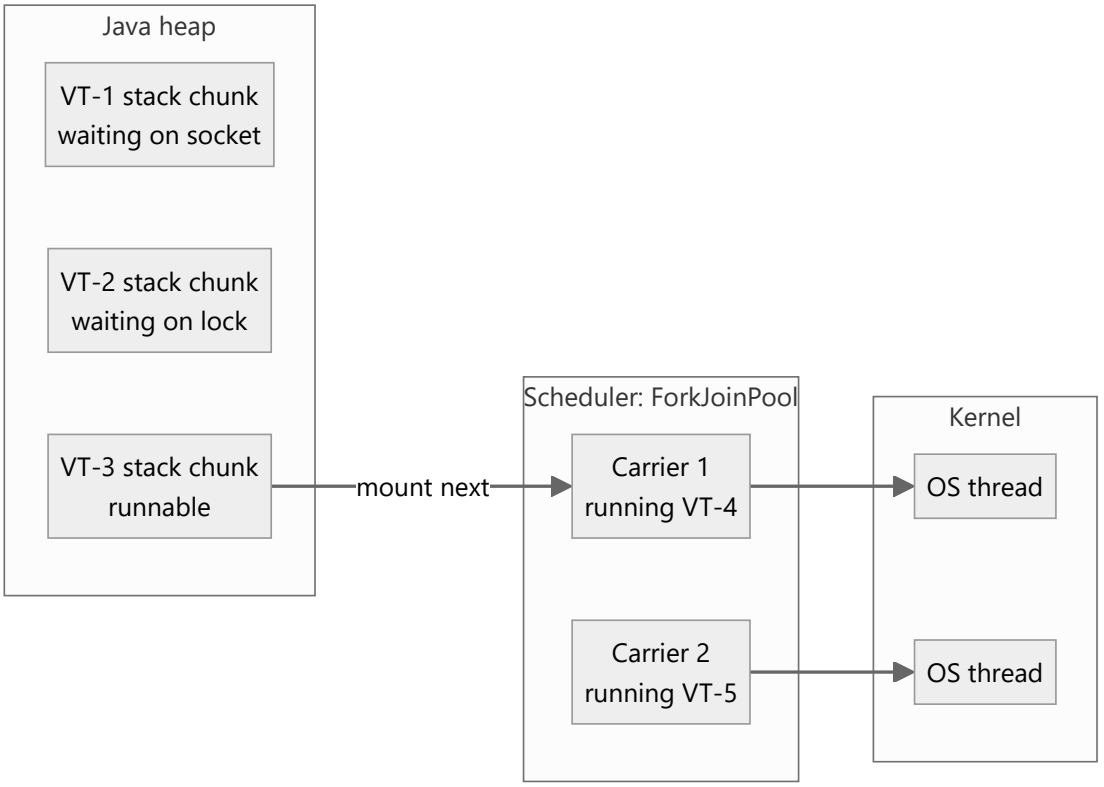
try (ExecutorService exec = Executors.newVirtualThreadPerTaskExecutor()) {
    IntStream.range(0, 10_000).forEach(i -> exec.submit(() -> {
        Thread.sleep(Duration.ofSeconds(1));
        return i;
    }));
}   // close() waits for all: about 1 s total. A pool of 200 platform threads needs 10,000 / 200
× 1 s = 50 s.
```

10.3 How they work: continuations and carriers

A virtual thread is a `Continuation` (a resumable stack) plus a scheduler. The default scheduler is a **dedicated work-stealing** `ForkJoinPool` in FIFO mode, separate from the common pool, whose threads are called **carrier threads**. Parallelism defaults to the number of cores (`-Djdk.virtualThreadScheduler.parallelism`), and the pool can temporarily grow up to `-Djdk.virtualThreadScheduler.maxPoolSize` (256 by default).

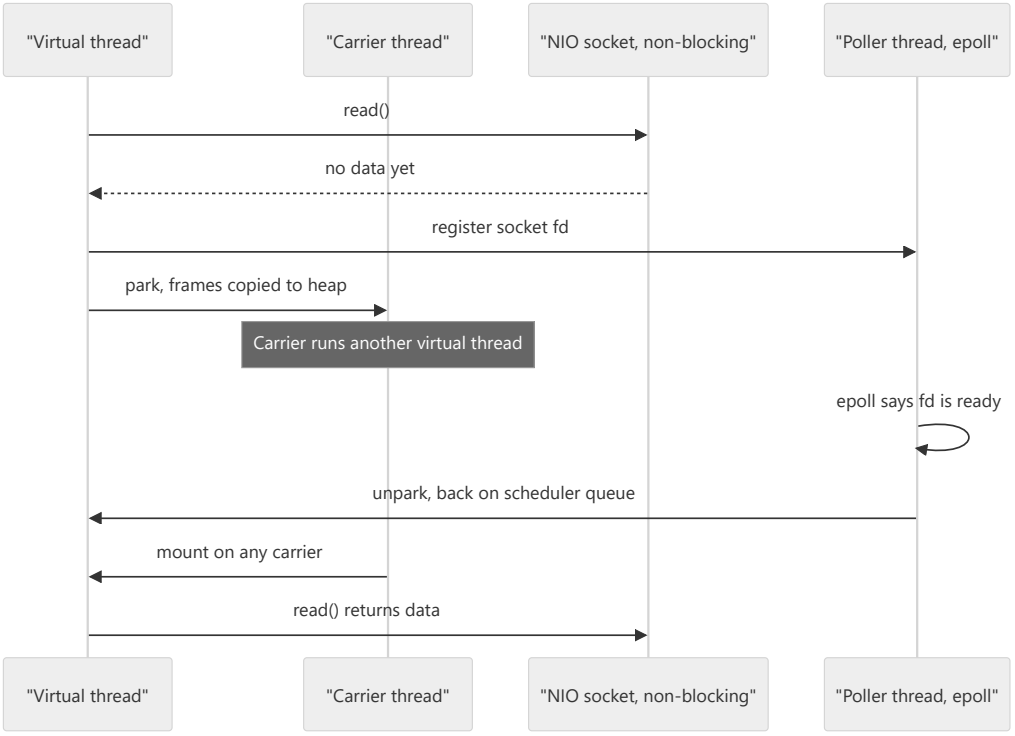
- **Mount**: the scheduler picks a runnable virtual thread and copies its top stack frames from heap chunks onto the carrier's stack, lazily. `Thread.currentThread()` returns the virtual thread, never the carrier.
- **Unmount (yield)**: when the code reaches a blocking point, the frames are copied to a heap `StackChunk` object and the carrier is free to take the next task.





What happens when a virtual thread blocks on a socket

The JDK's blocking APIs were rewritten for this (JEPs 353 and 373 reimplemented sockets). Code that looks blocking does not block the carrier:



Blocking points that unmount in Java 21: socket I/O, `Thread.sleep`, `LockSupport.park` (so `ReentrantLock`, `Semaphore`, `BlockingQueue`, `CompletableFuture.get`), and `Thread.join`. They stay mounted, though the

scheduler may compensate with an extra carrier: file I/O on most platforms, and `Object.wait`. DNS lookups go through the OS and are handled similarly.

There is **no time-slicing**: a virtual thread running a long computation keeps its carrier until it blocks. Enough CPU-heavy virtual threads will starve all the others.

10.4 Pinning

A virtual thread is **pinned** when it cannot be unmounted, so a blocking call blocks the carrier too. In Java 21 that happens in two cases:

1. Blocking **inside a `synchronized` block or method**, because the monitor is recorded against the carrier thread's identity.
2. Blocking with a **native frame** on the stack (a JNI call or a foreign-function upcall).

```
private final Object lock = new Object();
void bad() {
    synchronized (lock) {
        httpClient.send(request, ofString());   // pins the carrier for the whole HTTP call
    }
}

private final ReentrantLock rlock = new ReentrantLock();
void good() {
    rlock.lock();
    try { httpClient.send(request, ofString()); }   // parks and unmounts normally
    finally { rlock.unlock(); }
}
```

Pinning is not an error. Short `synchronized` sections with no blocking are harmless, and a global search-and-replace is unnecessary. It hurts when blocking is **long and frequent**: with 8 carriers, 8 pinned threads stop every other virtual thread in the JVM.

Java 24 (JEP 491) fixed the `synchronized` case: monitors now belong to the virtual thread, which can unmount while holding one. Native frames still pin, as do class initialisers.

How to find pinning:

Tool	Use
`-Djdk.tracePinnedThreads=full` or `short`	Prints a stack trace when a thread blocks while pinned. Java 21 to 23; removed in 24
JFR event `jdk.VirtualThreadPinned`	Production-safe; reported when pinned for over 20 ms by default
`jdk.VirtualThreadSubmitFailed` (JFR)	The scheduler could not accept a task

> **Trap:** Netflix published a real deadlock on Java 21. All carriers were pinned by virtual threads inside `synchronized` blocks, waiting for a `ReentrantLock`. The next owner of that lock was an *unmounted* virtual thread, which needed a free carrier to run. No carrier would ever come free, so Tomcat stopped answering while the sockets sat in `CLOSE_WAIT`. Pinning can turn into a full hang, not merely slowness.

10.5 When virtual threads help, and when they do not

Help: many concurrent tasks that spend most of their time waiting. Throughput is bounded by concurrency, not CPU (Little's law: `throughput = concurrency / latency`). They raise the *concurrency* limit. They do not lower *latency*.

Do not help:

- **CPU-bound work.** Carriers equal cores, so you get the same throughput as a fixed pool, minus time-slicing. Use a pool sized to the CPU count, or fork/join.
- **Small, already-comfortable concurrency.** At 50 concurrent requests on 200 threads, nothing changes.
- **Native-heavy code.** Every native call pins.

> **RefinePoint:** we run on platform threads today, and our pools show why virtual threads are not automatically the answer. `RunWorker` has `runs.workers` (default 2) slots, and `ExportWorker` has `exports.workers` (default 2), because reconciliation and export are bounded by heap, DuckDB memory and CPU, not by waiting. The engine also calls DuckDB through JDBC over native code, where frames pin. The candidates are places that mostly wait: an LLM call in ai-service can hold a thread for many seconds, and that is exactly the thread-per-task profile virtual threads suit.

Do not pool them

A pool exists to reuse something expensive and, as a side effect, to limit concurrency. A virtual thread is not expensive, so `Executors.newFixedThreadPool(100, Thread.ofVirtual().factory())` only adds a queue and a lock. To limit concurrency, say so directly with a `Semaphore`.

The semaphore replaces the pool as the limiter

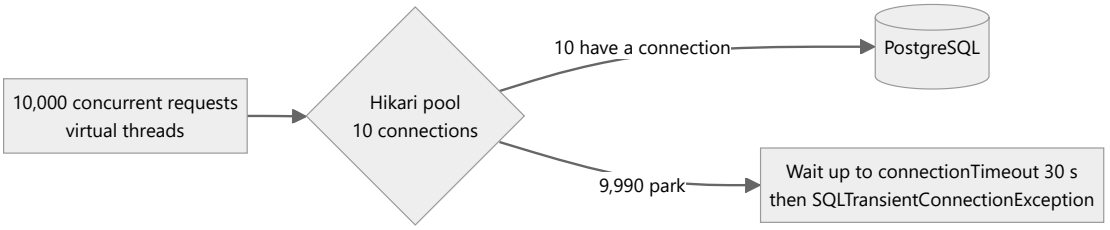
```
private final Semaphore llmPermits = new Semaphore(10);     // provider allows 10 in flight

LlmResponse call(LlmRequest req) throws InterruptedException {
    llmPermits.acquire();           // parks the virtual thread cheaply; no carrier is held
    try {
        return llmClient.complete(req);
    } finally {
        llmPermits.release();
    }
}
```

With platform threads, a pool of size 10 silently protected the downstream. With virtual threads that protection is gone: moving to thread-per-task removes the accidental limit, and 50,000 tasks can hit a service that handled 200.

10.6 The JDBC connection pool is the real bottleneck

Removing the thread limit exposes the next narrowest resource, and for most Spring services that is HikariCP (default `maximumPoolSize` = 10).



- Before: 200 Tomcat threads queued on 10 connections, with the rest of the backlog held in the accept queue. After: thousands queue on Hikari, and failures move from "slow" to "connection timeouts".
- The DB pool cannot grow to match: each PostgreSQL connection is a backend process. Throughput is still `connections / query time`.
- Parked waiters are cheap, but their request objects and buffers are not. Heap use goes up with concurrency; expect higher GC activity.
- Holding a connection across a slow remote call is now the most damaging mistake in the codebase. Keep `@Transactional` narrow, never call an external service inside it, and consider `spring.jpa.open-in-view=false`, which stops the session holding on to its connection for the rest of the request.

Check the libraries: pgjdbc replaced `synchronized` with `ReentrantLock` in 42.6.0 for exactly this reason, and Boot 3.4 ships a newer one. Older JDBC drivers (and some connection pools or HTTP clients) still block inside monitors.

> **Interview tip:** answering "we enabled virtual threads, what else changes?" with "the connection pool becomes the bottleneck, so I'd add a concurrency limit in front of it and fail fast with 503" is the mark of someone who has thought about this beyond the blog posts.

10.7 `ThreadLocal` with virtual threads

`ThreadLocal` works, and `InheritableThreadLocal` is inherited. The issues are about scale and assumptions:

- **Memory**: a million threads with a few locals each means millions of map entries.
- **Caching breaks**: `ThreadLocal<SimpleDateFormat>` as a per-thread cache now allocates once per task, because each thread is used once. Expensive objects get rebuilt repeatedly. Move them to a shared immutable object (`DateTimeFormatter`) or a real pool.
- **Unbounded per-thread buffers** in old libraries multiply.
- `-Djdk.traceVirtualThreadLocals` prints a stack trace when a virtual thread sets a thread local, which helps find these.

Context locals (MDC, `SecurityContextHolder`) are fine: one thread per task and short-lived, and there is no pool to leak into, so the classic "forgot to clear the tenant" bug largely disappears for virtual-thread tasks.

10.8 Spring Boot: `spring.threads.virtual.enabled=true`

From Boot 3.2 on Java 21, one property switches the default executors.

Component	Without	With the property
Tomcat / Jetty request handling	Platform pool (`server.tomcat.threads.max`, 200)	New virtual thread per request
`@Async`, `applicationTaskExecutor`	`ThreadPoolTaskExecutor`	`SimpleAsyncTaskExecutor` on virtual threads
`@Scheduled`	`ThreadPoolTaskScheduler`	`SimpleAsyncTaskScheduler`, a virtual thread per run
Kafka and RabbitMQ listener containers	Platform thread per consumer	Virtual threads
`RestClient` / `RestTemplate`	Blocks a platform thread	Blocks and unmounts a virtual thread

What people miss:

- **Tomcat's `threads.max` no longer limits anything.** Set `server.tomcat.max-connections` (default 8,192) and add your own limit (a filter with a `Semaphore`) for overload protection.
- **Queue-based back-pressure disappears.** `spring.task.execution.simple.concurrency-limit` caps the async executor, and `spring.task.scheduling.simple.concurrency-limit` the scheduler.
- **Fixed-delay tasks still share one thread.** `SimpleAsyncTaskScheduler` runs `fixedDelay` tasks on its single scheduler thread to keep the delay semantics; only fixed-rate and cron tasks get their own threads, and a slow fixed-rate task can therefore overlap with itself. All our relays and pollers are `fixedDelay`, so one that hangs would still delay the others.
- **The JVM can exit.** Virtual threads are daemons, so a worker app with no web server needs `spring.main.keep-alive=true`.
- **WebFlux ignores it** for its event loop. Spring Cloud Gateway (WebFlux on Netty) keeps its model, so our api-gateway would not change.

10.9 Virtual threads vs reactive

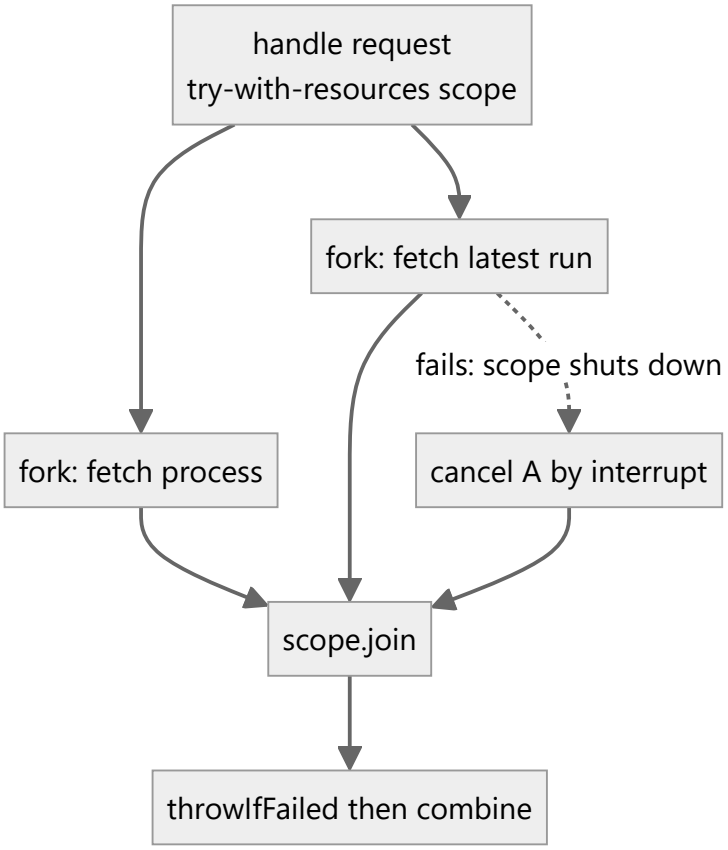
	Virtual threads	WebFlux / Reactor
Programming model	Plain blocking, sequential	Operators on `Mono` / `Flux`
Stack traces	Complete	Fragmented; needs `Hooks` / `checkpoint`
Debugging, `try/catch`, `ThreadLocal`	All normal	Harder; `Context` instead of `ThreadLocal`
Back-pressure	None built in: add semaphores	First-class, end to end
Streaming, event-driven pipelines	Possible, manual	Natural fit
Memory per in-flight request	Deeper stacks on the heap	Often smaller
Learning curve	Very low	Steep
Library needs	Blocking is fine (avoid long `synchronized`)	The whole stack must be non-blocking (R2DBC, not JDBC)

Rule of thumb: a new request/response service on JDBC should use virtual threads when it is thread-limited. Choose reactive for streaming, operators or back-pressure that you genuinely use, or an existing reactive stack. Reactive is not dead, but its main selling point (scalability without threads) no longer justifies its complexity for ordinary CRUD.

10.10 Structured concurrency

Unstructured concurrency: `executor.submit` creates a task whose lifetime has no relation to the code that started it. If one of two calls fails, the other keeps running; if the parent is interrupted, both keep running, and thread dumps cannot show which task belongs to which request.

Structured concurrency (JEP 453, **preview in 21**, needs `--enable-preview`) treats a group of subtasks as one unit: they start in a block and cannot outlive it.




```
OverviewDto overview(UUID id) throws Exception {
    try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
        Subtask<Recon> recon = scope.fork(() -> reconClient.get(id));
        Subtask<RunSummary> run = scope.fork(() -> runClient.latest(id));

        scope.joinUntil(Instant.now().plusSeconds(2));   // deadline for all
        scope.throwIfFailed();                           // rethrow first failure

        return new OverviewDto(recon.get(), run.get());
    }   // close() guarantees all subtasks have finished
}

// First success wins; the losers are cancelled
String fastest() throws Exception {
    try (var scope = new StructuredTaskScope.ShutdownOnSuccess<String>()) {
```

```
        scope.fork(() -> mirrorA.fetch());
        scope.fork(() -> mirrorB.fetch());
        return scope.join().result();
    }
}
```

	`CompletableFuture.allOf`	`StructuredTaskScope`
Failure of one	Others keep running	Others cancelled by interrupt
Parent interrupted	Children unaffected	Propagates to children
Lifetime	Unbounded	Ends with the block
Thread-dump relationship	None	Parent and child visible in JSON dumps
Status	Final	Preview (Java 21)

The API has changed across previews: Java 25 (JEP 505) replaced the subclasses with `StructuredTaskScope.open(Joiner...)`. Know the concept and say that the API is still moving. Do not ship preview features into production without accepting the rework.

10.11 Scoped values vs `ThreadLocal`

`ThreadLocal` is mutable, lives as long as the thread unless removed, and is copied to child threads by `InheritableThreadLocal`. Scoped values (JEP 446, **preview in 21**, **final in Java 25** through JEP 506) are immutable and bound to a dynamic scope, and they share cheaply with structured-concurrency children.

```
static final ScopedValue<OrgContext> ORG = ScopedValue.newInstance();

void handle(Request req) {
    ScopedValue.where(ORG, resolve(req)).run(() -> service.process(req));
    // outside run(): ORG is unbound. No remove() to forget.
}

void deepInTheCallStack() {
    OrgContext org = ORG.get();    // cheap read, no parameter threading
}
```

	`ThreadLocal`	`ScopedValue`
Mutability	`set` anywhere	Immutable within a binding; rebinding nests
Lifetime	Until `remove()` or thread death	The `run()` / `call()` block
Leak risk	Real, especially in pools	None
Inheritance	Copies a map per child	Shared with `StructuredTaskScope` children, no copy
Read cost	Hash lookup	Small and cacheable
Status in 21	Final	Preview

> **RefinePoint:** per-request tenant context is the textbook use. `OrgContextService` resolves the organisation for each request; a scoped value would make it impossible for an org id to outlive the request. On Java 21 this remains a design note, because we do not run preview features in production.

10.12 Migrating and proving it worked

A safe migration, in order:

1. **Prove you need it.** Thread pool saturated while CPU and DB have room? If the DB or CPU is already the limit, virtual threads will make things worse, not better.
2. **Upgrade the libraries** (drivers, HTTP clients, Kafka client, Resilience4j) to versions without long-held monitors.
3. **Run with pinning detection** (`jdk.VirtualThreadPinned` in JFR) against realistic load. Fix only long pins.
4. **Audit the implicit limits**: anything previously protected by the pool size needs a semaphore.
5. **Audit `ThreadLocal` caches** and connection-hold times.
6. **Flip the property on one instance** behind the load balancer and compare p50, p99, error rate, Hikari pending count, heap and GC.
7. **Have a rollback**: one property.

How to benchmark. Use an open-model load generator (Gatling, or k6 with an arrival rate), not closed-loop `ab`, so the load does not politely wait. Raise the arrival rate until errors appear, and compare the knee of the curve. Expect the following shape; these are illustrative, so measure your own:

```
Blocking service: 200 ms downstream wait, small CPU per request.

                        Platform (200 threads)     Virtual threads
Rate limit              200 / 0.2 s = 1,000 req/s  Bound by CPU / downstream
p50 at low load         ~200 ms                    ~200 ms  (no faster!)
Overload behaviour      Queue, latency climbs      Accepts all; pressure moves downstream
```

> **Interview tip:** only claim a migration you actually did. "We run on platform threads because our workers are memory-bound, and here is the one place I'd adopt virtual threads, and how I'd validate it" is a far stronger answer than an invented benchmark.

10.13 Debugging and thread dumps

`jstack` and `jcmd Thread.print` show platform threads and mounted carriers only. For the full picture:

```
jcmd <pid> Thread.dump_to_file -format=json /tmp/threads.json
jcmd <pid> Thread.dump_to_file /tmp/threads.txt      # plain text
```

- The dump groups threads by container, and structured scopes appear as a tree (the point of the design).
- Virtual threads have **no name by default**. Name them via `Thread.ofVirtual().name("export-", 0)`.
- Debuggers step through them normally; JFR records `jdk.VirtualThreadStart`, `End`, `Pinned` (the first two are off by default).
- Stack traces show the virtual thread's frames only, and the carrier is hidden.

- `Thread.getAllStackTraces()` does not include virtual threads.

10.14 Interview questions and answers

Q1. What is a virtual thread?

A lightweight `java.lang.Thread` managed by the JVM rather than the OS. It runs on a carrier (platform) thread only while executing and is unmounted to the heap when it blocks, so millions can exist while using a handful of OS threads. Final in Java 21 through JEP 444.

Q2. How is it different from a platform thread?

A platform thread maps 1:1 to an OS thread for its lifetime, with a fixed stack reservation and kernel scheduling. A virtual thread maps M:N onto carriers, has a growable heap-allocated stack, is scheduled by a JDK `ForkJoinPool`, and is cooperative at blocking points. It costs about a microsecond and a few hundred bytes to start.

Q3. Do virtual threads make code faster?

No. Each operation takes the same time, or slightly longer. They raise **throughput for waiting workloads** by allowing far more concurrent tasks, as Little's law says. A single request's latency is unchanged. Say "scale, not speed".

Q4. What is a carrier thread?

A platform thread from the virtual-thread scheduler's `ForkJoinPool` that a virtual thread is mounted on while it runs. By default there is one per core. A virtual thread may resume on a different carrier after each block, so code must never depend on carrier identity. `Thread.currentThread()` always returns the virtual thread.

Q5. Explain mounting and unmounting.

Mounting copies the continuation's top frames from heap stack chunks to the carrier's stack and starts running; further frames thaw lazily through a return barrier. Unmounting happens at a blocking point: the frames are frozen into a `StackChunk` on the heap and the carrier picks the next runnable virtual thread. Only the frames that changed are copied, so it is cheap.

Q6. What happens when a virtual thread does a blocking socket read?

The JDK's socket code runs the channel in non-blocking mode. If no data is ready, it registers the file descriptor with a poller thread (epoll, kqueue or IOCP) and parks the virtual thread, which unmounts it. When the poller sees readiness, it unparks the thread, the scheduler re-queues it, a free carrier mounts it, and the read returns. The code never notices.

Q7. Which operations do not unmount?

Long CPU work (no time-slicing), any blocking while pinned (in `synchronized` on Java 21 to 23, or in a native frame) and file I/O on most platforms. File I/O and `Object.wait` make the scheduler temporarily add a compensating carrier, up to `maxPoolSize`, so progress continues at a cost.

Q8. What is pinning?

The state where a virtual thread cannot unmount, so a blocking call blocks its carrier. On Java 21 it occurs inside `synchronized` and with native frames on the stack. Short, non-blocking pinned sections are harmless. Long blocking pins can starve or even deadlock the scheduler. JEP 491 in Java 24 removed the `synchronized` case.

Q9. Why does `synchronized` pin on Java 21?

HotSpot's monitor implementation recorded the owner as the OS thread, in effect the carrier. If the virtual thread unmounted while holding the monitor, another virtual thread on that carrier would appear to own it. JEP 491 changes ownership to track the virtual thread itself.

Q10. Should I replace all `synchronized` with `ReentrantLock`?

No. Replace only the blocks that hold across blocking calls and appear in pinning reports. A short counter update in `synchronized` never blocks while pinned. A blanket rewrite risks forgotten `unlock()` bugs and is unnecessary from Java 24 on.

Q11. How do you detect pinning?

On Java 21 to 23, `-Djdk.tracePinnedThreads=full` prints a stack trace when a pinned thread blocks. In production, JFR's `jdk.VirtualThreadPinned` event (default threshold 20 ms), viewed in JMC or `jfr print --events jdk.VirtualThreadPinned`. The flag was removed in Java 24, so JFR is the durable answer.

Q12. Why should you not pool virtual threads?

Pools exist to reuse expensive things, and virtual threads are cheap and intended for one task each. A pool adds a queue and lock for no gain, and it keeps stale `ThreadLocal`s alive. If the real purpose was limiting concurrency, use a `Semaphore`.

Q13. How do you limit concurrency with virtual threads?

A `Semaphore` with permits equal to the downstream's capacity. Acquire before the call, release in `finally`. Waiting virtual threads park cheaply. Prefer `tryAcquire(timeout)` at the edge so overload becomes a fast 429/503 rather than an endless queue.

Q14. What is the scheduler, and can you configure it?

A dedicated FIFO work-stealing `ForkJoinPool`, not the common pool. It is tuned with `-Djdk.virtualThreadScheduler.parallelism` (default: cores), `maxPoolSize` (default 256) and `minRunnable`. Custom schedulers are not a public API in Java 21. Increasing parallelism beyond cores rarely helps and is usually a sign of a pinning problem being hidden.

Q15. Are virtual threads preempted?

Not by time slice in Java 21. A virtual thread yields only at a blocking operation (the OS may preempt the carrier, but that does not switch virtual threads). So N long-running CPU tasks, where N is the number of carriers, stop every other virtual thread until they block or finish.

Q16. What happens if I run CPU-bound work on virtual threads?

Throughput is the same as a pool sized to the CPU count at best, and latency for other virtual threads can get much worse, since there is no time-slicing. Run CPU work on a bounded platform pool and have the virtual thread wait on it. In RefinePoint the match work is CPU-, memory- and native-heavy, so it stays on `RunWorker`'s small fixed pool.

Q17. Can a virtual thread be a non-daemon?

No. They are always daemon; `setDaemon(false)` throws `IllegalArgumentException`. The JVM exits when no platform non-daemon threads remain, which is why background apps need `spring.main.keep-alive=true`. Priority is likewise fixed at `NORM_PRIORITY`.

Q18. Do `ThreadLocal`s work on virtual threads?

Yes, fully, including inheritance. The concerns are memory at huge thread counts and per-thread caches, which stop caching because each thread runs one task. Context values like MDC are fine and clean up with the thread. Find offenders with `-Djdk.traceVirtualThreadLocals`.

Q19. Why does `ThreadLocal<SimpleDateFormat>` become a problem?

It was a pooled-reuse trick that depended on long-lived threads. With a thread per task, each task builds its own `SimpleDateFormat`, so you get the allocation you tried to avoid, with an extra map entry. Use immutable `DateTimeFormatter` (shared) or a real object pool for truly expensive objects.

Q20. How does `spring.threads.virtual.enabled=true` change a Boot app?

Tomcat/Jetty handle each request on a new virtual thread; `applicationTaskExecutor` becomes a virtual-thread `SimpleAsyncTaskExecutor`; `@Scheduled` uses `SimpleAsyncTaskScheduler`; Kafka and RabbitMQ listener containers use virtual threads. It requires Java 21 and Boot 3.2+. Custom `ThreadPoolTaskExecutor` beans you defined stay platform-based.

Q21. What happens to `server.tomcat.threads.max` with virtual threads on?

It has no effect on concurrency, since the worker pool is replaced by per-request virtual threads. Connections remain limited by `server.tomcat.max-connections`. You lose the implicit bulkhead, so add a limiter for anything downstream that cannot keep up.

Q22. How do `@Scheduled` methods behave with virtual threads on?

A single scheduler thread triggers them, and each fixed-rate or cron run gets its own virtual thread, so one slow job no longer blocks others, but a slow fixed-rate job can overlap itself. `fixedDelay` tasks run on the scheduler thread itself, to preserve their semantics, and therefore still share it. Add overlap protection (an `AtomicBoolean` guard or `FOR UPDATE SKIP LOCKED`) wherever overlap matters. Ours already use `SKIP LOCKED`.

Q23. Is `@Async` still needed?

It is still useful for fire-and-forget decoupling, and it becomes cheaper. For fan-out inside a request, an explicit virtual-thread executor or structured scope is clearer. The default limit disappears, so set `spring.task.execution.simple.concurrency-limit` if a burst could overwhelm a downstream.

Q24. Why did my service get errors after enabling virtual threads?

Most often the implicit limit vanished: Hikari timeouts (`Connection is not available, request timed out`), downstream 429s, or `OutOfMemoryError` from many in-flight request bodies. Thread count was your accidental rate limiter. Fix by adding explicit limits, narrowing transactions and failing fast.

Q25. Why can't you just raise Hikari's pool size to match?

PostgreSQL gives each connection a backend process, and throughput falls past a point because of contention and context switching. A common starting formula is `connections = cores × 2 + effective spindles`. Concurrency in the database is a fixed budget across all instances. Move waiting to the app, keep transactions short, and use PgBouncer where instance counts make the total too high.

Q26. How does holding a transaction across an HTTP call hurt more now?

Each `@Transactional` scope holds a connection. A 2 s LLM call in a transaction blocks one of 10 connections for 2 s, capping that code path at 5 req/s for the whole instance, and thousands of virtual threads then queue behind it. Do the remote call outside the transaction, then open a short one to write the result.

Q27. Virtual threads vs WebFlux: which do you choose?

Virtual threads for request/response services on a blocking stack (JDBC, `RestClient`), since you keep simple code, stack traces and `ThreadLocal`. WebFlux for streaming, operators and real end-to-end back-pressure, or where a reactive stack already exists. Our api-gateway stays on Spring Cloud Gateway (Netty, reactive), while the business services use plain blocking Spring MVC.

Q28. Does a virtual thread solve back-pressure?

No, it removes one form of it. A bounded pool pushed back through its queue; with virtual threads, everything is accepted and pressure lands on whatever is next. You must add limits (a semaphore, rate limiter, or connection cap). Reactive streams have demand signals built in; plain blocking code does not.

Q29. Can virtual threads call `CompletableFuture`?

Yes. `get()` / `join()` park and unmount cheaply. But in most cases sequential blocking calls in a virtual thread replace `thenCompose` chains entirely; keep `CompletableFuture` for parallel fan-out on Java 21, where structured concurrency is still preview. Pass `newVirtualThreadPerTaskExecutor()` as the executor to `supplyAsync`.

Q30. What is the output and duration of this code?

```
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 100_000; i++) ex.submit(() -> { Thread.sleep(1000); return 1; });
}
System.out.println("done");
```

It prints `done` after about one to two seconds, because `close()` waits for all tasks and `Thread.sleep` unmounts. Replace `sleep` with a tight CPU loop and it takes about `100,000 × work / cores`. With `newFixedThreadPool(200)` and sleep, it takes about 500 s.

Q31. Why is this code a potential problem on Java 21?

```
synchronized (cache) { value = httpClient.send(req, ofString()).body(); cache.put(k, value); }
```

It pins the carrier for the duration of the HTTP call and serialises all callers. Fetch outside the lock, or use a `ReentrantLock`, or better still `ConcurrentHashMap.computeIfAbsent` with a fast function and a per-key lock for the fetch.

Q32. What is the Netflix pinning deadlock?

Virtual threads inside `synchronized` were pinned while waiting on a `ReentrantLock`, taking every carrier. The lock's next owner was unmounted and needed a carrier, which would never be free. The service stopped accepting work with sockets in `CLOSE_WAIT`. The lesson: pinning plus a lock can deadlock. Monitor it and upgrade to Java 24+ where possible.

Q33. Does `Thread.getAllStackTraces()` or `jstack` show virtual threads?

Not unmounted ones. `jstack` shows platform threads and any mounted carrier's frames. Use `jcmd <pid> Thread.dump_to_file -format=json file.json`, which lists every virtual thread and their container tree.

Q34. How does GC see virtual threads?

Parked virtual threads are stack chunks on the heap, so they count as heap objects and are collected once unreachable. They are not GC roots in the way platform thread stacks are: a blocked virtual thread that nothing references can in principle be collected, though by default the JDK keeps it observable (`jdk.trackAllThreads`). Heavy use raises allocation and heap pressure.

Q35. Do you need to change blocking code to use virtual threads?

Mostly not, since the JDK's networking, locks and queues already work. Check third-party libraries that pin during I/O (old JDBC drivers, some native clients), audit concurrency limits, and audit `ThreadLocal` caches. That is the whole design goal: existing code gains scale without a rewrite.

Q36. What is structured concurrency?

Treating concurrent subtasks as one unit with a lexical lifetime, like a block: subtasks cannot outlive the scope, failure cancels siblings, and cancellation of the parent reaches the children. In Java 21 it is `StructuredTaskScope` (JEP 453), a preview feature requiring `--enable-preview`.

Q37. `ShutdownOnFailure` vs `ShutdownOnSuccess`?

`ShutdownOnFailure` is "all must succeed": the first failure cancels the rest, and `throwIfFailed()` rethrows. `ShutdownOnSuccess` is "first to succeed wins": on success the others are cancelled, and `result()` returns it. Custom policies subclass `StructuredTaskScope` and override `handleComplete`. From Java 25 these become `Joiner`s passed to `StructuredTaskScope.open`.

Q38. Why is this better than `CompletableFuture.allOf`?

`allOf` does not cancel the other tasks when one fails; the work wastes resources and you must wire up timeouts and propagation manually. A scope cancels by interrupt, propagates parent cancellation, guarantees no leaked tasks at `close()`, and shows parent-child relationships in thread dumps.

Q39. What must you call before reading a `Subtask` result?

`scope.join()` (or `joinUntil`), and for `ShutdownOnFailure`, `throwIfFailed()`. Calling `subtask.get()` before `join` throws `IllegalStateException`, as does calling it on a failed subtask. Forking from a thread that is neither the owner nor inside the scope throws `WrongThreadException`.

Q40. What are scoped values?

Immutable values bound for the dynamic extent of a block with `ScopedValue.where(KEY, value).run(...)`, readable anywhere deeper in the call stack via `KEY.get()`. They were preview in Java 21 (JEP 446) and final in Java 25 (JEP 506). They are the replacement for `ThreadLocal` in "pass context implicitly" use cases.

Q41. Scoped values vs `ThreadLocal`?

`ThreadLocal` is mutable, must be removed manually, can leak in pools and copies on inheritance. A scoped value is immutable, ends automatically with the block, shares cheaply with structured-concurrency children and cannot leak. `ThreadLocal` remains right for per-thread mutable state, such as a buffer; scoped values fit one-way context like tenant, principal or trace id.

Q42. Can you rebind a scoped value?

Yes, by nesting: an inner `ScopedValue.where(KEY, other).run(...)` shadows the outer value for that block, and the outer binding comes back afterwards. A `set` does not exist. A confined rebinding is much easier to reason about than a mutated `ThreadLocal`.

Q43. Do Spring's `SecurityContextHolder` and `@Transactional` work on virtual threads?

Yes. Both use `ThreadLocal`s (`TransactionSynchronizationManager` holds the connection), and a virtual thread is a thread. They behave as usual for a request handled on one virtual thread. As with pools, a task forked to *another* thread does not inherit the transaction, and it needs its own context propagated.

Q44. What is the memory cost of a virtual thread?

From a few hundred bytes for the object plus its stack chunks, growing with actual stack depth. Deep Spring stacks (100+ frames) cost several KB each while parked. A million deep stacks is therefore gigabytes. It is much cheaper than platform threads, yet not free, and it moves memory pressure from native stack space to the Java heap.

Q45. Would you use virtual threads in RefinePoint? Where and where not?

We run on platform threads today. I would not use them for `RunWorker` or `ExportWorker`: heap, DuckDB memory and CPU limit those, so 2 slots per instance is deliberate. The DuckDB JDBC driver also goes through native code. I would try them where threads mostly wait: LLM calls in ai-service, with a semaphore per provider, since a call can take many seconds. I'd validate with JFR pinning events and load tests, and fall back with a single property. The gateway stays reactive.

Q46. How would you roll this out and measure success?

One instance behind the load balancer first, under an arrival-rate load test. Compare p99, error rate, Hikari pending threads, heap, GC time and `jdk.VirtualThreadPinned` counts. Success means a higher knee on the throughput curve with no worse p99 and no new errors. A flat curve means threads were never your bottleneck, so roll back.

Q47. What other "limits" typically surface after this change?

File descriptors (`ulimit -n`), ephemeral ports and HTTP client connection pools (`maxConnections` per route), Kafka producer buffers, DB connections, memory for in-flight payloads, and downstream rate limits. Each was hidden behind the thread cap. Watch them all in the first load test.

Q48. Are virtual threads a replacement for Kafka or a job queue?

No. They are about in-process concurrency, and they disappear with the process. A durable claim from a table or topic is what survives crashes. Virtual threads could run the work after claiming, but the claim-loop design and its limits stay exactly as they are.

Q49. What changed across Java 21, 24 and 25 for this topic?

Version	Change
19, 20	Virtual threads in preview
21	Virtual threads final (JEP 444); structured concurrency and scoped values preview
24	`synchronized` no longer pins (JEP 491); `jdk.tracePinnedThreads` removed
25	Scoped values final (JEP 506); structured concurrency reworked around `Joiner` (JEP 505, still preview)

An interviewer asking about pinning on "modern Java" expects you to separate Java 21 behaviour from what JEP 491 fixed, and to remember that native frames and class initialisers still pin.

Q50. Does enabling virtual threads make Kafka listeners process more messages?

Not by itself. The container's concurrency still decides how many consumers exist, and each partition is processed by one consumer in order. Virtual threads make each consumer thread cheaper, which matters when you run many consumers. To increase throughput you add partitions and consumers, or fan out inside the listener with limits while protecting commit order. `max.poll.interval.ms` still applies: a batch that blocks too long gets the consumer kicked out of the group.

Q51. Is `parallelStream()` inside a virtual thread a good idea?

Generally no reason to. The stream's work runs on the common `ForkJoinPool` with the calling thread helping, so CPU parallelism is unchanged, and it is shared by every caller in the JVM. For I/O fan-out, submit each call to a virtual-thread executor instead. For CPU work, use a dedicated platform pool.

Q52. How do you give virtual threads meaningful names and exception handling?

Use a builder: `Thread.ofVirtual().name("ai-call-", 0).uncaughtExceptionHandler(handler).factory()`, then `Executors.newThreadPerTaskExecutor(factory)`. Without a name, `getName()` is empty, and dumps and logs become much less useful. The handler catches anything escaping `run()`; the usual advice still applies, so handle errors inside the task and keep the futures you create.

Q53. A virtual-thread task never finishes. How do you investigate it?

Take `jcmd <pid> Thread.dump_to_file -format=json dump.json` and search for the task's frames. Is it parked on a lock, a semaphore or a socket, or stuck in a pinned frame? Compare two dumps a minute apart to separate slow from stuck. Check JFR for pinned events and whether all carriers are busy (`ForkJoinPool-1-worker-N` in `jstack`). Then make sure every downstream call has a timeout, because a virtual thread with no timeout waits forever, just like a platform thread, only with fewer symptoms.

Q54. What is a `Continuation`, and can I use it directly?

It is the JDK-internal one-shot delimited continuation (`jdk.internal.vm.Continuation`) with `run()` and `yield()`. `yield` freezes the frames to the heap, and `run` thaws and resumes them. A virtual thread is a continuation plus a scheduler. It is not public API and needs `--add-exports` to touch, so application code should only use `Thread`. Mentioning it shows you know the layering, not that you should use it.

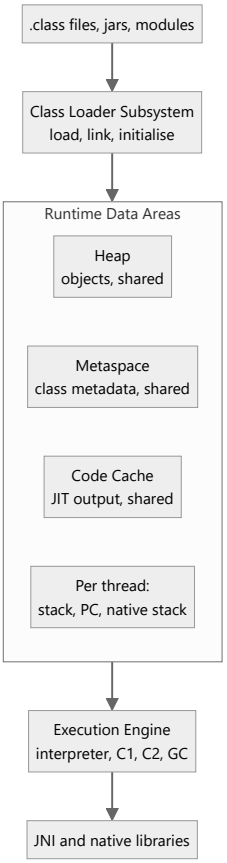
Q55. Explain virtual threads to a junior developer.

Imagine a restaurant. Platform threads are waiters who stand at a table until the food arrives: you can hire few of them, and most are just waiting. Virtual threads are order tickets: a small number of waiters (carriers) carry tickets, drop one when the kitchen is busy (blocking), and pick up another. The kitchen (the database) is not any bigger, though, so if you accept every order at once it still falls behind, and you must limit orders with a semaphore.

11. JVM Architecture: Class Loading, Memory & Execution

"Explain the JVM" is a test of whether your mental model survives contact with production. When a service dies with `Metaspace`, starts slowly, or throws `NoClassDefFoundError` only on the server, the fix comes from knowing which subsystem owns the failure. This chapter walks a class from `.jar` to running machine code: how it gets loaded, where its data lives, what an object physically looks like, and how bytecode becomes optimised native code. Garbage collection gets its own chapter next.

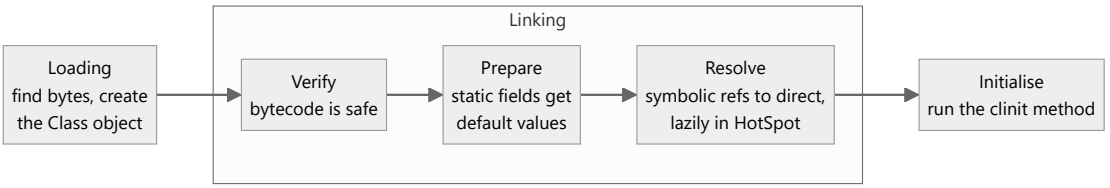
11.1 The big picture



Three separate things are called "Java": the **JVM specification** (what bytecode means), an **implementation** (HotSpot in OpenJDK, the one everyone means, plus OpenJ9 and GraalVM) and the **JDK/JRE** that bundle it with libraries. Everything below is HotSpot on a 64-bit JVM, Java 21.

11.2 Class loading

A class is loaded lazily, the first time it is needed, in three phases.



- **Loading:** a class loader reads the bytes, parses them into metadata in metaspace and creates a `java.lang.Class` mirror on the heap.
- **Verification:** checks type safety, stack balance and valid jumps. It rejects bad bytecode with `VerifyError`. Disabling it (`-Xverify:none`) is deprecated and pointless.
- **Preparation:** allocates static fields with default values (`0`, `null`). `static final int X = 5` already holds its constant here.
- **Resolution:** turns constant-pool references into direct pointers, on first use in HotSpot.
- **Initialisation:** runs `<clinit>`, the compiler-merged static field initialisers and `static {}` blocks, in textual order. The JVM holds a per-class init lock, so it runs **exactly once, even with many threads**. That guarantee is what makes the holder-class singleton safe with no `volatile`.

When is a class initialised?

Triggers: `new`, calling a static method, reading or writing a non-constant static field, `Class.forName(name)`, initialising a subclass (superclass first), being the main class, and reflection that instantiates it.

Not triggers (favourite trick question):

```
class Parent { static int value = 1; static { System.out.println("Parent init"); } }
class Child extends Parent { static final int K = 42; static { System.out.println("Child init");
} }

System.out.println(Child.value);   // "Parent init" then 1. Child is NOT initialised (field
declared in Parent)
System.out.println(Child.K);       // 42 only. Compile-time constant, inlined at the call site
Child[] arr = new Child[3];        // nothing printed: creating an array does not initialise the
element class
Class<?> c = Child.class;          // nothing printed
loader.loadClass("Child");         // loads, does NOT initialise. Class.forName("Child") does.
```

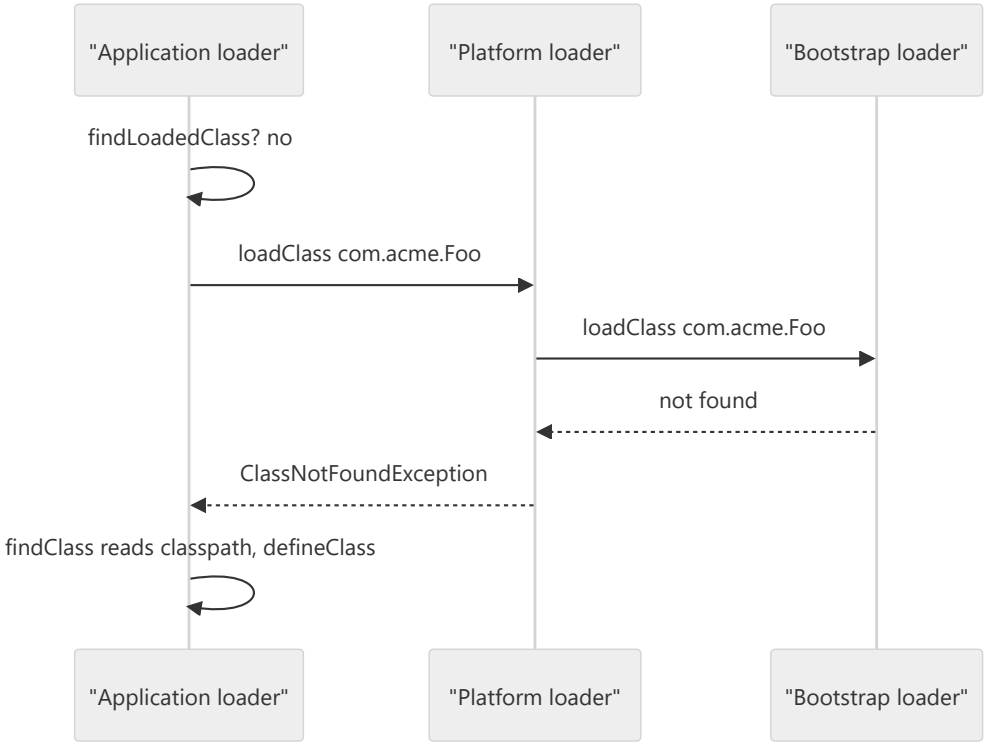
Interfaces are initialised only when one of their non-constant fields is accessed, or when an implementing class initialises and the interface declares default methods.

The loader hierarchy

Loader	Loads	Notes
Bootstrap	`java.base` and core modules, from `lib/modules` (jimage)	Written in C++; `String.class.getClassLoader()` returns `null`
Platform (Java 9+)	Other JDK modules (`java.sql` , ...)	Replaced the Extension loader and `jre/lib/ext`
Application (system)	Your classpath and module path	`ClassLoader.getSystemClassLoader()`
Custom	Whatever you decide	Plugins, Spring Boot jars, app servers, hot reload

Parent delegation

`loadClass` asks: already loaded? Then my parent? Only if the parent fails, my own `findClass` .



Why it exists: **security and consistency**. Nobody can substitute a malicious `java.lang.String` , because the request always reaches bootstrap first (and `defineClass` refuses names in `java.*`).

Class identity is (name, defining loader). The same class loaded by two loaders gives two different types, which is how `ClassCastException: com.acme.Foo cannot be cast to com.acme.Foo` appears, with the message even telling you the two loaders.

Delegation is deliberately broken in two important places:

- **Thread context class loader (TCCL):** `ServiceLoader` and JDBC live in bootstrap or platform code but must instantiate classes on the application classpath, so they use `Thread.currentThread().getContextClassLoader()` , looking downward.

- **Child-first web apps:** Tomcat's `WebappClassLoader` checks `WEB-INF` before the parent, so each app keeps its own libraries.

Custom class loader and Spring Boot's fat jar

```
class PluginLoader extends ClassLoader {
    private final Path dir;
    PluginLoader(Path dir, ClassLoader parent) { super(parent); this.dir = dir; }
    @Override protected Class<?> findClass(String name) throws ClassNotFoundException {
        try {
            byte[] b = Files.readAllBytes(dir.resolve(name.replace('.', '/') + ".class"));
            return defineClass(name, b, 0, b.length);
        } catch (IOException e) { throw new ClassNotFoundException(name, e); }
    }
}
```

Override `findClass`, not `loadClass`, to keep delegation.

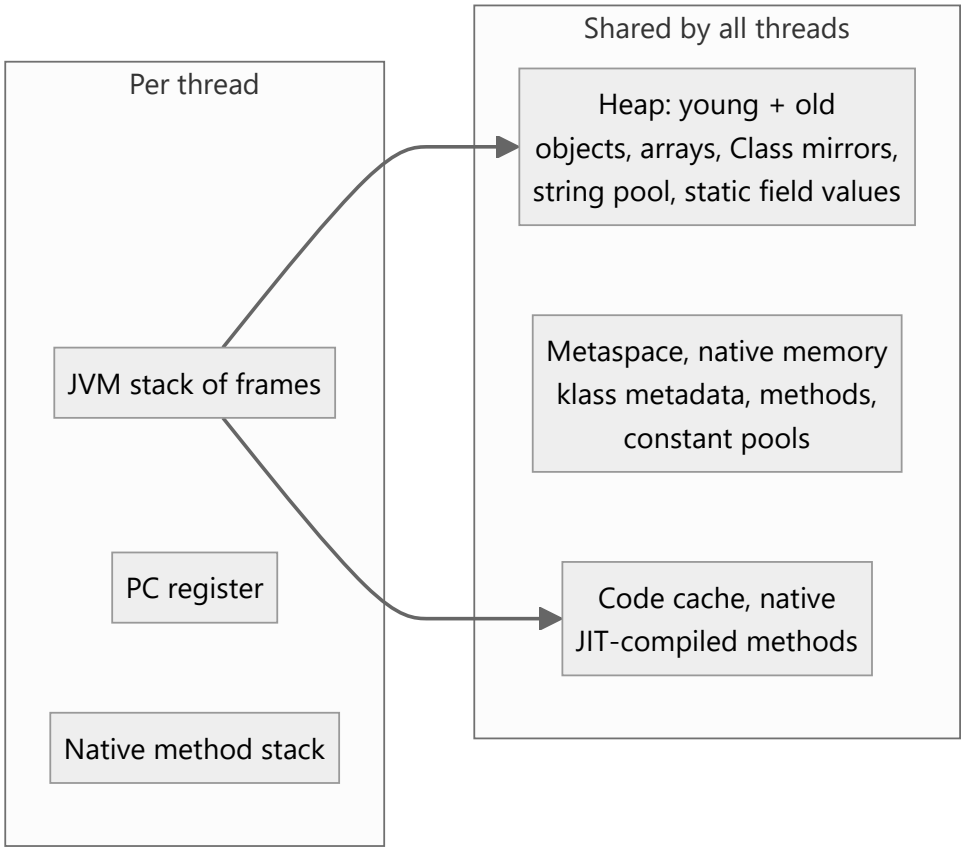
A standard JVM cannot load classes from a jar nested inside a jar. Spring Boot solves this with its own launcher:

```
app.jar
├── META-INF/MANIFEST.MF    Main-Class: org.springframework.boot.loader.launch.JarLauncher
│                           Start-Class: com.ryfnflow.crm.RunServiceApplication
├── org/springframework/boot/loader/...   the launcher classes, plain (not nested)
├── BOOT-INF/classes/       your code
└── BOOT-INF/lib/*.jar      dependencies, stored uncompressed so they can be read in place
```

`JarLauncher` builds a `LaunchedClassLoader` (named `LaunchedURLClassLoader` before Boot 3.2) using a custom `jar:nested:` URL handler over `BOOT-INF/classes` and `BOOT-INF/lib/*`, sets it as the TCCL and calls `Start-Class.main()` by reflection.

> **RefinePoint:** `Plugins` in ryfn-plugin-api builds a `URLClassLoader` over the jars in `RYFN_PLUGINS_DIR`, with the service's own loader as its parent, and runs `ServiceLoader.load(extensionPoint, loader)` on it. Parent delegation is why a plugin sees the API and the JDK and brings anything else itself. It also catches `ServiceConfigurationError | RuntimeException | LinkageError`, so one broken jar is logged and left out while the others still load.

11.3 Runtime data areas



Area	Contents	Shared	Size flag	Failure
Heap	Objects, arrays	Yes	`-Xms` , `-Xmx`	`OutOfMemoryError: Java heap space`
Metaspace	Class metadata	Yes	`-XX:MaxMetaspaceSize`	`OutOfMemoryError: Metaspace`
Compressed class space	Klass structures	Yes	`-XX:CompressedClassSpaceSize` (1 GB)	`OutOfMemoryError: Compressed class space`
Code cache	JIT native code	Yes	`-XX:ReservedCodeCacheSize` (240 MB)	Warning; JIT stops; slow
JVM stack	Frames	Per thread	`-Xss` (1 MB on Linux x64)	`StackOverflowError`
Direct memory	NIO buffers	Yes	`-XX:MaxDirectMemorySize`	`OutOfMemoryError: Direct buffer memory`

Metaspace vs PermGen. Before Java 8, class metadata lived in **PermGen**, a fixed-size part of the managed heap. Sizing it was guesswork, and `OutOfMemoryError: PermGen space` was a routine redeploy failure. Java 8 moved metadata to **metaspace** in native memory, which grows automatically and has **no limit by default**. Interned strings and static field values had already moved to the heap (Java 7). A classloader leak still fills metaspace; it just takes the whole machine's memory first if you do not set `MaxMetaspaceSize`.

A stack frame holds a local variable array (slot 0 is `this` for instance methods), an operand stack (bytecode is a stack machine), a reference to the constant pool for dynamic linking, and the return address. References sit on the stack; the objects they point to sit on the heap.

```
int add(int a, int b) { return a + b; }
```

```
Compiled from add(int, int)
 0: iload_1        // push local 1 (a) on the operand stack
 1: iload_2        // push local 2 (b)
 2: iadd           // pop two, push the sum
 3: ireturn        // return the top of stack
```

The **PC register** holds the current bytecode index per thread (undefined while in a native method). The **native stack** is used by JNI code.

Trap: `-Xmx` is not the memory the process uses. Total footprint is heap + metaspace + code cache + thread stacks + direct buffers + GC structures + native libraries. A service with `-Xmx2560m` can use well over 3 GB of RSS.

RefinePoint: native memory is not hypothetical for us. `DuckDbEngine` runs DuckDB in-process through JDBC, and its working memory is **outside the Java heap**, so `-Xmx` does not bound it. We bound it separately with `SET memory_limit` (default 40% of machine memory, shared among running engines) and let it spill to `temp_directory`. Any memory plan for run-service has to add the heap and that budget together.

11.4 Object layout

Every object is a header followed by fields, padded to 8 bytes.

```
64-bit HotSpot, compressed oops and class pointers on (the default below 32 GB heap)

new Object()                          new long[3]
+----------------------+ 8 bytes      +----------------------+ 8
| Mark word            |              | Mark word            |
+----------------------+ 4            +----------------------+ 4
| Klass pointer        |              | Klass pointer        |
+----------------------+ 4            +----------------------+ 4
| Padding              |              | Array length         |
+----------------------+              +----------------------+ 24
Total: 16 bytes                       | 3 elements of 8      |
                                      +----------------------+
                                      Total: 40 bytes
```

The **mark word** changes meaning with the lock state (Java 21, 64-bit):

Lowest bits	State	Upper bits hold
`01`	Unlocked	31-bit identity hash, 4-bit GC age
`00`	Thin (stack) locked	Pointer to lock record on the owner's stack
`10`	Inflated	Pointer to `ObjectMonitor`
`11`	Marked by GC	Forwarding information

- **Klass pointer:** points to class metadata in metaspace. It is what `invokevirtual` uses to find the vtable.
- **GC age** has 4 bits, which is why `MaxTenuringThreshold` cannot exceed 15.
- **Biased locking** added another state until Java 15 disabled it and later versions removed it.
- **Compact object headers** (JEP 519, a product feature in Java 25) squeeze the header to 8 bytes.

Compressed oops: a 64-bit reference is stored in 32 bits as an offset shifted by 3 (objects are 8-byte aligned), addressing 2^32 × 8 = 32 GB. Above about 32 GB they switch off, so every reference grows to 8 bytes; a 34 GB heap can hold *less* than a 31 GB one. Stay under the limit, or go well above it.

Field layout: the JVM reorders fields by size to minimise padding. A subclass's fields come after its parent's.

```
class RunRow { int id; boolean ok; long amount; Object ref; }
// 12 header + 4 id + 8 amount + 1 ok + 3 gap + 4 ref = 32 bytes. Inspect with JOL:
// System.out.println(ClassLayout.parseClass(RunRow.class).toPrintable());
```

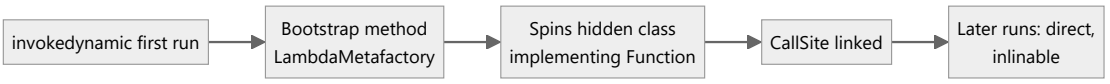
Memory cost, as a check on intuition: a boxed `Integer` is 16 bytes plus a 4-byte reference, against 4 for an `int`. `HashMap<Integer,Integer>` costs roughly 70+ bytes per entry. That is why a large reconciliation's rows are so heavy in the heap, and why engines store data in columnar form.

11.5 Bytecode and the `invoke*` instructions

`javap -c -v -p MyClass.class` disassembles a class file: constant pool, methods, bytecode, line and local-variable tables.

Instruction	Dispatch	Used for
`invokestatic`	Direct	Static methods
`invokespecial`	Direct, non-virtual	Constructors, `super.m()`, some private methods
`invokevirtual`	vtable slot	Instance methods on classes
`invokeinterface`	itable search	Instance methods via interface
`invokedynamic`	Bootstrap method links a `CallSite` at runtime	Lambdas, string concat, records, pattern switch

`invokedynamic` (Java 7) is a call site with no fixed target. On first execution the JVM runs a **bootstrap method**, which returns a `CallSite` holding a `MethodHandle`. Later executions go straight to that target, and the JIT can inline through it.



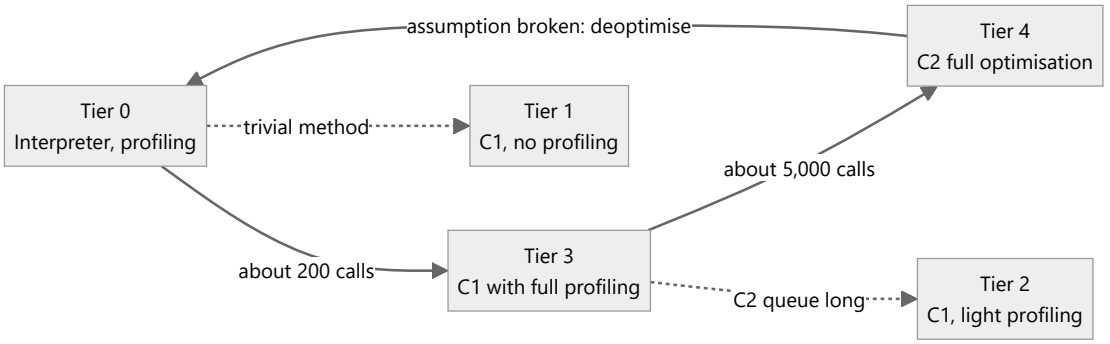
Uses since Java 8: lambdas (`LambdaMetafactory` , so no anonymous class files on disk), string concatenation from Java 9 (`StringConcatFactory`), `toString/equals/hashCode` of records (`ObjectMethods`), and pattern `switch` (`SwitchBootstraps`).

Since Java 11, nest-based access lets inner classes reach each other's private members with no synthetic accessors, and `private` calls may compile to `invokevirtual` .

11.6 The execution engine

Interpreter and JIT tiers

Bytecode starts out interpreted (a template interpreter generating native snippets at startup). HotSpot counts invocations and loop back-edges, then compiles hot code.



- **C1 (client):** compiles fast with light optimisation, adding profile counters at tier 3.
- **C2 (server):** compiles slowly but uses the profile to optimise aggressively.
- **On-stack replacement (OSR):** swaps a long-running loop from interpreted to compiled while it runs.
- **Warm-up:** this is why the first requests are slow and why a benchmark must discard warm-up iterations (use JMH).

What the JIT does

Optimisation	What it does
Inlining	Replaces a call with the body. The most important one, because it enables the others. Limits: `MaxInlineSize` 35 bytes for cold, `FreqInlineSize` 325 for hot
Devirtualisation	One observed receiver type → direct call with a guard; class-hierarchy analysis can remove the guard
Escape analysis	An object that never leaves the method: **scalar replacement** removes the allocation, **lock elision** removes its locks
Lock coarsening	Merges adjacent lock/unlock pairs on the same object
Loop opts	Unrolling, range-check elimination, loop-invariant code motion, auto-vectorisation (SIMD)
Dead code elimination	Removes unused results; why naive benchmarks measure nothing
Intrinsics	`System.arraycopy`, `String.equals`, CAS, `Math.*` replaced by hand-written CPU instructions

```
int total(List<Row> rows) {
    int sum = 0;
    for (Row r : rows) {           // the Iterator may never be allocated: inlined, then scalar-replaced
        sum += new Point(r.x(), r.y()).sum();   // Point does not escape; no allocation at all
    }
    return sum;
}
```

Speculation and deoptimisation

C2 optimises on assumptions: "this call site only sees `ArrayList`", "this branch never runs", "`Foo` has no subclasses". Each is guarded by an **uncommon trap**. When an assumption breaks (a new subclass is loaded, a rare branch is taken), the JVM **deoptimises**: it rebuilds the interpreter frame and continues, then recompiles later with better information. It shows up as a latency blip after a new code path first runs, for instance a second implementation of an interface loaded late.

Check it with `-XX:+PrintCompilation` (`made not entrant` lines), JFR's `jdk.Deoptimization` and `jdk.Compilation` events.

> **Trap:** `-XX:ReservedCodeCacheSize` too small: you get "CodeCache is full. Compiler has been disabled" and a mysterious gradual slowdown, because new hot methods stay interpreted. Watch it with `jcmd <pid> Compiler.codecache`.

Safepoints

JIT code polls at method returns and loop back-edges. To stop all threads for GC, deoptimisation or a biased-era revocation, the VM arms a safepoint and waits until every thread reaches a poll. A long counted loop

without polls used to delay everyone (loop strip mining reduced that). In a dump, "time to safepoint" is often the hidden source of a long pause.

11.7 Startup: AppCDS, AOT and native image

Technique	Idea	Gain	Cost
CDS (default)	JDK classes archived and memory-mapped	Faster startup, shared memory	None; on by default
AppCDS	Same for your classes	Often 30–50% less startup	Extra training run; same classpath required
Spring AOT + CDS (Boot 3.3+)	AOT-generated bean definitions plus archive	Faster context startup	Conditions fixed at build time
Project Leyden AOT cache (JEP 483, Java 24)	Also archives loaded and linked state	More savings	Newer JDKs only
CRaC	Snapshot a warmed process, restore it	Near-instant, stays warm	Linux and CRaC JDK; connections must reopen
GraalVM native image	Compile everything ahead of time	Tens of ms startup, low memory	Long builds, closed-world limits

```
java -Djarmode=tools -jar app.jar extract --destination app       # Boot 3.3+ layout
java -XX:ArchiveClassesAtExit=app.jsa -Dspring.context.exit=onRefresh -jar app/app.jar   # training run
java -XX:SharedArchiveFile=app.jsa -jar app/app.jar                                      # use it
```

Native image runs a **closed-world** static analysis at build time. Only code reachable from `main` is compiled into a standalone executable, with the Substrate VM runtime and no JIT. Reflection, proxies, resources and JNI need reachability metadata; Spring AOT (`mvn -Pnative native:compile`) generates that and pre-computes bean definitions.

	JVM (HotSpot)	Native image
Startup	Seconds	Milliseconds
Memory	Higher	Much lower
Peak throughput	Best, with JIT profile	Lower (PGO in Oracle GraalVM helps)
Dynamic class loading	Yes	**No**, closed world
Build	Fast	Minutes, memory-hungry
Best for	Long-running services	Scale-to-zero, CLI, functions

> **RefinePoint:** native image does not suit us. Dropping a plugin jar into `RYFN_PLUGINS_DIR` and restarting relies on run-time class loading, which a closed world forbids, and DuckDB's JNI library adds native configuration. Our services also run for weeks, so peak throughput beats startup. AppCDS, which keeps dynamic loading, would be the first step if startup time ever mattered.

11.8 Errors: `StackOverflowError` and the `OutOfMemoryError` family

Message	Meaning	First action
`Java heap space`	Heap full after GC	Heap dump; leak or undersized
`GC overhead limit exceeded`	Parallel GC: over 98% of time in GC for under 2% freed	Same; almost out of heap
`Metaspace`	Too much class metadata	Classloader leak, CGLIB or Groovy class spinning
`Compressed class space`	Klass area full	Same as metaspace; raise the 1 GB limit if legitimate
`Direct buffer memory`	NIO buffers beyond `MaxDirectMemorySize`	Netty/NIO leak; buffers not released
`unable to create native thread`	OS refused a thread	Thread leak, `ulimit -u`, cgroup `pids.max`, memory
`Requested array size exceeds VM limit`	Array over about `Integer.MAX_VALUE - 2`	Bug, usually an unbounded buffer
`Map failed`	`mmap` failure	File mappings, `vm.max_map_count`
(process killed, no Java error)	OS or container OOM killer	Native memory over the limit; check `dmesg`

`StackOverflowError`: a thread's stack is exhausted, nearly always by unbounded recursion (`toString` on a bidirectional JPA entity, `equals` cycles, a cyclic AOP call). It is an `Error` in that thread, and other threads continue. Deep recursion in legitimate code is worth rewriting iteratively; raising `-Xss` applies to every thread.

> **RefinePoint:** our rule language's recursive-descent parser and AST evaluator recurse in proportion to nesting depth, so a stack depth limit is part of the design question: a pathological user rule must yield a clear error, not a `StackOverflowError`.

Catching `OutOfMemoryError` is rarely safe, because the heap may be left in an inconsistent state. Our deployment takes the opposite approach:

```
# systemd unit for each service
ExecStart=/usr/bin/java -Duser.timezone=UTC -XX:+ExitOnOutOfMemoryError $JAVA_OPTS -jar
/opt/refinepoint/%i/%i.jar

# datasource-service: capture evidence as well
JAVA_OPTS="-Xmx2048m -XX:+UseSerialGC -XX:+HeapDumpOnOutOfMemoryError -
XX:HeapDumpPath=/opt/refinepoint/datasource/oom.hprof"
```

A service that runs out of memory exits and systemd restarts it, instead of staying up half broken. Because work is queued in PostgreSQL with heartbeats, the runs it held go back to the queue.

11.9 JVM flags you should know

Flag	Meaning
`-Xms`, `-Xmx`, `-Xss`	Initial heap, max heap, thread stack size
`-XX:MaxRAMPercentage=75`	Heap as a percentage of the container limit (no `-Xmx`)
`-XX:MaxMetaspaceSize=256m`	Cap metaspace
`-XX:MaxDirectMemorySize=`	Cap NIO direct buffers
`-XX:ReservedCodeCacheSize=`	JIT code cache
`-XX:+UseSerialGC`, `UseG1GC`, `UseZGC`	Choose a collector
`-XX:+HeapDumpOnOutOfMemoryError`, `HeapDumpPath`	Evidence on OOM
`-XX:+ExitOnOutOfMemoryError`, `CrashOnOutOfMemoryError`	Fail fast
`-Xlog:gc*`, `-Xlog:class+load`	Unified logging
`-XX:TieredStopAtLevel=1`	C1 only: fast startup, for dev or tests
`-XX:+PrintFlagsFinal`	Print every flag's final value
`-XX:NativeMemoryTracking=summary`	Enable `jcmd <pid> VM.native_memory`
`-XX:ActiveProcessorCount=`	Override detected CPUs

`jcmd <pid> VM.flags` shows a running JVM's values, and `jinfo` can change some manageable flags live.

11.10 Interview questions and answers

Q1. Explain the JVM architecture in one minute.

The JVM has three subsystems. The **class loader subsystem** finds `.class` bytes and loads, links (verify, prepare, resolve) and initialises classes. The **runtime data areas** are the shared heap, metaspace and code cache, plus per-thread stacks, PC registers and native stacks. The **execution engine** has an interpreter, the C1 and C2 JIT compilers that turn hot bytecode into native code, and the garbage collector. JNI connects it to native libraries. Then go one level deeper on whichever part the interviewer follows up on.

Q2. JDK vs JRE vs JVM?

The JVM runs bytecode. The JRE was JVM plus the standard libraries. The JDK is the JRE plus tools (`javac`, `jar`, `jcmd`, `jlink`, `jfr`). Since Java 11 there is no separate JRE download; you ship a JDK, or a trimmed runtime built with `jlink` containing only the modules you need.

Q3. Is Java compiled or interpreted?

Both. `javac` compiles source to platform-neutral bytecode. At run time the JVM interprets bytecode at first, then JIT-compiles hot methods to native machine code using profile data. With GraalVM native image, compilation is fully ahead of time instead.

Q4. What are the phases of class loading?

Loading (read bytes, create metadata and the `Class` object), linking (verification of bytecode safety, preparation that gives static fields default values, and resolution of symbolic references, lazily in HotSpot) and initialisation (run `<clinit>`: static initialisers and static blocks in textual order, exactly once, under a per-class lock).

Q5. When exactly is a class initialised?

On first active use: `new`, invoking a static method, reading or writing a non-constant static field, `Class.forName(name)` (with `initialize = true`, the default), reflection that instantiates it, initialising a subclass, or being the main class. Not on: referencing a compile-time constant, creating an array of the type, `X.class`, `ClassLoader.loadClass`, or accessing a parent's static field through a subclass name.

Q6. What does this print?

```
class A { static { System.out.print("A "); } static int x = 1; }
class B extends A { static { System.out.print("B "); } static final String S = "s"; static int y
= 2; }
System.out.print(B.x + " "); System.out.print(B.S + " "); System.out.print(B.y);
```

`A 1 s B 2`. `B.x` is declared in `A`, so only `A` initialises. `B.S` is a compile-time constant inlined by `javac`, so nothing initialises. `B.y` is a non-constant static of `B`, so `B` initialises now (its parent is already done).

Q7. Why is the holder-class singleton thread-safe without `volatile`?

```
class Config { private Config() {} private static class Holder { static final Config I = new
Config(); } static Config get() { return Holder.I; } }
```

`Holder` is initialised only when `get()` first reads `Holder.I`, and the JVM guarantees class initialisation runs once under a lock, with a happens-before edge to every thread that later uses the class. Laziness and safe publication come from the class-loading rules for free. An `enum` singleton gives the same plus serialisation safety.

Q8. What happens if a static initialiser throws?

The first access gets `ExceptionInInitializerError` wrapping the cause. The class is marked as failed, and **every later access** gets `NoClassDefFoundError: Could not initialize class X`, with no original cause (Java 17+ adds "Caused by: ExceptionInInitializerError: exception in initializer" detail). That is why searching logs for the *first* error matters: the root cause is printed once, at startup.

Q9. Name the built-in class loaders and what each loads.

Bootstrap (native code; `java.base` and core modules; represented as `null`), Platform (other JDK modules such as `java.sql`; replaced the Extension loader in Java 9) and Application or System (the classpath and module path). Custom loaders usually have the application loader as parent.

Q10. What is parent delegation and why does it exist?

A loader first checks whether it already loaded the class, then asks its parent, and only defines the class itself if the parent cannot. It prevents application code from replacing core classes (a fake `java.lang.String`), guarantees one definition of JDK classes, and avoids duplicate loading. To write a delegating loader, override `findClass`, not `loadClass`.

Q11. When is parent delegation deliberately broken?

Service providers loaded by JDK code (JDBC drivers, `ServiceLoader`, JAXP) use the **thread context class loader** to look downward into the application. Servlet containers use child-first loaders so each web app uses its own library versions. OSGi and JBoss Modules use a graph of loaders rather than a tree. Hot-reload tools such as Spring DevTools use a restart loader for your classes and a base loader for jars.

Q12. How are two classes with the same name considered different?

A runtime type is the pair (fully qualified name, defining class loader). If `com.acme.Foo` is defined by loader L1 and again by L2, they are unrelated types: casting between them throws `ClassCastException: com.acme.Foo cannot be cast to com.acme.Foo` (the message names both loaders), and static fields exist twice. Typical cause: a library in both the server's lib folder and the app.

Q13. `ClassNotFoundException` vs `NoClassDefFoundError`?

`ClassNotFoundException` is a checked exception thrown when code asks for a class **by name** (`Class.forName`, `loadClass`, `ServiceLoader`) and no loader finds it, often a missing dependency or wrong plugin path. `NoClassDefFoundError` is an `Error`: the class **was present at compile time** and is referenced directly, but at run time the JVM cannot get its definition. Causes are a jar missing at run time, a version conflict where a different jar lacks the class, or an earlier failed static initialiser ("Could not initialize class").

Q14. What is `NoSuchMethodError` and how do you debug it?

Bytecode calls a method signature that the loaded class does not have. The code was compiled against one version of a library and runs with another. Run `mvn dependency:tree -Dverbose` to find the two versions, check which jar won (`-verbose:class` or `-Xlog:class+load` prints the source jar of each class), and pin the version with `dependencyManagement`. Spring Boot's dependency BOM exists largely to prevent this.

Q15. How does a Spring Boot fat jar run if Java cannot load nested jars?

The manifest's `Main-Class` is Boot's `JarLauncher`, stored unnested at the root. It creates a `LaunchedClassLoader` (formerly `LaunchedURLClassLoader`) that reads `BOOT-INF/classes` and each `BOOT-`

`INF/lib/*.jar` in place, through a custom nested-jar URL handler (the jars are stored uncompressed, so entries can be read at an offset). It sets that loader as the thread context loader and calls the `Start-Class`'s `main` by reflection. For containers, `java -Djarmode=tools -jar app.jar extract` unpacks it into layers, which also enables CDS.

Q16. Why would you write a custom class loader?

Plugins loaded from a folder at run time, isolation of conflicting library versions, loading classes generated or downloaded at run time, encrypted or signed class files, and hot reloading (throw the loader away to unload its classes). Our `Plugins` class is the first case: a `URLClassLoader` over the plugin jars, parented by the service's loader, queried with `ServiceLoader`.

Q17. Can a class be unloaded?

Yes, but only together with its defining loader. When the loader, all its classes and all instances of those classes are unreachable, a GC can collect them and free their metaspace. Classes of the bootstrap, platform and application loaders are never unloaded. That is why redeploying an app into a long-running server can leak: one leftover reference (a `ThreadLocal`, a JDBC driver in `DriverManager`, a thread started by the app) pins the old loader and all its classes.

Q18. What lives in the heap and what lives in metaspace?

Heap: every object and array, `Class` mirror objects, static field values (inside the mirror since Java 8), and the interned string pool (since Java 7). Metaspace (native memory): class metadata (klass structures, method bytecode, constant pools, annotations, vtables). The JIT's compiled code lives in the code cache, also native.

Q19. Why was PermGen replaced by metaspace?

PermGen was a fixed-size region of the managed heap, hard to size, with frequent `OutOfMemoryError: PermGen space` on redeploys and frameworks that generate classes. Its collection was tied to full GCs. Metaspace (Java 8, JEP 122) lives in native memory, grows on demand, and is freed per class loader in chunks. It is unlimited by default, so set `-XX:MaxMetaspaceSize` to turn a slow leak into a clear error instead of a container OOM kill.

Q20. What causes `OutOfMemoryError: Metaspace` and how do you diagnose it?

Too many classes: a classloader leak on redeploy, unbounded class generation (CGLIB proxies created per call, Groovy or script engines compiling per request, reflection-based serializers spinning classes) or simply a large app with a tiny limit. Diagnose with `jcmd <pid> VM.classloader_stats` and `jcmd <pid> VM.metaspace`, `-Xlog:class+load` to see what is being loaded, and a heap dump: look for many instances of the same loader class, then trace paths to GC roots from the old ones.

Q21. What is in a stack frame?

A local variable array (parameters, with `this` in slot 0 for instance methods, then locals; `long` and `double` take two slots), an operand stack where bytecode instructions push and pop values, a reference to the

method's runtime constant pool for dynamic linking, and the return address and saved state. The frame's size is fixed at compile time (`max_locals` , `max_stack` in the class file).

Q22. Are local variables always on the stack and objects always on the heap?

Primitive locals and references are in the frame (or in CPU registers once compiled). Objects are logically on the heap. Escape analysis lets C2 remove an allocation entirely, by scalar replacement, when an object never escapes the compiled method. Its fields then live in registers, so in compiled code some "objects" never exist in memory. HotSpot does not do true stack allocation of objects, a frequent misstatement.

Q23. What determines how deep recursion can go?

The thread's stack size (`-Xss` , default 1 MB on 64-bit Linux, set per thread through the `Thread` constructor's `stackSize` hint) divided by the frame size, which differs between interpreted and compiled frames. That is why the same recursion fails at different depths in a cold and a warm JVM. Expect several thousand to tens of thousands of frames.

Q24. `StackOverflowError` vs `OutOfMemoryError` ?

`StackOverflowError` means one thread's stack is exhausted, nearly always by unbounded recursion (bidirectional JPA `toString` , cyclic `equals` , recursive bean calls). Only that thread is affected, and the stack trace shows the repeating frames. `OutOfMemoryError` means a shared memory area (heap, metaspace, direct memory, native threads) could not satisfy an allocation, potentially affecting every thread. Both are `Error` s and should not normally be caught.

Q25. List the kinds of `OutOfMemoryError` and their causes.

`Java heap space` (live data larger than the heap, or a leak), `GC overhead limit exceeded` (Parallel GC spends over 98% of time collecting and frees under 2%), `Metaspace` and `Compressed class space` (class metadata), `Direct buffer memory` (NIO and Netty off-heap buffers), `unable to create native thread` (OS thread or memory limits), `Requested array size exceeds VM limit` (an attempt to allocate an enormous array) and `Map failed` (mmap). A process killed by the Linux or container OOM killer prints no Java error at all, because native memory went over the limit.

Q26. What does `OutOfMemoryError: unable to create new native thread` mean?

The OS refused `pthread_create` . The heap is irrelevant. Causes: a thread leak (pools created per request, `newCachedThreadPool` under bursts), the user process limit (`ulimit -u`), the cgroup `pids.max` in containers, or native memory exhausted by stack reservations. Check the thread count with `jcmd <pid> Thread.print | grep -c '^"'` or `/proc/<pid>/status` , and look at which thread names multiply.

Q27. Why can a Java process use much more memory than `-Xmx` ?

`-Xmx` limits only the heap. Add metaspace, the compressed class space, the code cache, thread stacks (about 1 MB reserved each, committed as used), GC data structures (G1's remembered sets can be 10% or more of the heap), direct and mapped buffers, native libraries (a JDBC driver with native code, compression, DuckDB)

and malloc arenas. Measure with `-XX:NativeMemoryTracking=summary` and `jcmd <pid> VM.native_memory summary`. In containers, leave 25–40% headroom over the heap.

Q28. Describe the memory layout of a Java object.

A header, then fields, padded to a multiple of 8 bytes. On 64-bit HotSpot with compressed class pointers, the header is a 64-bit mark word (identity hash, GC age, lock state) plus a 32-bit klass pointer; arrays add a 32-bit length. `new Object()` is 16 bytes. Fields are reordered (longs and doubles, then ints, then shorts and chars, then bytes and booleans, then references) to reduce padding. Compact headers in Java 25 reduce the header to 8 bytes. JOL (`org.openjdk.jol`) prints exact layouts.

Q29. What is the mark word used for?

It stores the identity hash code (once computed), the GC age (4 bits, hence a maximum tenuring threshold of 15), and the lock state: unlocked, thin-locked (a pointer to a lock record on the owner's stack), inflated (a pointer to an `ObjectMonitor`), or GC-marked (forwarding information). Calling `System.identityHashCode` writes the hash into it, and in old JVMs that revoked biased locking.

Q30. What are compressed oops and why does 32 GB matter?

Compressed ordinary object pointers store 64-bit references as 32-bit offsets, scaled by 8 because objects are 8-byte aligned, addressing up to 32 GB. They save memory and cache space, often 20–30% of heap. Above about 32 GB (exactly where depends on the heap base) the JVM falls back to full 64-bit references, so a 33 GB heap can hold fewer objects than a 31 GB one. Check with `-XX:+PrintFlagsFinal | grep UseCompressedOops`. A larger `-XX:ObjectAlignmentInBytes=16` extends the range to 64 GB at the cost of padding.

Q31. How big is an `Integer`, and why do boxed collections hurt?

An `Integer` is 16 bytes (12 header + 4 value), plus a 4-byte reference wherever it is stored, against 4 bytes for an `int`. A `HashMap<Integer, Integer>` entry costs a `Node` (32 bytes), two `Integer`s (32) and a table slot, about 70 bytes to store 8 bytes of data. With millions of rows, choose primitive arrays, a primitive collections library, or a columnar engine.

Q32. What does `javap -c` show, and why would you use it?

The disassembled bytecode of each method; `-v` adds the constant pool, flags, stack sizes and attributes, and `-p` includes private members. Use it to see what the compiler really generated: string concatenation becoming `invokedynamic makeConcatWithConstants`, autoboxing as `Integer.valueOf`, enhanced `for` over arrays versus iterators, bridge methods for generics, synthetic lambda methods (`lambda$main$0`), and record methods via `ObjectMethods`.

Q33. Explain the five `invoke` instructions.

`invokestatic` calls static methods directly. `invokespecial` calls constructors, `super.method()` and some private methods without virtual dispatch. `invokevirtual` dispatches on the receiver's class through a vtable

slot. `invokeinterface` dispatches through an interface method table (itable), which needs a search. `invokedynamic` delegates linking to a bootstrap method at run time, which returns a `CallSite`. The JIT turns most virtual and interface calls into direct, inlined calls when profiling shows one or two receiver types.

Q34. How are lambdas implemented?

`javac` moves the lambda body into a private synthetic method (`lambda$process$0`) and emits an `invokedynamic` whose bootstrap is `LambdaMetafactory.metafactory`. On first execution it spins a hidden class implementing the functional interface that calls the synthetic method, and links the call site. A non-capturing lambda becomes a single cached instance; a capturing lambda allocates per evaluation (and escape analysis may remove that). No `Outer$1.class` files are written, unlike anonymous classes.

Q35. Why does string concatenation use `invokedynamic` since Java 9?

JEP 280 replaced the `StringBuilder` chains that `javac` used to generate with `invokedynamic` to `StringConcatFactory`. The JVM can then pick the best strategy at run time (exact sizing, no intermediate copies) and improve it in later JDKs without recompiling the code. Concatenation in a loop still builds a new string each iteration, so use a `StringBuilder` there.

Q36. How does the JIT decide what to compile?

HotSpot keeps per-method invocation counters and back-edge (loop) counters. Under tiered compilation, a method moves from the interpreter (tier 0) to C1 with profiling (tier 3) after roughly 200 invocations, or a combination with loop iterations, and to C2 (tier 4) after roughly 5,000 to 15,000, with thresholds adjusted by compiler queue length. Very small methods may go to C1 tier 1 and stay there. Compilation happens on background compiler threads while the code keeps running.

Q37. C1 vs C2?

C1 compiles quickly with basic optimisations (local inlining, constant folding, simple register allocation) and can insert profiling. C2 compiles slowly but optimises aggressively from the profile: deep inlining, escape analysis, loop transformations, vectorisation and speculative devirtualisation. Tiered compilation combines them, using C1 for fast warm-up and C2 for peak performance. Graal is an alternative JIT written in Java, the default in Oracle GraalVM.

Q38. What is on-stack replacement?

Switching a method that is currently running in the interpreter to compiled code in the middle of a loop. Without it, a `main` method with one long loop would never benefit from compilation, because it is only ever invoked once. The back-edge counter triggers an OSR compilation, and execution jumps into it at the loop header. `-XX:+PrintCompilation` marks these with `%`.

Q39. Why is inlining the most important JIT optimisation?

It removes call overhead, and, more importantly, it gives the optimiser a larger region to work on. After inlining, escape analysis can see that an object does not escape, constants propagate across former call

boundaries, and dead branches disappear. Small, focused methods are therefore free in hot code. Huge methods (over 8,000 bytes of bytecode, `HugeMethodLimit`) are not compiled at all by default, a rare but real performance trap with generated code.

Q40. What is escape analysis and what does it enable?

C2's analysis of whether an object can be seen outside the compiled method or thread. If it does not escape at all, **scalar replacement** replaces the object with its fields in registers, so there is no allocation and no GC cost. If it does not escape the thread, **lock elision** removes synchronisation on it (old `StringBuffer` and `Vector` code benefits). It only works on what was inlined, which ties it back to Q39.

Q41. What is deoptimisation and when does it happen?

Reverting compiled code to the interpreter when a speculative assumption fails: a class-hierarchy assumption is broken by loading a new subclass, an "unreachable" branch is reached (an uncommon trap), a type check fails at a call site assumed monomorphic, or a null was seen where none was profiled. The JVM rebuilds interpreter frames from the compiled frame's debug information and continues. The method is later recompiled with the new profile. Frequent deoptimisation loops are visible in `-XX:+PrintCompilation` ("made not entrant") and JFR.

Q42. Why does a Java service get faster after a few minutes, and what can you do about it?

JIT warm-up: code is interpreted, then C1-compiled, then C2-compiled with profile-guided optimisation, and caches and connection pools fill. Mitigations: send warm-up traffic before joining the load balancer (readiness probe after warm-up), AppCDS or the Leyden AOT cache for class loading, CRaC to restore an already-warm process, and avoiding heavy work in the first requests. Benchmark with JMH, which handles warm-up and dead-code elimination correctly.

Q43. What is the code cache, and what happens when it fills?

The native memory region holding JIT-compiled code, split into non-method, profiled and non-profiled segments (240 MB reserved by default with tiered compilation). When it fills, the JVM logs "CodeCache is full. Compiler has been disabled", and new hot methods stay interpreted, causing a gradual slowdown with no error. Raise `-XX:ReservedCodeCacheSize` and check with `jcmd <pid> Compiler.codecache`. Large apps with many generated proxies are typical victims.

Q44. What is a safepoint?

A point where a thread's state is fully known to the VM, so the VM can inspect or move its objects. Stop-the-world operations (most GC phases, deoptimisation, class redefinition, some `jcmd` operations) arm a safepoint and wait for every thread to reach a poll at a method return or loop back-edge. A thread in a long loop without polls delays everyone ("time to safepoint"). Unified logging `-Xlog:safepoint` shows reaching time and pause time separately.

Q45. What is CDS/AppCDS and how much does it help?

Class Data Sharing stores parsed and verified class metadata in an archive that is memory-mapped at startup, skipping that work and sharing the pages between JVMs on the same host. The JDK ships a default archive for core classes. AppCDS extends it to application classes: create one with `-XX:ArchiveClassesAtExit=app.jsa` on a training run and use it with `-XX:SharedArchiveFile=app.jsa`. Expect a meaningful cut in startup for Spring apps, with no change to how the app behaves at run time.

Q46. What is GraalVM native image and what are its limits?

An ahead-of-time compiler that analyses the application from `main` under a closed-world assumption and produces a standalone executable with a small runtime (Substrate VM). Startup is in milliseconds and memory is low. Limits: no loading of classes unknown at build time; reflection, proxies, resources and serialisation need metadata; builds are slow; peak throughput is usually lower than C2 without PGO; and some libraries are incompatible. Spring Boot 3's AOT engine generates the metadata and fixes bean definitions at build time, so `@Profile` s and `@Conditional` s are frozen then.

Q47. Would you use native image for RefinePoint?

No. Plugins are discovered at run time from `RYFN_PLUGINS_DIR` through `ServiceLoader` and a `URLClassLoader`, which a closed world rules out. DuckDB brings a JNI library, and the services are long-running, so peak throughput and JIT profiling matter more than startup. Native image is a better fit for something like a CLI or a scale-to-zero function. I would reach for AppCDS first if startup became a problem.

Q48. What does `-XX:+ExitOnOutOfMemoryError` do and why would you use it?

The JVM exits immediately on the first `OutOfMemoryError` instead of throwing it into whichever thread happened to allocate. After an OOM, caches, pools and half-finished operations can be left inconsistent, and a "zombie" service that is up but unable to answer is worse than one that restarts. We start every service with it under systemd, which restarts the process. With database-backed work queues and heartbeats, the runs it held are re-queued. Pair it with `HeapDumpOnOutOfMemoryError` where you need evidence, as we do for datasource-service.

Q49. Which JVM flags would you set for a service in a container?

No hard `-Xmx` if the memory limit varies; use `-XX:MaxRAMPercentage=70` to 75 (lower if heavy native memory such as DuckDB or Netty is involved), `-XX:+ExitOnOutOfMemoryError`, `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps`, a collector chosen deliberately (`UseG1GC` for most, `UseSerialGC` for small heaps and one CPU), `-Xlog:gc*:file=/logs/gc.log:time,uptime:filecount=5,filesize=20m`, `MaxMetaspaceSize` as a safety net, and `-XX:ActiveProcessorCount` only if CPU detection is wrong. Since JDK 10, the JVM reads cgroup v1 and v2 limits.

Q50. Our services run with `-XX:+UseSerialGC` and small heaps. Defend or attack that choice.

Defend: each service has a small heap (256–512 MB for most), and Serial has the lowest footprint and CPU overhead, with no concurrent GC threads competing on a shared, modest server. With small young

generations, pauses are a few milliseconds. Attack: run-service runs with `-Xmx2560m` and holds a reconciliation's rows in the heap, so old-generation collections there are single-threaded and could take seconds. That service is the one where G1 (or Parallel for throughput) deserves a measured trial. Chapter 12 covers the measurement.

Q51. How would you find which jar a class was loaded from in production?

`-Xlog:class+load=info` (or `-verbose:class`) at startup prints each class with its source. At run time, `jcmd <pid> VM.class_hierarchy` or `jcmd <pid> VM.classloaders`, or in code `Foo.class.getProtectionDomain().getCodeSource().getLocation()`. In a Boot fat jar the location points into `BOOT-INF/lib/<jar>`.

Q52. What is the difference between `Class.forName()` and `ClassLoader.loadClass()`?

`Class.forName(name)` loads **and initialises** the class (running static blocks), using the caller's class loader. `Class.forName(name, false, loader)` loads without initialising. `loader.loadClass(name)` only loads (and may not even link it). Old JDBC code used `Class.forName("org.postgresql.Driver")` because the driver's static block registers it with `DriverManager`; since JDBC 4, `ServiceLoader` does that automatically.

12. Garbage Collection & Performance Troubleshooting

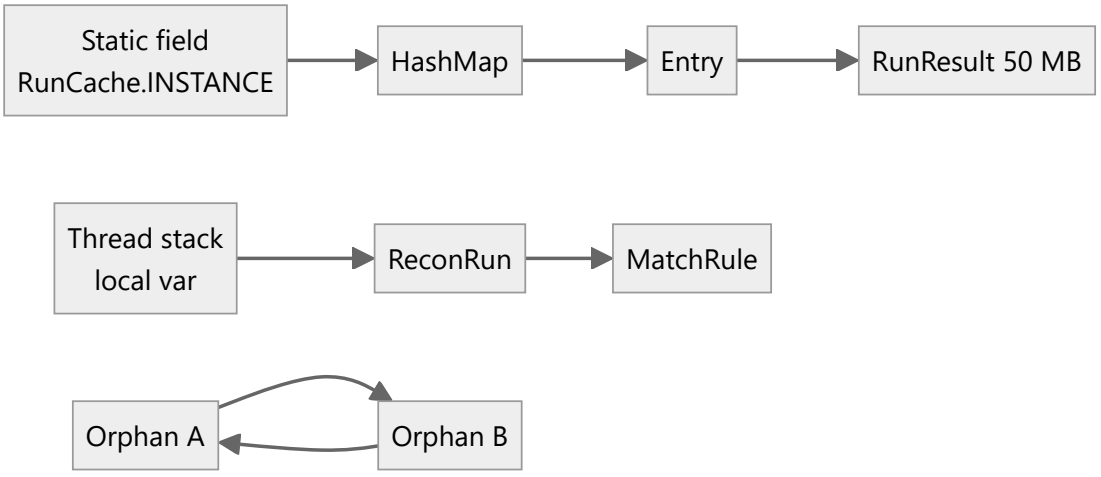
Garbage collection questions separate people who have read about GC from people who have been paged for it. The theory (generations, regions, barriers) is necessary, but the senior part of the interview is the incident: "p99 latency tripled at 10 a.m., CPU is at 400%, what do you do, in order?" This chapter builds the theory you need to read a GC log and a heap dump, then turns it into a step-by-step production playbook, with examples from RefinePoint's run-service, export worker and DuckDB engine.

12.1 What garbage is: reachability and GC roots

An object is **live** if it can be reached by following references from a **GC root**. Everything else is garbage, whatever its reference count. Cycles are collected without trouble, because the collector never looks at counts.

GC roots in HotSpot:

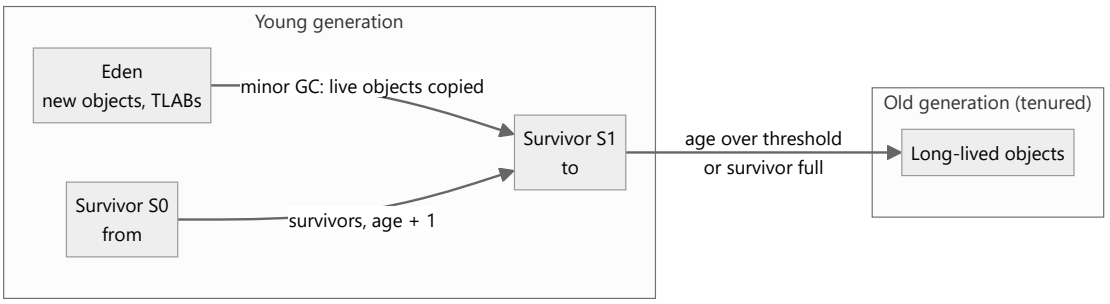
- Local variables and operand-stack slots in every live thread's frames (including JIT-compiled frames, through their oop maps)
- Static fields, which belong to `Class` objects that are reachable through their class loader
- JNI global and local references
- Objects used as monitors (`synchronized`) by running threads
- JVM internals: system class loaders, pending exceptions, the string table and other VM structures



`F` and `G` reference each other but no root reaches them, so both are garbage. `D` is live as long as the static map holds it — the shape of most real leaks: **a leak in Java is an object that is still reachable but no longer needed.**

12.2 The generational hypothesis and the young/old heap

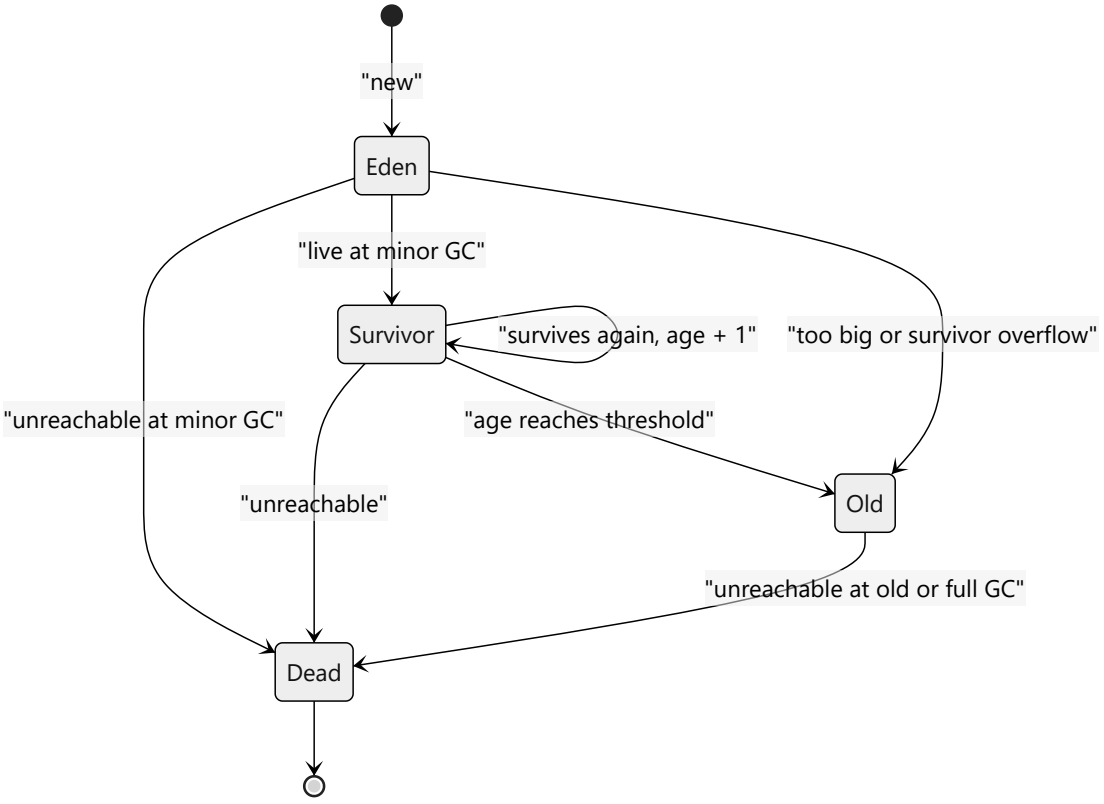
The **weak generational hypothesis**: most objects die young (request DTOs, iterators, temporary strings), and the few that survive tend to live a long time (caches, pools, Spring beans). So the heap is split, and the young part is collected often and cheaply.



How a minor GC works:

1. New objects are allocated in **Eden**, almost always in the thread's **TLAB** (thread-local allocation buffer), so allocation is a pointer bump with no locking.
2. When Eden is full, a **minor (young) GC** stops the application, finds live young objects starting from roots **plus the remembered set** (old-to-young references), and **copies** them to the empty survivor space. Eden and the "from" survivor are now entirely garbage and are reset in one step.
3. Each survivor's **age** (4 bits in the mark word) increases with every copy. At the **tenuring threshold** (adaptive, max 15) or when the survivor space overflows, it is **promoted** to the old generation.

The cost of a young GC is proportional to the **live** objects, not to the garbage — which is why allocation-heavy but short-lived workloads are cheap, and why a young GC with huge survivor volume is slow.



Old-to-young references. To collect only the young generation, the collector must know which old objects point into it, without scanning the whole old generation. A **write barrier** (a few instructions the JIT adds to every reference store) marks a **card** (a 512-byte region) dirty in the **card table** whenever an old object is updated. A minor GC scans only dirty cards. G1 refines this into per-region **remembered sets**.

Minor, major, full

Term	Collects	Notes
Minor / young GC	Young generation	Frequent, short, stop-the-world in most collectors
Major / old GC	Old generation	Often concurrent (G1 marking, ZGC); the term is loose
Mixed GC (G1)	Young + selected old regions	G1's normal way of reclaiming old space
Full GC	Whole heap + metaspace	Stop-the-world, compacting; in G1 it is a failure fallback

Mark, sweep, compact, copy

Algorithm	How	Pros	Cons
Mark-sweep	Mark live, free the rest into free lists	No copying	Fragmentation, slower allocation
Mark-compact	Mark live, slide them together	No fragmentation, bump allocation	Moving costs; needs reference updates
Copying	Copy live objects to an empty space	Cost ∝ live; compacts as a side effect	Needs spare space (survivors)

Young collections use copying (few survivors); old collections use mark-compact (Serial, Parallel, G1 full GC) or concurrent region evacuation (G1 mixed, ZGC, Shenandoah).

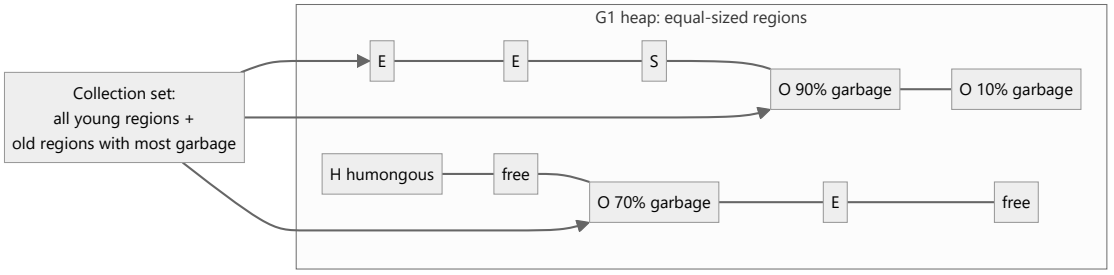
12.3 The collectors

Collector	Flag	Young	Old	Pauses	Best for
Serial	`-XX:+UseSerialGC`	Copying, 1 thread	Mark-compact, 1 thread	Short on small heaps, long on big	Small heaps, 1–2 CPUs, many small JVMs
Parallel	`-XX:+UseParallelGC`	Copying, N threads	Mark-compact, N threads	All STW, proportional to heap	Batch throughput
CMS	removed in Java 14	—	Concurrent mark-sweep	Low, fragmentation	Historical only
G1	default since Java 9	Region evacuation	Concurrent mark + mixed evacuation	Target 200 ms	General purpose, 1–64 GB
ZGC	`-XX:+UseZGC`	Generational since 21	Concurrent everything	Under 1 ms	Low latency, large heaps
Shenandoah	`-XX:+UseShenandoahGC`	Not generational (in 21)	Concurrent compaction	Low ms	Low latency (OpenJDK builds)
Epsilon	`-XX:+UseEpsilonGC`	none	none	none	Benchmarks, short-lived jobs

The JVM picks a collector if you do not: G1 on a "server-class" machine, **Serial** if it sees fewer than 2 CPUs or less than ~1,792 MB of memory — a frequent surprise in small containers.

G1: regions, young and mixed collections

G1 divides the heap into equal **regions** (1–32 MB, chosen so there are about 2,048; `-XX:G1HeapRegionSize`). Each region is Eden, Survivor, Old, Humongous or free at any moment, so generations are logical sets of regions, not contiguous areas.



The G1 cycle:

1. **Young-only phase:** young GCs evacuate Eden and Survivor regions to new regions in parallel, stop-the-world. G1 sizes the young generation to meet `-XX:MaxGCPauseMillis` (default 200 ms).
2. When old occupancy crosses the **IHOP** (initiating heap occupancy, adaptive, starting at 45%), G1 starts **concurrent marking** alongside the application, using snapshot-at-the-beginning (SATB) barriers to stay correct while references change.

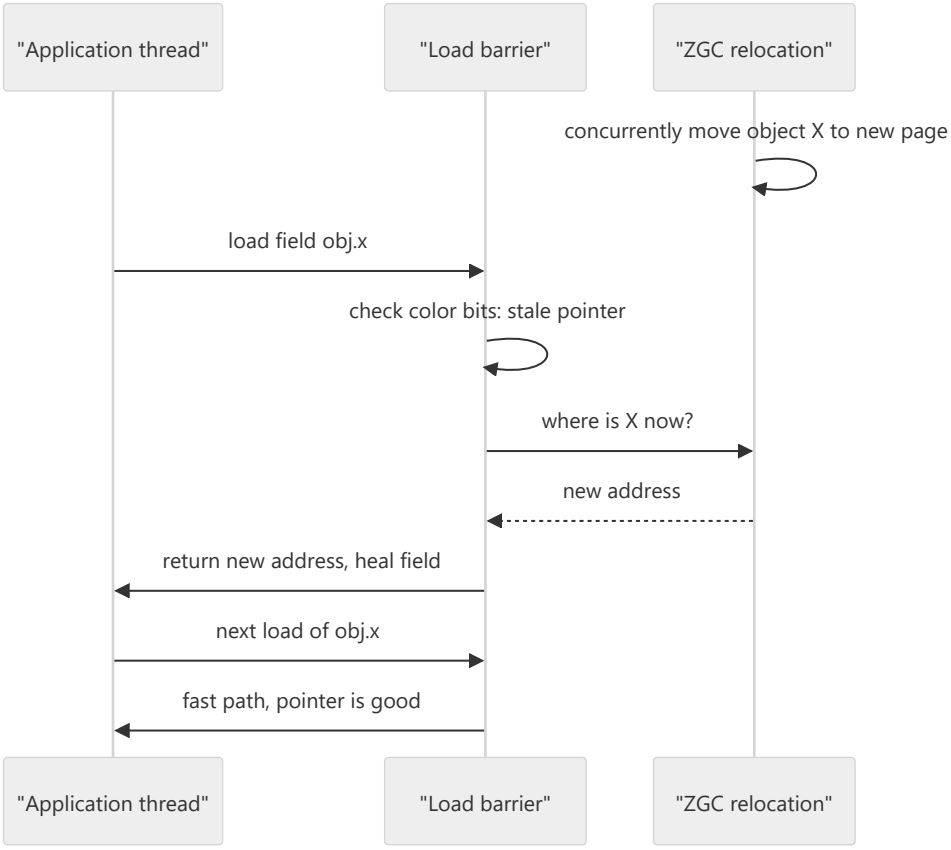
3. **Remark** and **cleanup** pauses finish marking; completely empty regions are freed immediately.
4. **Space-reclamation phase:** several **mixed GCs** evacuate young regions plus the old regions with the most garbage first ("garbage first"), within the pause budget.
5. If evacuation cannot find free regions (**to-space exhausted / evacuation failure**) or marking cannot keep up, G1 falls back to a **full GC** (parallel since Java 10, but still a long stop-the-world pause).

Humongous objects — any object of at least half a region — are allocated directly in contiguous old regions. Many large arrays (big `byte[]` buffers, huge `ArrayList` backing arrays) cause fragmentation and premature marking cycles. Fix with a larger `G1HeapRegionSize`, or by not allocating giant arrays.

ZGC: colored pointers and load barriers

ZGC does almost all work concurrently — marking, relocation (moving objects) and reference updating — with pauses that do not grow with heap or live-set size (typically well under 1 ms), for heaps from MBs to 16 TB.

- **Colored pointers:** ZGC stores metadata bits (marked, remapped, and in generational ZGC more state) *inside the 64-bit reference itself*. The pointer tells the collector the state of the object reference without touching the object.
- **Load barrier:** whenever the application loads a reference from the heap, a tiny check of those bits runs. If the pointer is "bad" (the object was moved, or needs marking), the barrier's slow path fixes it — relocates the object or looks up the new address — and **heals** the field in place, so the next load is fast.

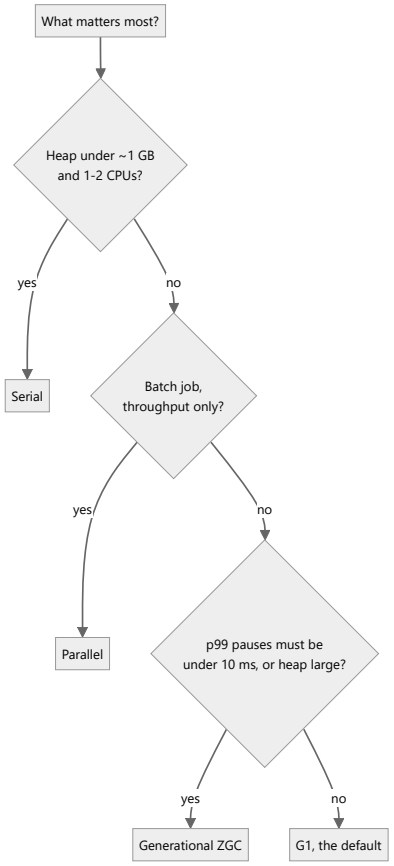


Generational ZGC (JEP 439, Java 21) adds young and old generations so short-lived objects are collected more cheaply: enable with `-XX:+UseZGC -XX:+ZGenerational` on Java 21. It became the default ZGC mode in Java 23, and the non-generational mode was removed in Java 24. Costs: barriers take some throughput (a few

percent), ZGC needs heap headroom because it collects while you allocate (if allocation outruns it, threads stall — "allocation stall" in the log), and compressed oops are not used.

Shenandoah also compacts concurrently, using load-reference barriers and forwarding pointers. It is in most OpenJDK builds (Red Hat, Amazon Corretto, Temurin) but not Oracle's JDK; generational mode arrived as experimental in Java 22.

Choosing a collector



RefinePoint: every service runs `-XX:+UseSerialGC` with an explicit heap sized to its job — `-Xmx256m` for the api-gateway, 384–512 MB for most services, `-Xmx2048m` for datasource-service and `-Xmx2560m` for run-service — all on one shared server. For the small heaps Serial is a sound choice: the smallest footprint, no concurrent GC threads competing for CPU with thirteen other JVMs, and young pauses of a few milliseconds. run-service is the honest exception to examine: it holds a reconciliation's rows on the heap, so a single-threaded old collection over 2.5 GB can take seconds. The pause hurts less than it sounds, because runs are background work, not request latency — but it is the first service where G1 or Parallel deserves a measured trial.

12.4 Tuning flags and GC logs

Start with the fewest flags. Most tuning is: choose the collector, set the heap, set a pause goal, log GC. Everything else needs evidence.

Flag	Purpose
`-Xms` = `-Xmx`	Fixed heap: no resize pauses, predictable footprint (common on servers)
`-XX:MaxRAMPercentage=75`	Heap as a share of container memory (default only 25%)
`-XX:MaxGCPauseMillis=100`	G1 pause target (a goal, not a guarantee)
`-XX:InitiatingHeapOccupancyPercent`	G1 marking start, if adaptive IHOP starts too late
`-XX:G1HeapRegionSize=16m`	Fewer humongous allocations
`-XX:ParallelGCThreads`, `-XX:ConcGCThreads`	GC thread counts (defaults follow CPU count)
`-XX:SoftMaxHeapSize` (ZGC)	Try to stay below this, allow growth to `Xmx`
`-XX:+AlwaysPreTouch`	Commit heap pages at startup, avoid page faults later
`-XX:+UseStringDeduplication`	G1/ZGC deduplicate identical `String` contents

GC logging (unified logging, Java 9+), cheap enough to always leave on:

```
-Xlog:gc*,safepoint:file=/var/log/app/gc-%p.log:time,uptime,level,tags:filecount=5,filesize=20m
```

```
[2026-10-30T10:02:11.481+0000][3605.112s][info][gc] GC(812) Pause Young (Normal) (G1 Evacuation Pause) 1520M->410M(2560M) 18.204ms
[2026-10-30T10:02:40.002+0000][3633.633s][info][gc] GC(813) Pause Young (Concurrent Start) (G1 Humongous Allocation) 1630M->1490M(2560M) 41.730ms
[2026-10-30T10:02:44.910+0000][3638.541s][info][gc] GC(814) Pause Full (G1 Compaction Pause) 2550M->2390M(2560M) 3812.445ms
```

How to read it: `before->after(capacity)` and the pause time. Line 1 is healthy (most of Eden was garbage). Line 2 was triggered by a humongous allocation and freed little — the live set is growing. Line 3 is a full GC that reclaimed only 160 MB of 2.5 GB: the live set nearly fills the heap. That is a leak or an undersized heap, and an `OutOfMemoryError` is close. Tools such as GCeasy or GCViewer chart these logs; the numbers to extract are **pause percentiles, throughput (% time not in GC), allocation rate, promotion rate and live set after full/old GC**.

12.5 Reference types

Type	Cleared when	`get()`	Use
Strong	Never while reachable	the object	Normal references
`SoftReference`	Before `OutOfMemoryError`, under memory pressure (LRU-ish: `-XX:SoftRefLRUPolicyMSPerMB`)	object or `null`	Memory-sensitive caches (with caution)
`WeakReference`	At the next GC once only weakly reachable	object or `null`	Canonicalising maps, `WeakHashMap` keys, `ThreadLocalMap` keys, listener registries
`PhantomReference`	After the object is finalisable; enqueued for post-mortem cleanup	always `null`	Releasing native resources (`Cleaner`)

```
// Cleaner (Java 9+): the replacement for finalize()
public final class NativeBuffer implements AutoCloseable {
    private static final Cleaner CLEANER = Cleaner.create();
    private final Cleaner.Cleanable cleanable;

    public NativeBuffer(long size) {
        long address = allocateNative(size);
        // the cleanup action must NOT reference 'this', or the object can never become phantom reachable
        this.cleanable = CLEANER.register(this, () -> freeNative(address));
    }
    @Override public void close() { cleanable.clean(); }   // deterministic path; Cleaner is the safety net
}
```

Trap: soft-reference caches look attractive ("the GC will free it when needed") but behave badly: the JVM clears them in bulk under pressure, causing a cache stampede exactly when the system is already struggling, and they keep the heap full, which makes GC work harder. Use a bounded cache with explicit size and expiry (Caffeine), or a small bounded map like our `TtlCache`.

`finalize()` is deprecated for removal (JEP 421): objects with finalisers need two GC cycles to reclaim, finalisers run on one thread in unspecified order, and they can resurrect objects.

12.6 Memory leaks in Java

Source	How it leaks	Fix
Static collections	`static Map` that is added to but never removed from	Bound it, expire entries, or do not cache
Unbounded caches	Per-key cache keyed by request data (ids, SQL text)	Size limit + TTL (Caffeine), metrics on size
`ThreadLocal` in pools	Value stays on a pooled thread forever, and `ThreadLocalMap` entries keep values strongly	`remove()` in `finally`
Listeners and callbacks	Registered, never deregistered; the publisher keeps subscribers alive	Deregister on close, or weak listeners
Inner and anonymous classes	Non-static inner class holds `Outer.this`	Static nested class
Classloader leaks	One reference from a parent loader (a thread, a JDBC driver, a `ThreadLocal`) pins all classes of an old loader	Clean shutdown hooks; avoid redeploy-in-place
JPA persistence context	A batch job loads 1 million entities in one `EntityManager`; the first-level cache holds all of them	`flush()` + `clear()` every N, or `StatelessSession`/JDBC
Unclosed resources	Streams, result sets, direct buffers	try-with-resources
Collections holding results	Whole result set read into a `List`	Stream with a fetch size, process in chunks

> **RefinePoint:** `TtlCache` in ryfn-platform caches answers from other services (access decisions, plan limits, API keys) and is bounded the bluntest way possible: when it reaches `maxEntries` it is **cleared**. The javadoc states the trade-off — a clear only costs a few extra calls — and that is the right call for a cache that exists to save latency, not to hold state. An unbounded `ConcurrentHashMap` keyed by API key would have been a slow leak sized by the number of distinct callers.

Finding a leak: heap dumps and Eclipse MAT

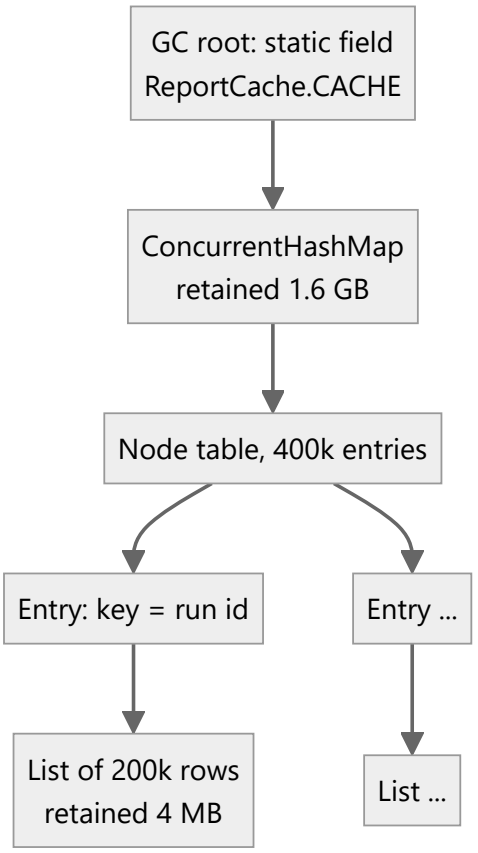
```
jcmd <pid> GC.class_histogram | head -25          # quick look: which classes have the most instances/bytes
jcmd <pid> GC.heap_dump /tmp/heap.hprof           # full dump (live objects only by default via a full GC)
jmap -dump:live,format=b,file=/tmp/heap.hprof <pid>   # the older equivalent
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps   # capture automatically at the moment of failure
```

A heap dump pauses the JVM and writes a file about the size of the used heap — take it from an instance that is out of the load balancer if you can, and make sure the disk has space.

In **Eclipse MAT**:

- **Shallow heap** is an object's own size; **retained heap** is everything that would be freed if it were collected.

- The **dominator tree** lists objects by retained heap. Object X *dominates* Y if every path from the roots to Y goes through X. The top entries are almost always the leak.
- **Leak Suspects** report does the first pass automatically.
- **Path to GC roots (excluding weak/soft references)** on a suspect shows *who* holds it — the chain that ends at a static field, a thread, or a class loader.
- Compare two dumps taken minutes apart (or histograms) to see what grows.



Reading this: one map dominates 1.6 GB; it is held by a static field; its keys are run ids that are never removed. The fix is in the code (bound or remove the cache), not in `-Xmx`.

12.7 Thread dumps, JFR and profilers

Thread dumps

```
jcmd <pid> Thread.print > dump1.txt      # or: jstack -l <pid>, or kill -3 <pid> (to stdout)
```

Read a dump for: thread **states** (`RUNNABLE`, `BLOCKED` on a monitor, `WAITING` / `TIMED_WAITING` parked), **what they wait on** (`- waiting to lock <0x...>` and who `- locked <0x...>` it), pool names (`http-nio-8080-exec-*`, `export-*`, `run-*`), and the deadlock section the JVM appends automatically. Always take **three dumps a few seconds apart**: a thread in the same frame every time is stuck; one that moves is just busy.

High CPU recipe:

```
top -H -p <pid>                      # per-thread CPU: note the hottest TID, e.g. 48213
printf '%x\n' 48213                  # -> bc55
```

```
grep -A 20 'nid=0xbc55' dump1.txt    # the Java stack of that thread
```

If the hot threads are `GC Thread#n` or `VM Thread`, the problem is GC (go to the heap). If they are application threads, the stack shows the code: a tight loop, regex backtracking, a huge sort, JSON serialisation of a giant object.

Java Flight Recorder and Mission Control

JFR is a low-overhead (about 1% with the default settings) event recorder built into the JVM: CPU samples, allocations (by stack), GC phases, locks and monitor contention, thread parking, I/O, exceptions, class loading, safepoints, virtual-thread pinning.

```
java -XX:StartFlightRecording=duration=120s,filename=/tmp/rec.jfr,settings=profile -jar app.jar
jcmd <pid> JFR.start name=incident settings=profile duration=120s filename=/tmp/incident.jfr
jcmd <pid> JFR.dump name=incident                    # dump what is in the buffer now
jfr print --events jdk.GarbageCollection /tmp/incident.jfr | head
jfr view hot-methods /tmp/incident.jfr               # Java 21 built-in summary views
```

Open the file in **JDK Mission Control** for flame graphs, the allocation view ("who allocates the most"), the lock view and the automated analysis page. A continuous recording with a rolling buffer (`maxage=1h`) in production means you can dump the last hour *after* an incident.

async-profiler

A sampling profiler that combines `perf_events` with HotSpot's `AsyncGetCallTrace`, so it has **no safepoint bias** (safepoint-based profilers only see threads at safepoint polls and misattribute time) and includes native and kernel frames.

```
asprof -d 30 -e cpu   -f /tmp/cpu.html   <pid>      # CPU flame graph
asprof -d 30 -e alloc -f /tmp/alloc.html <pid>      # allocation flame graph
asprof -d 30 -e lock  -f /tmp/lock.html  <pid>      # lock contention
asprof -d 30 -e wall  -t -f /tmp/wall.html <pid>    # wall clock per thread: finds waiting, not
just burning
```

Wall-clock mode is the one people forget: a slow endpoint that uses no CPU is waiting — on a connection pool, a lock, or a downstream — and only a wall-clock profile shows where.

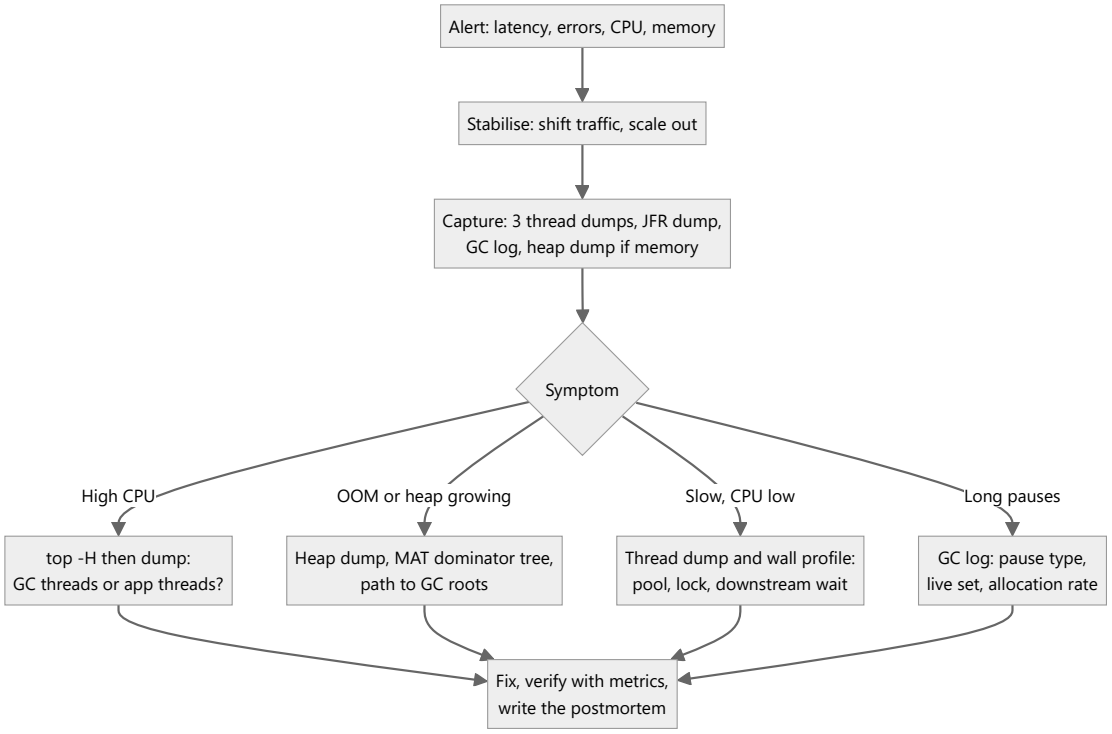
12.8 The JVM in containers

- Since JDK 10 (and 8u191) the JVM is **container-aware** (`-XX:+UseContainerSupport` , default on): it reads cgroup memory and CPU limits instead of the host's. cgroup v2 support came in JDK 15 (backported to 11.0.16 and 8u372).
- Without `-Xmx`, the default max heap is **25% of the container limit** (`MaxRAMPercentage=25`) — far too small for most services. Set `-XX:MaxRAMPercentage=60` – `75`, leaving room for metaspace, threads, direct buffers and native libraries.
- `availableProcessors()` follows the CPU quota, which sizes GC threads, the common pool and many library pools. A 1-CPU limit also switches the default collector to Serial.
- **OOMKilled is not an `OutOfMemoryError`.** If the *process* exceeds the cgroup limit, the kernel kills it with SIGKILL: no Java exception, no heap dump, exit code 137. That means native memory (or a heap set too

close to the limit). Diagnose with Native Memory Tracking: `-XX:NativeMemoryTracking=summary` and `jcmd <pid> VM.native_memory summary`.

12.9 The production incident playbook

Every incident follows the same discipline: **stabilise, capture evidence, then diagnose.** Restarting first destroys the evidence; diagnosing first prolongs the outage. Capture fast (thread dumps, a JFR dump, GC log, a heap dump if memory is the symptom), then take the instance out of rotation or restart.



High CPU

1. Is it the JVM? `top`. Which threads? `top -H -p <pid>`.
2. GC threads hot → it is really a memory problem: check the GC log for back-to-back collections and a live set near the max. Go to the OOM path.
3. Application threads hot → map TID to `nid` in a thread dump, or take a 30 s async-profiler CPU flame graph. Typical culprits: an infinite loop on a corrupted `HashMap` (old JDKs), catastrophic regex backtracking, a sort or JSON serialisation of a huge collection, logging in a hot loop, busy-spin retry without back-off.
4. Fix the code; add a guard (input size limits, regex timeouts via pre-validation).

`OutOfMemoryError` or a steadily growing heap

1. Which OOM? The message tells you the area (heap, metaspace, direct, native thread). Chapter 11 lists each.
2. Heap: get the dump (`HeapDumpOnOutOfMemoryError` should already have written it). In MAT, dominator tree → biggest retainer → path to GC roots.

3. Distinguish a **leak** (live set after full GC keeps rising over hours) from **undersizing** (live set is stable but a legitimate peak does not fit). A leak needs a code fix; undersizing needs a bigger heap or less data held at once.
4. Prefer fixing the data flow: stream instead of loading everything, bound caches, clear persistence contexts.

Slow responses, CPU normal

Threads are waiting, not working. Thread dumps show where: many `http-nio` threads in `HikariPool.getConnection` (DB pool exhausted — find the slow query or the transaction held across a remote call), in `SocketInputStream.read` to one host (a slow downstream with no timeout), `BLOCKED` on one monitor (lock contention). A wall-clock profile confirms. Fixes: timeouts, narrower transactions, bulkheads, indexes, caching.

Long GC pauses

1. From the GC log: what kind of pause? Young pauses scaling with survivor volume, mixed pauses, or **full GCs**?
2. Full GCs in G1 mean evacuation failure or marking that started too late: give more headroom (bigger heap, lower IHOP), find humongous allocations, reduce allocation rate.
3. Is the pause actually GC? Check `safepoint` logs — time-to-safepoint can dominate. Also check the host: swapping or CPU throttling (container CPU quota) can stretch any pause.
4. If the latency requirement is strict and the heap large, move to generational ZGC.

> **RefinePoint:** we built the memory playbook into the services rather than relying on reacting to it. (1) **Admission control:** `RunWorker` claims another run only while at least `runs.min-free-heap-percent` (default 25%) of its heap is free, measured from `Runtime.totalMemory() - freeMemory()` against `maxMemory()`. (2) **Containment:** if a run still fails with `OutOfMemoryError`, `ReconciliationExecutor` records it as failure category `RESOURCES` with a plain-language reason instead of leaving the run `RUNNING`. (3) **Fail fast:** every service starts with `-XX:+ExitOnOutOfMemoryError`, so in deployment a JVM-level OOM exits the process and systemd restarts it; the heartbeat re-queues its work. (4) **Evidence:** datasource-service also writes `HeapDumpOnOutOfMemoryError` to a known path. (5) **Early warning:** a Prometheus rule alerts when a service's heap is above 90% of max. Note the tension between (2) and (3): with the exit flag on, the JVM exits before the catch block can run, so the catch mainly matters where the flag is not set. Saying that out loud in an interview shows you understand both layers.

Large files: DuckDB and the export worker

Two patterns from our code that keep memory flat regardless of input size:

- **Push the data out of the heap.** The match engine runs on DuckDB in-process. DuckDB's memory is native and bounded by `SET memory_limit` (default 40% of machine memory, divided among runs active at that moment), and anything beyond it **spills to** `temp_directory` on disk. A 5-million-row reconciliation becomes a disk-I/O problem instead of an `OutOfMemoryError`. Two consequences: container and host memory budgets must count DuckDB's share on top of `-Xmx`, and disk space for the temp folder is now a monitored resource (our disk alert names "run's DuckDB temp folder" among the usual suspects).

- **Stream, never materialise.** `ExportWriter` reads rows with a JDBC fetch size of 2,000 on a forward-only statement inside a read-only `REPEATABLE_READ` transaction (pgjdbc streams only with a fetch size and autocommit off; otherwise it loads the whole result into memory), hands each row to a `RowCallbackHandler`, and writes Excel with `SXSSFWorkbook(500)`, which keeps a window of 500 rows in memory and flushes older rows to a temp file. The heap cost of an export is a few thousand rows, whether the file has 10,000 rows or 2 million. With `exports.workers` at 2, the worst case per instance is two such windows.

12.10 Interview questions and answers

Q1. How does the JVM decide an object is garbage?

By reachability, not reference counting. Starting from GC roots (thread stacks, static fields of loaded classes, JNI references, held monitors, VM internals), the collector traces every reference. Whatever it does not reach is garbage, including cycles of objects that reference each other. That is why Java has no cycle leaks, and why its real leaks are objects still reachable from a root but no longer needed.

Q2. What are GC roots?

Local variables and operand-stack slots of every live thread's frames, static fields (through their `Class` objects and the class loaders that keep those reachable), JNI local and global references, objects used as locks by running threads, and VM-internal references such as system class loaders and the string table. In MAT, "path to GC roots" ends at one of these, and the kind of root usually names the culprit.

Q3. What is the generational hypothesis?

Most objects die young, and those that survive tend to live long. So the heap is split: a young generation collected often and cheaply by copying the few survivors, and an old generation collected rarely. Young collection cost is proportional to live objects, not garbage, so high allocation rates of short-lived objects are cheap.

Q4. Walk through a minor GC.

Eden fills (allocation goes to per-thread TLABs in Eden). The JVM stops application threads, finds live young objects from the roots plus the remembered set or dirty cards (old-to-young references), copies them into the empty survivor space, increments their age, and promotes those past the tenuring threshold, or those that do not fit, into the old generation. Eden and the source survivor are then empty, and allocation resumes with a pointer bump.

Q5. Why are there two survivor spaces?

Copying collection needs an empty destination. On each minor GC, live objects from Eden and the "from" survivor are copied into the "to" survivor, and then the roles swap. One survivor is always empty, which costs a little memory but means no fragmentation and no free lists in the young generation. `-XX:SurvivorRatio` sets Eden's size relative to one survivor (default 8:1:1).

Q6. What is the tenuring threshold, and what is premature promotion?

The number of minor GCs an object must survive before promotion: adaptive, up to 15, because age has 4 bits in the mark word. Premature promotion happens when medium-lived objects (say, a request batch that lives a few seconds) are promoted because survivors are too small or the threshold dropped. They then die in the old generation and need expensive old or mixed collections. In a GC log, a high promotion rate with a stable live set points to it. Bigger survivors or a bigger young generation fix it.

Q7. How does a minor GC find old-to-young references without scanning the old generation?

Write barriers. Every reference store compiled by the JIT also marks a card (a 512-byte heap region) dirty in the card table. A minor GC scans only dirty cards in the old generation as extra roots. G1 keeps per-region remembered sets fed by a refinement thread. That barrier is part of the cost of every reference write in a generational collector.

Q8. Minor vs major vs full GC?

Minor (young) collects only the young generation. Major is loosely "old generation", often concurrent in G1 and ZGC. Full GC collects the whole heap (and metaspace), stop-the-world and compacting. In G1, a full GC is a failure fallback (evacuation failure, marking too slow, explicit `System.gc()`, metaspace threshold), so frequent full GCs in G1 are always a finding.

Q9. Explain mark-sweep, mark-compact and copying.

Mark-sweep marks live objects and frees the rest into free lists: no moving, but fragmentation and slower allocation. Mark-compact marks, then slides live objects together, giving no fragmentation and bump allocation at the cost of moving objects and updating references. Copying moves live objects into an empty space: cost proportional to live data, compaction for free, but it needs spare space. Young generations copy, old generations compact, and modern collectors evacuate regions concurrently.

Q10. What is a stop-the-world pause and why do collectors need it?

All application threads are halted at a safepoint so the collector sees a consistent object graph. Concurrent collectors shrink these pauses to short phases (root scanning, remark) by doing most work while the application runs, with barriers tracking changes. ZGC keeps its pauses independent of heap size. A pause also includes time-to-safepoint, the time for every thread to reach a poll.

Q11. Compare Serial, Parallel, G1 and ZGC.

Serial uses one GC thread with stop-the-world copying and compaction: the smallest footprint, fine for small heaps and few CPUs. Parallel does the same with many threads, giving the best raw throughput, with pauses that grow with the heap; it suits batch work. G1, the default, is region-based, with parallel young GCs, concurrent marking and mixed collections, aiming at a pause target (200 ms); it is the general-purpose choice. ZGC is concurrent marking and relocation with colored pointers and load barriers, sub-millisecond pauses at any heap size, generational since Java 21, trading a few percent of throughput and needing headroom.

Q12. Why was CMS removed?

CMS (concurrent mark-sweep) did not compact, so the old generation fragmented until a promotion failed and a long single-threaded full GC compacted everything. It was also complex to tune and maintain. G1 replaced it as the default in Java 9; CMS was deprecated in 9 and removed in 14 (JEP 363).

Q13. How does G1 work?

The heap is about 2,048 equal regions, each currently Eden, Survivor, Old, Humongous or free. Young GCs evacuate the young regions in parallel. When old occupancy crosses the adaptive IHOP, concurrent marking (SATB) measures the live data per region. Mixed GCs then evacuate young regions plus the old regions with the most garbage first, sized to fit `MaxGCPauseMillis`. If free regions run out, a full GC runs.

Q14. What is a humongous object in G1, and why does it matter?

An object of at least half a region, allocated directly into one or more contiguous old regions. Many large, short-lived arrays (large `byte[]` buffers, big `ArrayList` backings, huge `String`s from JSON) waste space and fragmentation, and trigger concurrent cycles. Spot them as "G1 Humongous Allocation" GC causes, then raise `G1HeapRegionSize` or stop allocating giant arrays, for instance by streaming instead of buffering whole payloads.

Q15. What does `MaxGCPauseMillis` do, and what happens if you set it to 10?

It is G1's pause goal (default 200 ms); G1 adapts young-generation size and the number of old regions per mixed GC to meet it. Setting it very low shrinks the young generation, which means more frequent GCs, more promotion, lower throughput, and eventually missed goals or full GCs. For real single-digit-millisecond needs, switch to ZGC rather than over-tightening G1.

Q16. What is an evacuation failure ("to-space exhausted")?

G1 could not find free regions to copy live objects into during a collection. Objects stay in place, the pause gets much longer, and often a full GC follows. Causes: a heap too small for the live set, an allocation spike, marking starting too late, or many humongous objects. Fix with more headroom (bigger heap, lower `InitiatingHeapOccupancyPercent`, `G1ReservePercent`) or less live data.

Q17. How does ZGC achieve sub-millisecond pauses?

It does marking, relocation (moving objects) and reference fixing concurrently. Metadata bits live in the references themselves (colored pointers), and a load barrier on every reference load checks them. If a reference is stale, the barrier's slow path finds the object's new location, or helps relocate it, and heals the field. Pauses only scan thread roots, so they stay below a millisecond regardless of heap or live set size.

Q18. What is a load barrier, and how is it different from a write barrier?

A load barrier runs on every reference *read* from the heap (ZGC, Shenandoah), so the application always sees a correct, up-to-date pointer even while objects move concurrently. A write barrier runs on reference *stores*

(card marking, G1's SATB and remembered sets) to track graph changes for the collector. Load barriers are executed far more often, so they must be very cheap on the fast path: a test and a branch.

Q19. What is generational ZGC and how do you enable it?

ZGC with separate young and old generations (JEP 439, Java 21), so short-lived objects are reclaimed in cheap young collections, which lowers CPU use and allocation stalls compared to single-generation ZGC. On Java 21 use `-XX:+UseZGC -XX:+ZGenerational`. It is the default mode from Java 23, and the non-generational mode was removed in Java 24.

Q20. What is an allocation stall in ZGC?

Application threads allocate faster than ZGC can reclaim memory concurrently, so they block until memory frees up. It appears as latency spikes, not GC pauses, and as "Allocation Stall" in the log. Fix it with more heap headroom, more concurrent GC threads, a lower allocation rate, or generational ZGC.

Q21. How do you choose a garbage collector?

Start with the default (G1) unless you have a reason. Use Serial for small heaps of 1 GB or less on 1–2 CPUs, or many small JVMs on one host. Use Parallel for batch jobs where only total throughput counts. Use generational ZGC when p99 latency must not include pauses over a few milliseconds, or for very large heaps. Then validate with GC logs under realistic load: pause percentiles, GC CPU and throughput.

Q22. Which collector does the JVM pick without flags, and why does that matter in containers?

G1 on a "server-class machine" (at least 2 CPUs and about 1,792 MB of memory). Otherwise Serial. In a container limited to 1 CPU or 1 GB you silently get Serial, plus a default max heap of 25% of the limit. Many "Java is slow in Kubernetes" stories are exactly that. Set the collector and `MaxRAMPercentage` explicitly.

Q23. How do you enable and read GC logs?

`-Xlog:gc*:file=gc.log:time,uptime,level,tags:filecount=5,filesize=20m` (Java 9+). Each pause line gives the type and cause, `before->after(capacity)` and duration. Track pause percentiles, GC throughput (percentage of time not paused), allocation rate, promotion rate and, most importantly, the **live set after old or full collections**. A rising live set over hours means a leak; a stable live set close to the max means an undersized heap.

Q24. Should you set `-Xms` equal to `-Xmx`?

On a server, usually yes: the heap is committed and sized once, with no resize-triggered GCs, and the footprint is predictable for capacity planning (add `AlwaysPreTouch` to commit pages at startup). On a desktop or shared dev machine, a smaller `-Xms` returns memory to the OS. In containers, the equivalent is `InitialRAMPercentage` equal to `MaxRAMPercentage`.

Q25. Why is calling `System.gc()` a bad idea?

It requests a full, stop-the-world collection, usually long, and it is almost always worse than letting the collector decide. Libraries sometimes call it (RMI's distributed GC every hour, some direct-buffer code), so `-XX:+DisableExplicitGC` exists, though that can break code that relies on it to free direct buffers. With G1, `-XX:+ExplicitGCInvokesConcurrent` turns it into a concurrent cycle instead.

Q26. Explain strong, soft, weak and phantom references.

Strong references keep objects alive. Soft references are cleared only under memory pressure, before an OOM, and were intended for caches. Weak references are cleared at the next GC once nothing strong refers to the object, as used by `WeakHashMap` keys and canonicalising maps. Phantom references always return `null` from `get()` and are enqueued after the object becomes unreachable, so you can run cleanup safely (`Cleaner`). All non-strong types can be registered with a `ReferenceQueue` to learn when they were cleared.

Q27. Why are `SoftReference` caches discouraged?

They clear unpredictably and in bulk under memory pressure, causing a stampede of reloads when the system is already struggling, and they keep the heap full, so GC runs longer and more often. Their sizing depends on `SoftRefLRUPolicyMSPerMB` and free heap, not on what is valuable. A bounded cache with explicit size and TTL (Caffeine) is predictable and observable.

Q28. How does `WeakHashMap` work, and what is its classic misuse?

Keys are held by weak references. When a key is no longer strongly reachable elsewhere, the GC clears it, the entry's reference is enqueued, and the map removes stale entries on its next operation. The misuse is a value that strongly refers to its own key (directly or indirectly): the key is then always reachable and never removed. Another one is using `String` literals as keys, since interned literals are effectively permanent.

Q29. What is `Cleaner` and why replace `finalize()`?

`java.lang.ref.Cleaner` (Java 9) runs a registered action on a cleaner thread after the object becomes phantom reachable. Unlike `finalize`, it cannot resurrect the object, costs no extra GC cycle, and its action is explicit. The action must not capture the object itself (or it never becomes unreachable). Use it only as a safety net behind an explicit `close()`. `finalize` was deprecated for removal by JEP 421.

Q30. Can Java have memory leaks? Give examples.

Yes: objects that stay reachable but are no longer needed. Examples include static or long-lived collections that only grow, unbounded caches keyed by request data, `ThreadLocal`s on pooled threads never removed, listeners never deregistered, non-static inner classes or lambdas capturing a large outer object, classloader leaks on redeploy, a JPA persistence context growing through a long batch, and whole result sets loaded into lists. Leaks show as a rising live set after each full or old GC.

Q31. How does a `ThreadLocal` leak?

`ThreadLocalMap` entries hold the `ThreadLocal` key weakly but the **value strongly**, and the map lives as long as the thread. In a pool, threads live forever, so a value set and never removed stays: memory grows with each distinct value, and context such as a tenant id or user can bleed into the next task. If the value's class came from a web app loader, it also pins that loader. Always `remove()` in `finally`, or use a `TaskDecorator` that does it.

Q32. What is a classloader leak?

When anything reachable from outside an application's class loader (a thread it started, a JDBC driver registered in `DriverManager`, a `ThreadLocal` on a container thread, a JMX bean, a shutdown hook) references one of its classes or objects, the whole loader and all its classes cannot be unloaded. Each redeploy then adds another full copy of the app's metadata, until `OutOfMemoryError: Metaspace`. In MAT, look for several instances of the same loader class and follow the path to GC roots from the old one.

Q33. How do you take a heap dump, and what do you watch out for?

`jcmd <pid> GC.heap_dump /path/heap.hprof` (or `jmap -dump:live,format=b,file=... <pid>`), or automatically with `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=...`. Cautions: it pauses the JVM for seconds to minutes, the file is about as big as the used heap (check disk space, since a full disk is a second incident), it contains sensitive data (customer records, tokens) and must be treated as such, and with `live` it triggers a full GC first. Take it from an instance pulled out of traffic when possible.

Q34. Shallow vs retained heap?

Shallow heap is the memory of the object itself (header plus fields). Retained heap is the total memory that would be freed if that object were collected: itself plus everything reachable only through it. A `HashMap` has a tiny shallow size but may retain gigabytes. Leaks are found by retained size.

Q35. What is the dominator tree in MAT?

A tree where X is the parent of Y if every path from the GC roots to Y passes through X ("X dominates Y"). Sorting it by retained heap lists the objects responsible for the most memory, which is nearly always the leak, or the legitimately big structure. From a top entry, "Path to GC roots, excluding weak and soft references" shows who holds it.

Q36. How do you find the cause of a slow, growing heap without an OOM yet?

Watch the live set after old or full GCs in the GC log, or with metrics (`jvm_memory_used_bytes` after GC). Take two class histograms (`jcmd <pid> GC.class_histogram`) an hour apart and diff them to see which classes grow. Then take a heap dump and open the dominator tree. JFR's Old Object Sample event (`jdk.OldObjectSample`) can also show where long-lived objects were allocated, with stack traces, at low overhead.

Q37. How do you take and read a thread dump?

`jcmd <pid> Thread.print` (or `jstack -l <pid>`, or `kill -3`). For each thread: name, state (`RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`), stack, the locks it holds (`- locked <0x..>`) and the lock it waits for (`- waiting to lock <0x..>`, or `parking to wait for`). The JVM appends any Java-level deadlock it detects. Take three dumps a few seconds apart and compare: same frame every time means stuck; changing frames means busy. Group threads by name to spot an exhausted pool.

Q38. How do you find the thread causing high CPU?

`top -H -p <pid>` (or `ps -L`) gives per-thread CPU by native thread id. Convert the hottest TID to hex and find `nid=0x...` in a thread dump taken at the same time, which gives you the Java stack. If the hot threads are GC threads, the problem is memory pressure. For a fuller picture, a 30-second async-profiler CPU flame graph shows exactly which methods burn CPU.

Q39. How do you detect a deadlock in production?

A thread dump reports "Found one Java-level deadlock" with the cycle of threads and locks (monitors and `ReentrantLock`s). `ThreadMXBean.findDeadlockedThreads()` can be polled by a health check. Symptoms are threads in `BLOCKED` or `WAITING` on each other's locks with zero CPU, and requests timing out. Database deadlocks are different: PostgreSQL detects them and aborts one transaction with SQLSTATE `40P01`, which you see as an exception in the logs.

Q40. What is JFR, and when would you use it instead of a profiler?

Java Flight Recorder is a built-in, low-overhead (about 1%) event recorder: CPU samples, allocation by stack, GC phases, locks, parking, I/O, exceptions, safepoints, virtual-thread pinning. It is safe to run continuously in production with a rolling buffer, then dump the last hour after an incident (`jcmd <pid> JFR.dump`). Use it as the default production diagnostic, and reach for async-profiler when you need native frames, wall-clock views or flame graphs without safepoint bias.

Q41. What is safepoint bias, and how does async-profiler avoid it?

Profilers based on `Thread.getStackTrace` or JVMTI sampling can only see threads at safepoint polls, so time gets attributed to the nearest poll rather than the code that actually burned it, which distorts results. async-profiler samples using `perf_events` or timers plus HotSpot's `AsyncGetCallTrace`, which can walk a Java stack at any instruction. That gives accurate attribution, including native and kernel frames.

Q42. What modes of async-profiler do you use for which problem?

`cpu` for high CPU, `alloc` for allocation hotspots (which code creates the garbage behind frequent GCs), `lock` for contention, and `wall` for slow requests with low CPU (it samples all threads regardless of state, so waiting on a pool, socket or lock shows up). Output flame graphs with `-f out.html`. Wall mode with `-t` (per thread) is the fastest way to explain "slow but idle".

Q43. How does the JVM behave in a container?

Since JDK 10 and 8u191, `UseContainerSupport` makes the JVM read cgroup memory and CPU limits; cgroup v2 support came in JDK 15 (backported to 11.0.16 and 8u372). Without `-Xmx`, max heap defaults to 25% of the container limit, usually too little. `availableProcessors()` follows the CPU quota and sizes GC threads and the common pool. Set `-XX:MaxRAMPercentage` to about 60–75 and leave headroom for native memory.

Q44. A pod is OOMKilled but there is no `OutOfMemoryError` in the logs. Why?

The kernel's cgroup OOM killer terminated the process (exit 137, SIGKILL) because the **process's total memory** exceeded the container limit. The Java heap never filled, so no Java error and no heap dump. The culprit is native memory (metaspace, thread stacks, direct buffers, GC structures, native libraries) or a heap set too close to the limit. Enable Native Memory Tracking (`-XX:NativeMemoryTracking=summary`, `jcmd <pid> VM.native_memory summary`), lower `MaxRAMPercentage`, and cap metaspace and direct memory.

Q45. How do you approach a production performance incident?

Stabilise first (shift traffic, scale out, roll back a recent deploy), but capture evidence before restarting: three thread dumps, a JFR dump, the GC log, and a heap dump if memory is the symptom. Then classify the symptom: high CPU (which threads), heap growth or OOM (dominator tree), slow but idle (thread dumps and wall profile: pools, locks, downstreams), or long pauses (GC log). Fix the cause, verify with the same metrics that alerted, and write a blameless postmortem with an alert or guard that catches it earlier next time.

Q46. Requests are slow but CPU is low. What do you look for?

Waiting. A thread dump will show most request threads parked in one place: `HikariPool.getConnection` (pool exhausted by slow queries or transactions held across remote calls), a socket read to one downstream (no or long timeouts), `BLOCKED` on one monitor (a global lock or a `synchronized` cache), or a full pool queue. Also check GC pauses and CPU throttling in containers. Fixes are timeouts, narrower transactions, bulkheads, query and index work, and removing global locks.

Q47. The GC log shows frequent full GCs that reclaim very little. What does it mean and what do you do?

The live set nearly fills the heap. Either there is a leak (the live set has been rising over time) or the heap is too small for the legitimate working set. An OOM or a "GC death spiral" with CPU in GC threads is close. Take a heap dump now, open the dominator tree, and decide: fix the retention if it is a leak, or reduce data held at once (streaming, chunking, bounded caches), or raise the heap if the working set really is that big.

Q48. How would you reduce GC pressure in a hot code path?

Measure allocation first (JFR allocation view or async-profiler `alloc`). Then remove unnecessary allocation: avoid boxing in hot loops (primitive collections and streams), presize collections and `StringBuilder`s, avoid building intermediate lists when streaming would do, reuse buffers where safe, avoid regex compilation per call, avoid logging string building when the level is off, and stream large results instead of materialising them. Escape analysis already removes many short-lived allocations, so trust the profile, not intuition.

Q49. How does run-service protect itself from running out of memory?

Admission control plus containment. `RunWorker` takes another run only while at least `runs.min-free-heap-percent` (default 25%) of the heap is free, and each instance runs a fixed number of slots (`runs.workers`). A run that still hits `OutOfMemoryError` ends `FAILED` with category `RESOURCES` and a clear reason. In deployment, `-XX:+ExitOnOutOfMemoryError` exits a JVM that ran out, systemd restarts it, and the database heartbeat re-queues its runs. A Prometheus alert fires above 90% heap. Scaling is by adding instances: a bigger run gets a larger run service, and more runs get more instances.

Q50. How do you process a file far larger than the heap?

Never hold it all. Stream rows (a JDBC fetch size inside a transaction, a streaming CSV parser, `SXSSFWorkbook` for Excel), process in bounded chunks, and push heavy set operations into an engine that can spill to disk. In RefinePoint, matching runs in in-process DuckDB with a `memory_limit` and a `temp_directory`, so a 5-million-row reconciliation spills to disk instead of failing, and `ExportWriter` keeps only a 2,000-row fetch window and a 500-row Excel window in memory however large the file is. Remember that the engine's native memory adds to the process footprint beyond `-Xmx`.

Q51. Why does PostgreSQL's JDBC driver load the whole result set unless you are careful?

By default pgjdbc fetches all rows into memory when the statement executes. It streams with a cursor only if a fetch size is set **and** autocommit is off (inside a transaction) **and** the result set is forward-only. Otherwise `setFetchSize` is silently ignored. Our `ExportWriter` does exactly this: it sets a fetch size of 2,000 on a forward-only statement inside a read-only `REPEATABLE_READ` transaction (which also gives the export a consistent snapshot), and handles rows one at a time in a `RowCallbackHandler`, never collecting them into a list.

Q52. When would you switch run-service from Serial GC, and how would you prove it helped?

When GC logs show old-generation (full) collections on the 2.5 GB heap taking seconds, or GC CPU becoming significant during large runs. Run the same large reconciliation under Serial, Parallel and G1 with GC logging, and compare total run time, longest pause, GC CPU share and peak RSS. Parallel likely wins on total run time (batch throughput), G1 on worst pause. Since runs are background work, total time and memory footprint matter more than pause time. Change one variable at a time and keep the logs as the evidence.

Q53. What JVM metrics would you alert on?

Heap used after GC relative to max (a sustained high value, as in our 90% heap alert), GC pause time percentiles and GC time share, old-generation or live-set growth rate, metaspace usage, thread count (a leak), direct buffer usage, and process RSS against the container limit. Pair them with application signals (latency percentiles, pool saturation, queue depth), because JVM metrics explain an incident while application metrics detect it.

Q54. What is the difference between throughput and latency in GC tuning?

Throughput is the share of time spent running application code rather than collecting (Parallel GC optimises it). Latency is the length of individual pauses (G1 targets it, ZGC minimises it). They trade off against each other and against footprint: smaller pauses cost more barrier work or more heap headroom. A batch job wants throughput, a trading or interactive API wants low latency, and many small services on one host want low footprint. Name the goal before tuning.